



Strathmore University

@iLabAfrica Centre

Applied Machine Learning

From First Principles to Production Systems

Javier Serrano
Emmanuel Lee Bundi

Master in Data Science and Analytics
DSA 8401

Nairobi, 2026

Applied Machine Learning: From First Principles to Production Systems in the African Fintech Ecosystem

Javier Serrano & Emmanuel Lee Bundi
iLabAfrica — Strathmore University, Nairobi, Kenya

Lecture notes for DSA 8401 — Applied Machine Learning,
Master in Data Science and Analytics.

© 2026 The authors. All rights reserved.
Draft edition — for enrolled students. Not for redistribution.

Typeset with L^AT_EX using the book class.
Figures produced with TikZ; code listings with listings.

Contact: jgarcia@strathmore.edu

Version 0.1 — September 14, 2026

Contents

Preface	xiii
---------	------

I Foundations and Classical Machine Learning	1
---	----------

1 Introduction to Machine Learning	3
1.1 Why This Chapter Comes First	4
1.2 What Is Machine Learning?	5
1.2.1 An operational definition	5
1.2.2 ML inverts the direction of programming	6
1.2.3 When ML earns its keep — and when it does not . . .	7
1.2.4 The formal core: empirical risk minimization	7
1.2.5 The Bayes-optimal reference	10
1.2.6 Notation for the rest of the book	11
1.3 The Machine-Learning Landscape	11
1.3.1 The four learning paradigms	11
1.3.2 Two orthogonal operational axes	13
1.3.3 No free lunch, and what it licenses	15
1.4 Why ML Systems Fail in Production	16
1.4.1 The small green box	16
1.4.2 The notebook-to-production gap, localized	17
1.5 The End-to-End ML Lifecycle	18
1.5.1 From CRISP-DM to a monitoring-driven loop	18
1.5.2 It is a loop, not a waterfall	19
1.6 Framing the Problem and Choosing the Metric	20
1.6.1 Step 1 is where projects are won	20
1.6.2 Regression metrics: how the business pays for errors	21
1.6.3 Classification under imbalance: why accuracy lies . .	21
1.7 A First Look at the Data	24
1.7.1 The raw material	24
1.7.2 A feature dictionary, declared up front	24
1.7.3 What to interrogate, and why	25
1.8 Data, Splits, and Leakage	27

1.8.1	Splits: the discipline that protects your estimate of $R(f)$	27
1.8.2	Data leakage: the silent killer of fintech models	28
1.9	Baselines, Evaluation, and Generalization	30
1.9.1	Why generalization is hard: the bias–variance decomposition	30
1.9.2	Always ship a baseline first	31
1.9.3	Cross-validation: cheap insurance on the estimate . .	31
1.10	Presenting and Deciding	33
1.10.1	The decision memo	33
1.10.2	Champion, challenger, and shadow mode	33
1.11	Fairness, Explainability, and Regulation	34
1.11.1	Making “fair” precise enough to test	34
1.11.2	Explainability as a deployment requirement	35
1.12	From Model to System: MLOps and the Agentic Outlook . .	35
1.12.1	The MLOps spine of this course	36
1.12.2	Where LLMs and agents fit — and where they do not	37
1.13	Chapter Summary	38
	Exercises	41
	Key Terms	42
	Further Reading	42
2	Data Preparation and Feature Engineering	45
2.1	Introduction: From Raw Data to Model-Ready Data	46
2.1.1	The Six Dimensions of Data Quality	47
2.1.2	The Unit of Prediction	48
2.2	Missing Data	48
2.2.1	Rubin’s Taxonomy of Missingness	49
2.2.2	Diagnosis Before Treatment	50
2.2.3	Imputation I: Simple Statistics with Indicators	50
2.2.4	Imputation II: Model-Based Methods	51
2.2.5	A Decision Framework	52
2.3	Outliers and Duplicates	53
2.3.1	Detecting Outliers	53
2.3.2	Treating Outliers, and When Not To	54
2.3.3	Duplicates and Entity Resolution	54
2.4	Scaling and Transformations	55
2.4.1	Why Scale, and With What	55
2.4.2	Taming Skew: Log and Power Transforms	56
2.5	Encoding Categorical Variables	57
2.5.1	Low Cardinality: One-Hot and Ordinal	57
2.5.2	High Cardinality I: Frequency Encoding and the Hashing Trick	58
2.5.3	High Cardinality II: Target Encoding	58
2.6	Feature Engineering	59

2.6.1	Engineered versus Learned Features	59
2.6.2	Domain Features for Mobile-Money Credit Scoring	60
2.6.3	Datetime Features and Cyclical Encoding	61
2.6.4	Interactions, Binning, and Crosses	61
2.6.5	What Makes a Feature Good?	62
2.7	Data Leakage	63
2.7.1	Definition and Why It Is Dangerous	63
2.7.2	Six Common Causes	64
2.7.3	Temporal and Group-Aware Splits	64
2.7.4	A Detection Protocol	65
2.7.5	The Structural Antidote: Pipelines	66
2.8	From Pipelines to Feature Platforms	67
	Assignment 2 (graded, 20 marks)	68
	Exercises	69
	Key Terms	70
	Further Reading	70
3	Classical Machine Learning and Rigorous Evaluation	71
3.1	Introduction	72
3.2	The Classical Toolbox	73
3.2.1	Why classical models still run production tabular ML	73
3.2.2	Logistic regression	73
3.2.3	Softmax regression for multiclass problems	75
3.2.4	Maximum-margin classification: support vector machines	75
3.2.5	Neighbours and Bayes: two more baselines	76
3.2.6	Choosing among the classical learners	77
3.3	Beyond Accuracy: Metrics that Match the Problem	78
3.3.1	The accuracy trap	78
3.3.2	The confusion matrix	78
3.3.3	Precision, recall, and the F_β family	79
3.3.4	From metrics to money: cost-sensitive evaluation	79
3.4	Ranking Quality: ROC and Precision–Recall Analysis	80
3.4.1	Scores, not labels: the threshold as a free parameter	80
3.4.2	The ROC curve and AUC	81
3.4.3	Precision–recall curves: the imbalance-honest view	82
3.5	Calibration: Do the Probabilities Mean Anything?	82
3.6	Cross-Validation and Honest Uncertainty	84
3.6.1	Why one split is not enough	84
3.6.2	Cross-validating the pipeline	85
3.6.3	Nested cross-validation for model selection	86
3.6.4	Comparing models honestly	86
3.7	Learning from Imbalanced Data	87
3.7.1	Three families of remedies	87

3.7.2	SMOTE: synthesising minority examples	88
3.7.3	The cardinal sin: resampling before splitting	89
3.8	Case Study: Evaluation Design for the Thin-File Credit Scorer	90
3.9	Chapter Summary	91
4	Decision Trees, Ensembles and Hyperparameter Tuning	95
4.1	Introduction	96
4.2	Decision Trees: The Building Block	98
4.2.1	A tree is a sequence of questions	98
4.2.2	The same object, drawn as a partition	98
4.2.3	How a tree is grown	99
4.2.4	Controlling complexity: stopping and pruning	100
4.2.5	The flaw that motivates the rest of the chapter	101
4.3	Why Ensembles Work	102
4.3.1	The wisdom of independent crowds	102
4.3.2	The variance of an average	103
4.4	Voting Classifiers	104
4.5	Bagging and Random Forests	106
4.5.1	Bootstrap aggregating	106
4.5.2	Out-of-bag evaluation	106
4.5.3	Random Forests: decorrelating the trees	107
4.6	Boosting	109
4.6.1	The idea, before the algebra	109
4.6.2	A worked trace	110
4.6.3	From residuals to gradients	111
4.6.4	AdaBoost, the historical special case	112
4.6.5	Shrinkage and early stopping	113
4.7	Regularised Gradient Boosting: XGBoost, LightGBM, CatBoost	113
4.7.1	Regularisation inside the objective	114
4.7.2	The library landscape	115
4.7.3	Monotonic constraints and regulated deployment	115
4.8	Stacked Generalisation	117
4.9	Hyperparameter Optimisation	118
4.9.1	What is a hyperparameter, and why is it different?	118
4.9.2	A ladder of strategies	119
4.9.3	Grid search versus random search	119
4.9.4	Learning from previous trials: Bayesian search and TPE	121
4.9.5	Spending less per trial: successive halving and Hyperband	122
4.9.6	Designing the search space	123
4.9.7	Honest evaluation of a tuned model	123
4.10	Case Study: Round Three of the Credit Scorer	125
	Exercises	127
	Key Terms	128

Further Reading	129
5 Unsupervised Learning and Dimensionality Reduction	131
5.1 Introduction: Learning Without Labels	132
5.2 The Unsupervised Landscape	134
5.3 Clustering with K -Means	136
5.3.1 The objective	136
5.3.2 Choosing k	137
5.3.3 Implementation	139
5.4 Density-Based Clustering: DBSCAN	140
5.5 Gaussian Mixtures and EM	142
5.5.1 The model	142
5.5.2 Fitting by Expectation–Maximisation	143
5.5.3 Selecting k by information criteria	145
5.5.4 Mixtures as anomaly detectors	146
5.6 The Curse of Dimensionality	147
5.7 Principal Component Analysis	148
5.7.1 Variance maximisation	148
5.7.2 Reconstruction-error minimisation	149
5.7.3 Computation and the choice of d	149
5.7.4 Variants you will actually need	151
5.8 Case Study: The Segmentation Pipeline in Production	152
5.8.1 Profiling: turning clusters into personas	152
5.8.2 Stability: a segment that moves is not a segment	153
Exercises	156
Key Terms	157
Further Reading	158
 II Deep Learning and Generative AI	 159
6 Deep Learning Fundamentals	161
6.1 Introduction: from engineered features to learned representations	162
6.1.1 When a network is the right instrument	164
6.2 The artificial neuron and the multi-layer perceptron	165
6.3 Activation functions	168
6.4 The training objective	170
6.4.1 Binary cross-entropy from the Bernoulli likelihood	171
6.4.2 Choosing the output layer and loss together	172
6.5 Gradient descent	173
6.5.1 Why the learning rate dominates everything else	174
6.6 Backpropagation	174
6.7 Optimisers beyond plain gradient descent	177

6.7.1	Momentum	177
6.7.2	Adam	178
6.7.3	Learning-rate schedules	179
6.8	Pathologies of deep training	179
6.9	Initialisation	180
6.10	Normalisation layers	181
6.11	Regularisation	182
6.11.1	Dropout	182
6.11.2	Early stopping and weight decay	184
6.12	Input scaling and conditioning	184
6.13	Entity embeddings	185
6.14	Implementation in TensorFlow and Keras	187
6.14.1	The training loop, written out	187
6.14.2	The baseline network	188
6.14.3	The embedding challenger	189
6.14.4	Hyperparameter search and debugging	190
6.15	Evaluation, calibration and governance	190
6.15.1	Ranking versus calibration	191
6.15.2	Explanation	191
6.15.3	Fairness and segment-level performance	192
	Exercises	194
	Key Terms	197
	Further Reading	198
7	Computer Vision with CNNs	199
7.1	Introduction	200
7.1.1	Four tasks, one backbone	202
7.2	Why Convolution?	203
7.2.1	Why a dense network fails on images	203
7.2.2	The convolution operator	205
7.2.3	Pooling and global average pooling	209
7.2.4	Receptive fields and the case for depth	210
7.2.5	The data-efficiency argument	212
7.3	Modern Architectures: ResNet and EfficientNet	212
7.3.1	Residual networks	213
7.3.2	EfficientNet and compound scaling	216
7.3.3	Choosing a backbone	218
7.4	Transfer Learning	218
7.4.1	Two regimes	219
7.4.2	The two-phase recipe	221
7.4.3	Implementation in TensorFlow and Keras	222
7.4.4	The batch-normalisation trap	224
7.4.5	Data augmentation as a statement about your domain	225
7.4.6	Reading the training curves	227

7.5	Evaluation, Explainability and Deployment	227
7.5.1	Evaluation under imbalance and asymmetric cost	227
7.5.2	Explaining a convolutional prediction	229
7.5.3	Fairness, governance and deployment	230
	Exercises	234
	Key Terms	236
	Further Reading	236
8	Sequence Models and Forecasting	239
8.1	Introduction: from images to sequences	240
8.2	The anatomy of a time series	242
8.2.1	The forecasting task, formally	243
8.2.2	Stationarity and autocorrelation	244
8.3	Evaluating a forecast without fooling yourself	245
8.3.1	Baselines that keep everyone honest	245
8.3.2	Backtesting, not random cross-validation	246
8.3.3	Metrics	247
8.3.4	The pinball loss, and why the business lives in the tail	247
8.4	Forecasting as supervised learning	248
8.4.1	Three ways to cover the horizon	249
8.5	Recurrent neural networks	251
8.5.1	Backpropagation through time, and why it fails	251
8.6	Gated cells: LSTM and GRU	253
8.6.1	The LSTM cell	253
8.6.2	The GRU cell	255
8.6.3	A forecaster we can actually train	256
8.6.4	From a point to a fan	256
8.7	Attention	258
8.7.1	The bottleneck	259
8.7.2	Attention as soft dictionary lookup	259
8.7.3	Self-attention, and what queries, keys and values are	260
8.7.4	Causal masking	261
8.7.5	Multiple heads	261
8.7.6	Reading an attention map	262
8.8	The Temporal Fusion Transformer	263
8.8.1	Gated residual networks	264
8.8.2	Variable selection	264
8.8.3	Interpretable attention, and the rest of the model	265
8.8.4	Choosing a model	267
8.9	The case study end to end	268
	Exercises	270
	Key Terms	273
	Further Reading	274

9	Transformers, LLMs and RAG	275
9.1	Introduction: the ceiling we hit in Chapter 8	276
9.2	Attention as a soft lookup	278
9.2.1	Queries, keys and values	279
9.2.2	Why divide by the square root of d_k	279
9.2.3	Causal masking	280
9.2.4	Reading attention maps, and the limits of doing so	280
9.3	Multiple heads and the price of context	281
9.3.1	Quadratic cost, and when it starts to hurt	282
9.4	Position, and the information attention discards	283
9.5	The Transformer block	284
9.6	Three architectural families	285
9.7	Tokenization: the real input layer	286
9.8	Pretraining, scaling and what emerges	288
9.8.1	Scaling laws	288
9.9	From base model to assistant	289
9.10	Decoding: how text actually comes out	290
9.11	Prompting, and the adaptation decision	291
9.12	The HuggingFace ecosystem	293
9.12.1	The Hub is a supply chain	293
9.13	Parameter-efficient fine-tuning	295
9.13.1	LoRA	296
9.13.2	QLoRA	298
9.13.3	What adaptation buys	299
9.14	Retrieval-augmented generation	300
9.14.1	Embeddings: meaning as geometry	300
9.14.2	Chunking, which quietly decides everything	302
9.14.3	The pipeline, and its two clocks	302
9.15	Evaluating a retrieval system	304
9.15.1	A bridge to agents	305
9.16	Governance, risk and the Kenyan context	305
9.17	Summary	306
	Exercises	307
	Key Terms	309
	Further Reading	310

III MLOps and Production Systems 311

10	MLOps: packaging, serving and CI/CD	313
10.1	Introduction: from notebook to production system	314
10.1.1	The MLOps lifecycle and maturity levels	316
10.2	Packaging the model artefact	317
10.2.1	What exactly is “the model”?	318

10.2.2	Training-serving skew	318
10.2.3	Serialisation formats	320
10.2.4	The threshold belongs in the package	321
10.2.5	Metadata and environment	322
10.3	Serving the model: design and API	323
10.3.1	Four ways to serve predictions	323
10.3.2	Latency, capacity and model placement	324
10.3.3	The FastAPI service	327
10.3.4	Operating the service: health, errors and tests	331
10.3.5	Serving language-model and agent workloads	333
10.4	Tracking and the registry	334
10.4.1	Experiment tracking with MLflow	335
10.4.2	Tracing language-model agents	339
10.4.3	The model registry	341
10.5	Testing machine learning systems	343
10.5.1	Why ML testing is different	343
10.5.2	Unit tests and data contracts	344
10.5.3	Behavioural tests	344
10.5.4	Statistically honest gates	346
10.5.5	Gating language-model agents	348
10.6	Shipping, delivery and governance	348
10.6.1	Containerising with Docker	348
10.6.2	Continuous integration, delivery and training	352
10.6.3	The three workflows	353
10.6.4	Deployment strategies and the whole pipeline	355
10.6.5	Governance and auditability	356
10.7	Conclusions	357
	Exercises	358
	Key terms	360
	Further reading	360
11	Model Monitoring, Drift and ML System Design	345
11.1	Introduction	345
11.2	Core Theory	345
11.3	Implementation	345
	Appendices	349
A	Capstone Project	349

B Solutions to Exercises	351
Chapter 1	351
Chapter 2	358
Chapter 3	363
Chapter 7 — Computer Vision with CNNs	388
Chapter 8 solutions	395
B.1 Chapter 9: Transformers, LLMs and retrieval	401
B.2 Chapter 10: MLOps, packaging, serving and CI/CD	406

Preface

This book collects the lecture notes of **DSA 8401 — Applied Machine Learning**, taught in the Master in Data Science and Analytics at iLabAfrica, Strathmore University. It is written for graduate students who already program and know basic statistics, and who want to go beyond training models in a notebook: the goal throughout is *production-grade* machine learning — data pipelines, rigorous evaluation, deployment, and monitoring.

Three commitments shape every chapter:

1. **An end-to-end view.** Models are treated as components of systems. MLOps (Docker, FastAPI, MLflow, CI/CD) is not an afterthought but the destination of the whole arc.
2. **African context as pedagogy.** Worked examples are anchored in the African technology and financial ecosystem: thin-file mobile-money credit scoring (the running thread of the course), trade-corridor underwriting, clinical pathway modelling, and streaming fraud detection. This is a deliberate instructional choice, not decoration.
3. **Modern architectures.** Transformers, LLMs, RAG pipelines, and an outlook on agentic systems are integrated where they matter for applied work.

How this book maps to the course

Each chapter corresponds to one teaching week (three hours: a 1.5-hour lecture plus a 1.5-hour practical lab). The two MLOps weeks are consolidated into a single chapter, and the capstone project brief lives in Appendix [A](#).

Chapter	Topic	Course week
1	End-to-End Machine Learning	1
2	Data Preparation & Feature Engineering	2
3	Classical ML & Rigorous Evaluation	3
4	Ensemble Methods & Hyperparameter Tuning	4
5	Unsupervised Learning & Dim. Reduction	5
6	Deep Learning Fundamentals	6
7	Computer Vision with CNNs	7
8	Sequence Models & Time Series	8
9	Transformers, LLMs & RAG	9
10	MLOps: Serving, Tracking & CI/CD	11–12
11	Monitoring, Drift & System Design	13
A	Capstone Project	14–15

Prerequisites and companion material

We assume the Data Mining unit, working Python, and comfort with linear algebra and probability at the level of a first graduate course. The primary textbook companion is [Géron \(2022\)](#); [Bishop and Bishop \(2024\)](#) and [Huyen \(2022\)](#) support the deep-learning and MLOps parts respectively. Each chapter closes with a lab section that mirrors the Jupyter notebook used in the practical session.

Javier Serrano & Emmanuel Lee Bundi
Nairobi, 2026

Part I

Foundations and Classical Machine Learning

Part II

**Deep Learning and Generative
AI**

CHAPTER 8

Sequence Models and Time-Series Forecasting

Course week 8

Learning Objectives

After working through this chapter you will be able to:

- Frame a forecasting problem as supervised learning through sliding windows, and classify every input feature into one of the three covariate families: static, past-observed and known-future.
- Evaluate a temporal model without leakage, using rolling-origin backtesting, scale-free point metrics and the quantile (pinball) loss, and explain why random k -fold cross-validation inflates every number it produces.
- Explain why a plain recurrent network forgets, and derive how the additive memory path in an LSTM or GRU cell repairs it.
- Implement a direct multi-horizon forecaster in `tf.keras`, with quantile heads that produce a forecast fan rather than a point estimate.
- Derive scaled dot-product attention from the seq2seq bottleneck, explain causal masking and multiple heads, and state what attention buys and what it costs relative to recurrence.
- Describe the Temporal Fusion Transformer as a composition of gating, variable selection, a recurrent local encoder and interpretable attention, and say when its extra cost is justified.
- Design a production forecasting system for agent liquidity in a Kenyan mobile-money network, including its fairness, data-protection and drift-monitoring obligations.

8.1 Introduction: from images to sequences

Chapter 7 specialised the architecture of Chapter 6 for data with *spatial* structure. Pixels that sit near one another are related, and a convolutional layer encodes that relationship directly, by applying the same small detector at every position. The gain was not subtlety but economy: a constraint that happens to be true of photographs bought generalisation that no volume of extra data would have bought.

This chapter performs the same manoeuvre along a different axis. Our data are now ordered in *time*, and the ordering carries information. Yesterday's transaction volume is related to today's in a way that a randomly chosen day from last year is not; a repayment trajectory means something that the average of its points does not. The architecture that encodes this is the recurrent layer, which applies the same small update at every time

step and carries a state forward. The parallel with convolution is exact, and worth holding on to: convolution is weight sharing across *space*, recurrence is weight sharing across *time*.

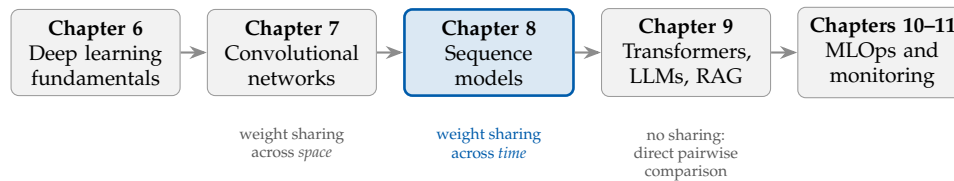


Figure 8.1: Part II specialises the architecture, never the training procedure. This chapter adds a notion of order; the next removes the recurrence and keeps only the attention mechanism we build in Section 8.7.

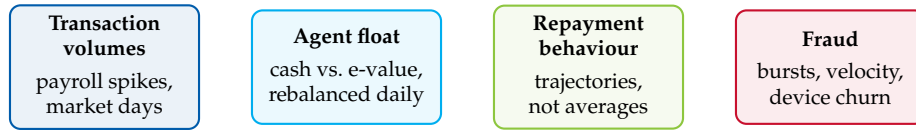
There is a second reason to treat sequences carefully, and it has nothing to do with architecture. Almost every mistake that matters in forecasting is a mistake about *time*, not about models. Fitting a scaler on the whole dataset, shuffling before splitting, using a feature that will not exist at the moment of prediction, or comparing against no baseline at all: each of these produces a validation number that is both excellent and meaningless. A good deal of this chapter is therefore about evaluation, and it comes before the architectures rather than after them.

Agent Liquidity, Round Seven

Our Nairobi lender does not only score borrowers. It also runs, with a partner mobile-money operator, a network of roughly five thousand *agents*: the kiosks, pharmacies and market stalls where customers deposit and withdraw cash.

Every agent holds two inventories. Physical **cash** funds customer withdrawals; electronic **float** funds customer deposits. When either runs out, the agent turns customers away, which is called a *float-out*. Each agent is rebalanced once or twice a day by a runner whose route is planned the evening before, so the operational question is fixed in shape: at 18:00 each day, for every agent, how much cash and float will be needed over the next twenty-four hours?

The costs are asymmetric in the manner we have grown used to. A float-out costs lost commission, a lost customer and reputational damage in a neighbourhood where the agent is the network's public face. Excess float costs financing and raises the risk of robbery. The lender's interest is direct: it advances float to agents against future commissions, so an under-provisioned agent is also a credit exposure that will not perform. The data are a stream. Each agent reports hourly deposit and withdrawal volumes, cash and float balances, and downtime minutes. This is our first dataset in which the row order is the point.



A static tabular snapshot throws the ordering away. Sequence models consume it.

Figure 8.2: Four streams inside one platform. Each was flattened into summary statistics in Chapters 2 and 3; each contains structure that summary statistics cannot represent.

8.2 The anatomy of a time series

Figure 8.3 shows two weeks of one agent’s hourly volume. Three structures are visible at once, and they are worth naming before any model is fitted, because a model that is not told about them has to rediscover them from data.

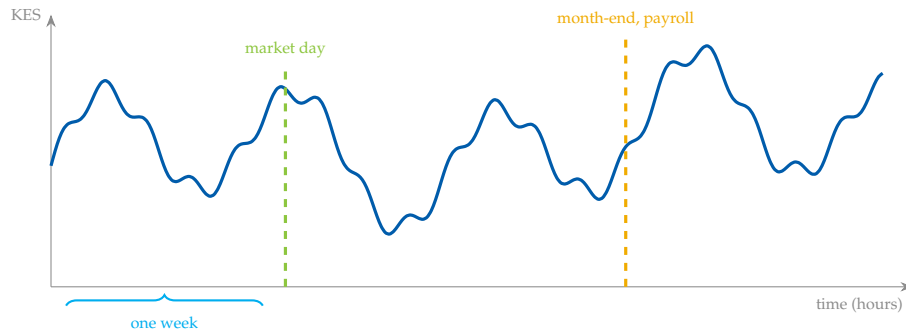


Figure 8.3: Two weeks of hourly transaction volume for one agent. A slow trend, nested daily and weekly cycles, and isolated calendar events are all present simultaneously.

The classical description separates a series into additive components,

$$y_t = T_t + S_t + E_t + \varepsilon_t, \quad (8.1)$$

where T_t is a slow *trend* (the agent network grows), S_t collects *seasonalities* (hour of day, day of week, and often month of year, stacked on top of one another), E_t carries isolated *events* (Eid, Christmas, school-fee deadlines, elections) and ε_t is irreducible noise. Figure 8.4 draws the separation.

A neural forecaster does not perform this decomposition explicitly, and does not need to. The reason to know it is practical. Seasonality and events are generated by a calendar we already possess, so withholding the calendar forces the model to infer, from noisy volume data alone, a fact we

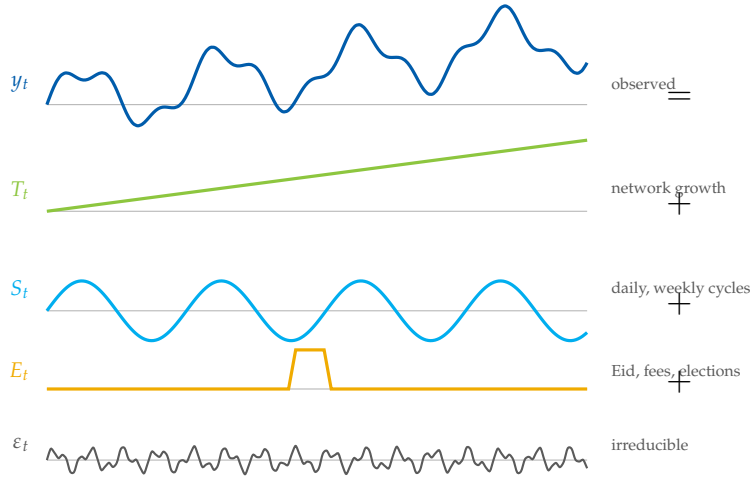


Figure 8.4: The decomposition (8.1). A deep model learns T , S and E jointly and is never told they exist, which is precisely why it should still be *fed the calendar* as explicit features.

could simply have supplied. Feeding day-of-week, holiday and payroll indicators is not feature engineering in the sense of Chapter 2; it is refusing to make the model guess at something known.

8.2.1 The forecasting task, formally

Definition 8.1 (Multi-horizon forecasting). Let y_t be a scalar target observed at regular intervals. Given a *forecast origin* t , a *lookback* L and a *horizon* H , a forecaster is a map

$$\hat{y}_{t+1:t+H} = f_{\theta} \left(\underbrace{y_{t-L+1:t}}_{\text{target history}}, \underbrace{\mathbf{z}_{t-L+1:t}}_{\text{past-observed}}, \underbrace{\mathbf{x}_{t-L+1:t+H}}_{\text{known-future}}, \underbrace{\mathbf{s}}_{\text{static}} \right). \quad (8.2)$$

The three covariate families in (8.2) are distinguished by *when the information becomes available*, not by what it describes. *Static* covariates $\mathbf{s}$ do not vary with time: the agent’s identity, county, urban or rural setting, tier. *Past-observed* covariates $\mathbf{z}_t$ are measured, and therefore exist only up to the origin: realised volumes, float balances, downtime. *Known-future* covariates $\mathbf{x}_t$ are known in advance for the whole horizon because they come from a calendar or a plan: hour of day, day of week, public holidays, school terms, the payroll dates of the large employers in the neighbourhood. Figure 8.5 draws the distinction against the forecast origin.

This looks like bookkeeping and turns out to be the single most useful piece of discipline in the chapter. It is the taxonomy the Temporal Fusion Transformer of Section 8.8 is built around, and it is also the checklist that catches leakage: a feature that cannot be produced at 18:00 for the following day does not belong in $\mathbf{x}$, however predictive it looks in a notebook.

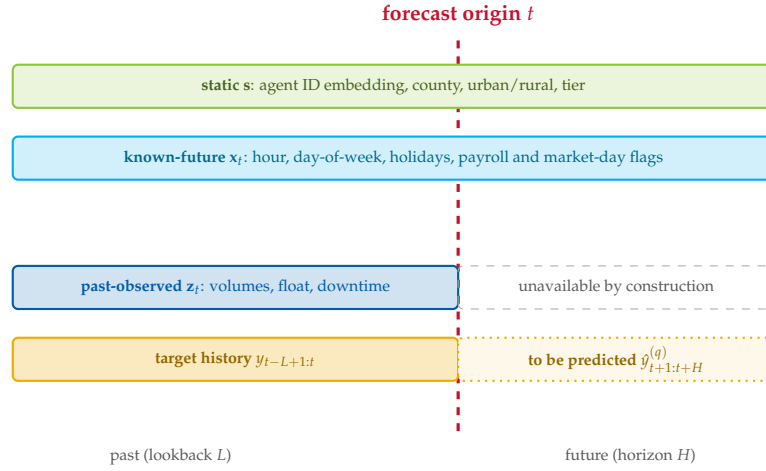


Figure 8.5: The taxonomy of Definition 8.1, drawn against the forecast origin. If a feature crosses the red line without genuinely being knowable in advance, the model has a leak rather than a signal.

The two window lengths come from different places, and confusing them is a common error. The horizon H is an *operational* fact: the runner's route is planned the evening before, so $H = 24$ hours. It is not a hyperparameter to be tuned. The lookback L is a modelling choice, and the next subsection shows where to read it off.

8.2.2 Stationarity and autocorrelation

A series is *weakly stationary* if its mean, variance and autocovariance do not depend on t . Almost no business series is: trend and seasonality both violate the condition by construction. The classical ARIMA family responds by *differencing*, replacing y_t with $\Delta y_t = y_t - y_{t-1}$ or with a seasonal difference $\Delta_m y_t = y_t - y_{t-m}$, until the result looks stationary.

Deep models do not require stationarity, and differencing them is usually counterproductive, because it discards level information the network could use. What they do require is inputs of a sane scale. For KES volumes, which have heavy right tails and occasional zeros, the recipe used throughout this book is to apply the transformation $y_t \mapsto \log(1 + y_t)$, then to standardise each series on its own, using training-span statistics alone.

The *autocorrelation function* is the cheapest diagnostic tool available and the one that sets L . At lag k ,

$$\rho_k = \frac{\text{Cov}(y_t, y_{t-k})}{\text{Var}(y_t)}. \quad (8.3)$$

For hourly agent data, ρ_k decays smoothly and spikes at $k = 24$ and $k = 168$: the same hour yesterday and the same hour last week (Figure 8.6).

Those spikes tell you two things before you have trained anything. They identify the seasonal period m for the baseline of Section 8.3, and they tell you that a lookback shorter than 168 cannot see the weekly cycle, whatever architecture sits on top of it.

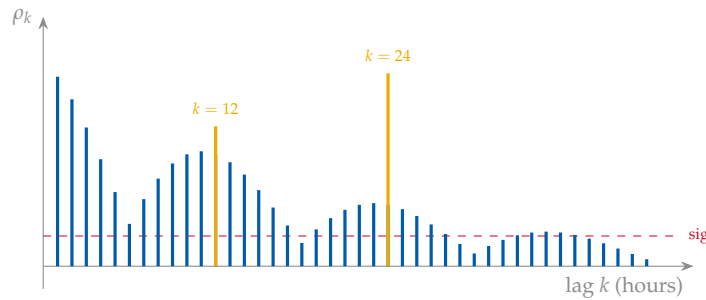


Figure 8.6: A stylised ACF for hourly agent volume: smooth decay plus periodic spikes. Read the seasonal period and the required lookback off this plot before choosing an architecture.

8.3 Evaluating a forecast without fooling yourself

We treat evaluation before models, deliberately. In forecasting, an architecture that is chosen against a broken protocol is worse than no model at all, because it carries the authority of a number.

8.3.1 Baselines that keep everyone honest

A forecast is only ever good *relative* to something. Four baselines cost almost nothing to compute and should always be present.

The **naïve** forecast is $\hat{y}_{t+h} = y_t$: tomorrow looks like today. It is surprisingly strong for random-walk-like series such as exchange rates. The **seasonal naïve** forecast is $\hat{y}_{t+h} = y_{t+h-m}$ for seasonal period m : this hour next Tuesday looks like this hour last Tuesday. For agent volumes it is the one to beat, and beating it is harder than it sounds. **Exponential smoothing** maintains a recursively updated weighted average, and **(S)ARIMA** fits a linear model to the differenced series; both remain regulator-friendly workhorses for low-frequency data. Finally, **gradient boosting on lag features**, the LightGBM of Chapter 4 fed lagged values and calendar dummies, is a ferocious baseline that many deep forecasters quietly fail to beat.

The unbeaten baseline

A large share of published claims that deep forecasting beats classical methods evaporate when the classical method is tuned properly. In a production review the first question is never which architecture you used. It is: by how much do you beat seasonal naïve, at which horizons, and on which segments? If the answer is that you do not, the correct engineering decision is to ship seasonal naïve, which has no training pipeline, no GPU bill and no drift surface, and to spend the saved effort on data.

8.3.2 Backtesting, not random cross-validation

The k -fold cross-validation of Chapter 3 assumed that examples are exchangeable. Time series are not. Shuffling rows before splitting places future observations in the training set, so the model is scored on its ability to interpolate between points it has already seen. Every metric improves and none of them means anything.

The correct protocol is *rolling-origin backtesting*, drawn on the right of Figure 8.7: train on $[1, t]$, test on $(t, t + H]$, advance the origin, and repeat. Each fold reproduces the deployment situation exactly, in which the model has seen the past and nothing else. scikit-learn's `TimeSeriesSplit` implements this directly.

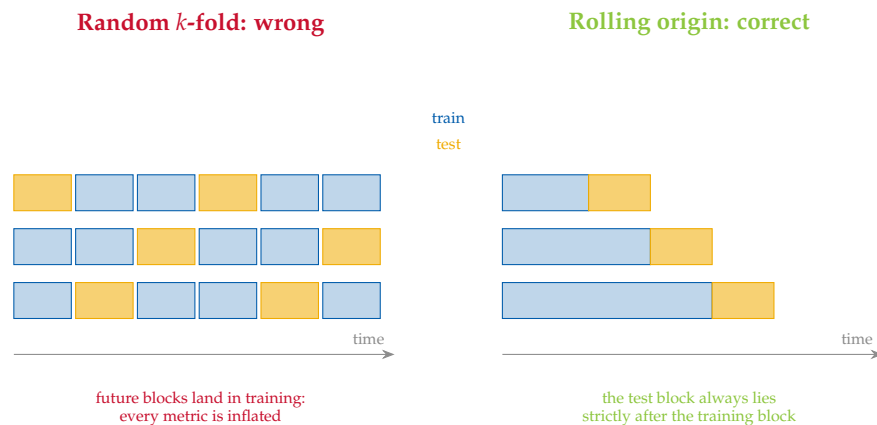


Figure 8.7: Random k -fold cross-validation against rolling-origin backtesting. The left-hand protocol is the most common leakage bug in industrial forecasting, and it always flatters the model.

Three further disciplines travel with it. Scalers and encoders are fitted *inside* each fold, on the training block only, exactly as in Chapter 2 but now in time rather than across rows. Windows are never allowed to straddle a fold boundary, since a single window spanning the split silently mixes train

and test. And one final block of data, later than every backtest fold, is held out untouched as an *out-of-time* test set for the last word before deployment.

8.3.3 Metrics

For point forecasts on test pairs $(y_i, \hat{y}_i)$ we use three quantities:

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{sMAPE} = \frac{200}{n} \sum_i \frac{|y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|}, \quad \text{MASE} = \frac{\text{MAE}}{\text{MAE}_{\text{naive}}}. \quad (8.4)$$

MAE is in KES and therefore not comparable between a market stall and a supermarket. sMAPE is scale-free but unstable when volumes approach zero, which they do every night. MASE solves both problems: it is scale-free by construction, and its unit interval has meaning, since $\text{MASE} < 1$ says exactly that you beat seasonal naïve.

Report metrics *per horizon step*. A model can be excellent at $h = 1$ and useless at $h = 24$, and it is $h = 24$ that pays for the runner's route.

8.3.4 The pinball loss, and why the business lives in the tail

A liquidity desk does not want the expected volume. It wants the level that will be exceeded only rarely, so that provisioning covers demand on all but the worst days. That is a request for a *quantile*, and quantiles have their own loss function.

Definition 8.2 (Pinball loss). For $q \in (0, 1)$ and error $e = y - \hat{y}^{(q)}$, the quantile or pinball loss is

$$L_q(y, \hat{y}^{(q)}) = \max(qe, (q-1)e) = \begin{cases} qe & e \geq 0 \text{ (under-forecast)}, \\ (q-1)e & e < 0 \text{ (over-forecast)}. \end{cases} \quad (8.5)$$

The asymmetry is the whole content of the definition, and it is drawn in Figure 8.8. At $q = 0.9$ an under-forecast is penalised nine times as heavily as an over-forecast of the same size, so the minimiser sits high. The next proposition is the reason the loss works, and it is the one place in this chapter where a short proof earns its keep.

Proposition 8.1 (The pinball loss recovers the quantile). *Let Y have distribution function F and finite mean. Then any minimiser of $g(a) = \mathbb{E}[L_q(Y, a)]$ over $a \in \mathbb{R}$ is a q -th quantile of Y .*

Proof. Write $g(a) = q \mathbb{E}[(Y - a)^+] + (1 - q) \mathbb{E}[(a - Y)^+]$, which is finite and convex in a . Differentiating under the expectation,

$$g'(a) = -q \Pr(Y > a) + (1 - q) \Pr(Y \leq a) = F(a) - q,$$

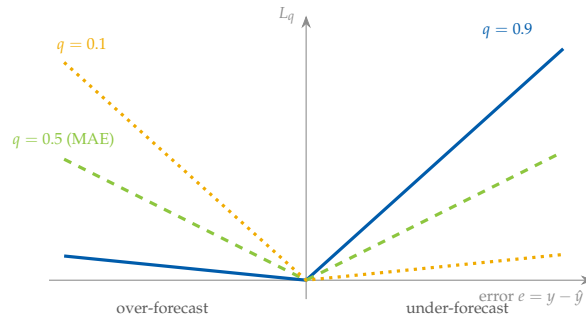


Figure 8.8: The asymmetry is the risk appetite. A steep right arm says that running out hurts far more than holding too much, which is exactly the agent-liquidity situation.

using $\Pr(Y > a) = 1 - F(a)$. Convexity means any a with $g'(a) = 0$ is a minimiser, and $F(a) = q$ is precisely the definition of a q -th quantile. When F jumps over q , the left and right derivatives straddle zero at the jump point, which is then the minimiser. $\square$

Two consequences are worth extracting. First, $q = 0.5$ gives $g'(a) = F(a) - 1/2$, so minimising MAE targets the *median* and not the mean: a model trained on MAE is systematically below the mean on right-skewed data such as transaction volumes, and that is correct behaviour rather than a bug. Second, training three heads at $q \in \{0.1, 0.5, 0.9\}$ produces a forecast interval directly, with no distributional assumption anywhere. We never assume the errors are Gaussian, which is fortunate, because they are not.

8.4 Forecasting as supervised learning

Nothing in Definition 8.1 yet looks like the $(\mathbf{X}, y)$ matrices of Part I. The bridge is the *sliding window*, and it is the single trick that lets every method in this book be applied to a series.

Cut the series into overlapping pairs: L consecutive observations become one input, the following H observations become its target, then advance by the *stride* and repeat (Figure 8.9). Each window is one training example, and forecasting becomes ordinary supervised learning.

The stride is usually one, so consecutive windows overlap in $L - 1$ of their positions. That feels wasteful and is not: the windows are different training examples because their targets differ, and the redundancy costs only compute.

Listing 8.1 shows the construction in Keras. Two details in it matter more than the rest. The temporal split happens *before* windowing, so that no window can span the boundary; and the scaler is fitted on the training

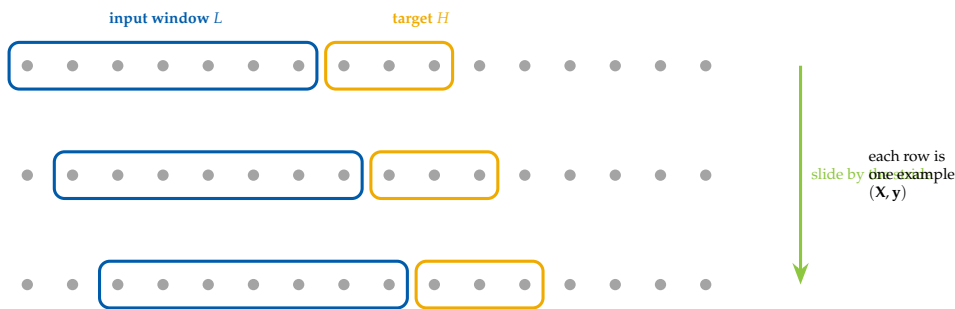


Figure 8.9: The framing trick behind all neural forecasting. The one non-negotiable condition is that the train, validation and test split is temporal, and that no window straddles a boundary.

span alone, which is the leakage discipline of Chapter 2 transposed into time.

```
import numpy as np, tensorflow as tf

L, H = 168, 24                                # lookback 1 week, horizon 1 day (hourly)
)
series = np.log1p(agent_df["volume_kes"].to_numpy("float32")) # tame the
tail
# temporal split BEFORE windowing: never window across the boundary
n = len(series)
train, valid, test = series[: int(.7*n)], series[int(.7*n): int(.85*n)],
series[int(.85*n):]

mu, sd = train.mean(), train.std() # scaler fitted on train only (leakage!)
scale = lambda a: (a - mu) / sd

def make_ds(arr, shuffle):
    return tf.keras.utils.timeseries_dataset_from_array(
        data=scale(arr)[: -H], targets=None, sequence_length=L + H,
        batch_size=256, shuffle=shuffle
    ).map(lambda w: (w[:, :L, None], w[:, L:])) # (X: [B,L,1], y: [B,H])

train_ds, valid_ds = make_ds(train, True), make_ds(valid, False)
```

Listing 8.1: Windowing a series with `tf.keras`. The split precedes the windowing.

8.4.1 Three ways to cover the horizon

Having framed the problem, we must decide how a model that naturally predicts one step produces H of them. There are three strategies, and the choice has consequences.

The **recursive** strategy predicts $t + 1$, appends the prediction to the history, and predicts $t + 2$ from it, iterating H times (Figure 8.10). It needs only one model for any horizon, which is why classical forecasting adopted

it. Its weakness is structural: after the first step the model is consuming its own output, which it never did during training, and errors accumulate along the horizon faster than the model's own uncertainty estimates suggest.

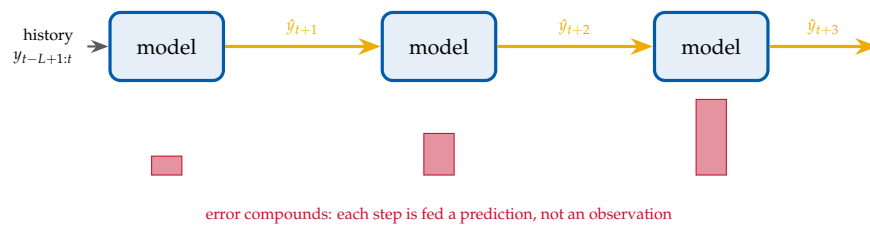


Figure 8.10: One model, applied H times to its own output. Cheap, classical, and biased towards over-confident long horizons.

The **direct** strategy emits all H values in a single forward pass, from a `Dense(H)` head on top of an encoder. There is no feedback, so there is no compounding; the price is that the model must learn a slightly different mapping for each horizon step inside one output layer. For operational horizons of a day or three this is hard to beat for the effort, and it is the default in this chapter.

The **sequence-to-sequence** strategy runs an encoder over the past and a decoder over the future, so that each future step can consume its own known-future covariates (Figure 8.11). It is the most flexible of the three, and it is the skeleton inside the Temporal Fusion Transformer. Reach for it when rich known-future covariates should shape individual future steps differently, which is exactly the case when a public holiday falls on hour 14 of the horizon and on no other.

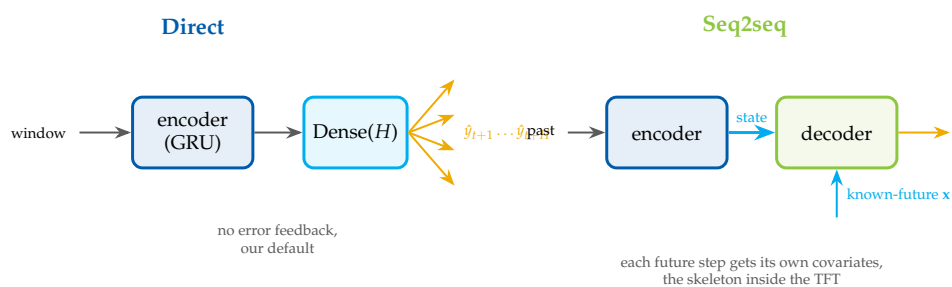


Figure 8.11: Two ways to emit the whole horizon in one pass. The seq2seq encoder-decoder is the structure the attention mechanism of Section 8.7 was invented to repair.

8.5 Recurrent neural networks

We now have a supervised dataset of windows, and could feed it to any model in Part I. A recurrent network is the architecture that respects what the window actually is: not a bag of L numbers, but L numbers *in order*.

Definition 8.3 (Recurrent layer). A recurrent layer maintains a hidden state $\mathbf{h}_t \in \mathbb{R}^d$ and updates it at every step with the *same* parameters,

$$\mathbf{h}_t = \tanh(\mathbf{W}_x^\top \mathbf{x}_t + \mathbf{W}_h^\top \mathbf{h}_{t-1} + \mathbf{b}), \quad \mathbf{h}_0 = \mathbf{0}. \quad (8.6)$$

Figure 8.12 draws (8.6) twice: once as a cell with a loop, and once unrolled through time. The unrolled picture is the useful one, because it makes clear that a recurrent network *is* a deep feed-forward network whose layers happen to share weights.

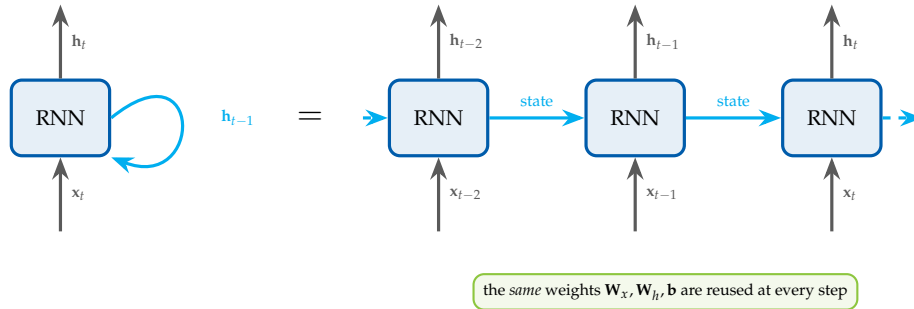


Figure 8.12: Left: the cell with its loop. Right: the same cell unrolled through time. Weight sharing across time is the exact analogue of the convolutional weight sharing across space in Chapter 7.

Three consequences follow, and they are worth stating plainly because the architecture is often described in more mystical terms than it deserves.

The state $\mathbf{h}_t$ is a *learned, lossy summary of everything seen so far*. Nothing constrains what it stores; the network decides, from the gradient of the loss, which aspects of the past are worth carrying forward in d numbers. The parameter count is *independent of the sequence length*, so the same cell handles $L = 50$ or $L = 500$. And because the unrolled network is a deep network with tied weights, every intuition about deep networks transfers, including the unwelcome ones.

8.5.1 Backpropagation through time, and why it fails

Training is ordinary backpropagation applied to the unrolled graph, a procedure traditionally given a name of its own, *backpropagation through time*. Because the weights are shared, the gradients arriving at each unrolled copy are *summed* to give the gradient of the single underlying parameter (Figure 8.13).

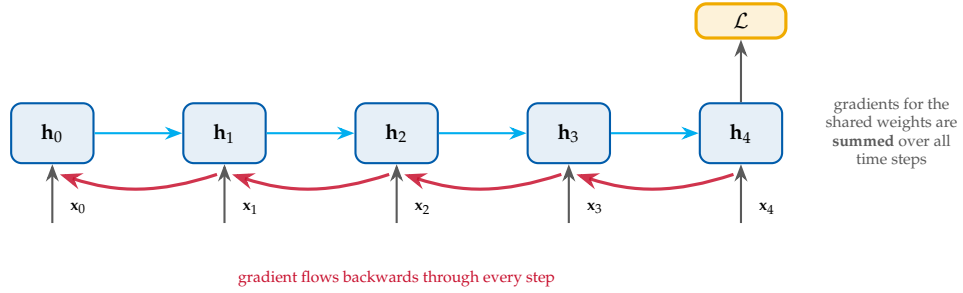


Figure 8.13: Nothing new in principle: unroll, run backprop, sum the gradients reaching each shared weight. The trouble is in the arithmetic of the product that the chain rule produces.

That arithmetic is where the architecture breaks. The sensitivity of a late state to an early one is a *product* of Jacobians,

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} = \prod_{j=t-k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}, \quad \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \mathbf{W}_h^\top \text{diag}(\tanh'(\cdot)). \quad (8.7)$$

Proposition 8.2 (Vanishing and exploding gradients). *Suppose every factor in (8.7) has spectral norm at most λ . Then*

$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-k}} \right\| \leq \lambda^k.$$

Consequently, if $\lambda < 1$ the influence of step $t - k$ on the loss decays geometrically in k ; and since $|\tanh'| \leq 1$, the bound holds with $\lambda = \|\mathbf{W}_h\|$.

Proof. Sub-multiplicativity of the operator norm gives $\|\prod_j J_j\| \leq \prod_j \|J_j\| \leq \lambda^k$. For the second claim, $\|\mathbf{W}_h^\top \text{diag}(\tanh')\| \leq \|\mathbf{W}_h\| \|\text{diag}(\tanh')\| \leq \|\mathbf{W}_h\|$, because a diagonal matrix with entries in $[0, 1]$ has norm at most one. $\square$

The bound is one-sided and that is enough. A geometric decay with $\lambda = 0.9$ leaves a factor of $0.9^{168} \approx 2 \times 10^{-8}$ across our one-week lookback: the gradient signal from the beginning of the window is numerically absent. Symmetrically, if the norms exceed one the product explodes and training diverges in a single step. In practice a plain recurrent network learns dependencies over roughly ten to fifty steps, which is far short of the 168 we need to see last Tuesday.

Several partial remedies exist, and all are worth using. Gradient clipping, the `clipnorm` argument, prevents the explosion outright; layer normalisation stabilises the scale of the recurrent pre-activations; and a causal one-dimensional convolution placed before the recurrent layer shortens the effective sequence, so that the product in (8.7) has fewer factors. None of them addresses the vanishing side, and for that we need a different cell.

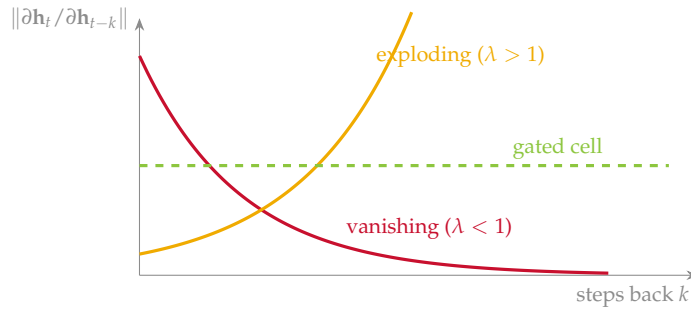


Figure 8.14: A gated cell keeps a path along which the gradient neither shrinks nor blows up. Constructing that path is the entire content of the next section.

8.6 Gated cells: LSTM and GRU

The insight behind gated cells is easy to state and was hard to find. If the problem is that state passes through a *multiplication* by $\mathbf{W}_h$ at every step, then give the state a route that is *additive* instead, and let the network learn when to write to it and when to read from it.

8.6.1 The LSTM cell

The long short-term memory cell of Hochreiter and Schmidhuber carries two states. The *long-term* state $\mathbf{c}_t$ runs along the top of Figure 8.15 like a conveyor belt, touched only by elementwise operations. The *short-term* state $\mathbf{h}_t$ is what the rest of the network sees. Three gates, each a sigmoid layer producing values in $(0, 1)$, decide what is erased, what is written and what is exposed.

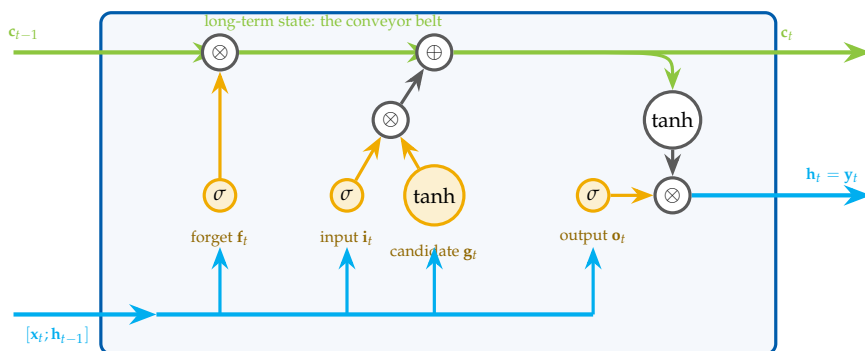


Figure 8.15: A memory with learned read, write and forget operations. The *long-term state* is modified only by an elementwise multiplication and an addition, which is exactly what Theorem 8.1 exploits.

Definition 8.4 (LSTM cell). With σ the logistic function and $\otimes$ elementwise multiplication,

$$\begin{aligned}
 \mathbf{i}_t &= \sigma(\mathbf{W}_{xi}^\top \mathbf{x}_t + \mathbf{W}_{hi}^\top \mathbf{h}_{t-1} + \mathbf{b}_i) && \text{input gate: what to write} \\
 \mathbf{f}_t &= \sigma(\mathbf{W}_{xf}^\top \mathbf{x}_t + \mathbf{W}_{hf}^\top \mathbf{h}_{t-1} + \mathbf{b}_f) && \text{forget gate: what to erase} \\
 \mathbf{o}_t &= \sigma(\mathbf{W}_{xo}^\top \mathbf{x}_t + \mathbf{W}_{ho}^\top \mathbf{h}_{t-1} + \mathbf{b}_o) && \text{output gate: what to reveal} \\
 \mathbf{g}_t &= \tanh(\mathbf{W}_{xg}^\top \mathbf{x}_t + \mathbf{W}_{hg}^\top \mathbf{h}_{t-1} + \mathbf{b}_g) && \text{candidate content} \\
 \mathbf{c}_t &= \mathbf{f}_t \otimes \mathbf{c}_{t-1} + \mathbf{i}_t \otimes \mathbf{g}_t && \text{memory update} \\
 \mathbf{h}_t &= \mathbf{o}_t \otimes \tanh(\mathbf{c}_t) && \text{exposed state}
 \end{aligned} \tag{8.8}$$

Everything rests on (8.8). The memory update is a *sum*, so its derivative with respect to the previous memory is not a weight matrix.

Theorem 8.1 (The constant error carousel). *For the cell of Definition 8.4, treating the gates as approximately constant over the interval,*

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \text{diag}(\mathbf{f}_t), \quad \text{so} \quad \frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-k}} = \text{diag}\left(\prod_{j=t-k+1}^t \mathbf{f}_j\right).$$

In particular, if the forget gate saturates at $\mathbf{f}_j \approx \mathbf{1}$ along a coordinate, the gradient traverses that coordinate over k steps undamped, for any k .

Proof. Differentiate (8.8). The second term does not involve $\mathbf{c}_{t-1}$, and the first is an elementwise product, whose Jacobian is the diagonal matrix of the multiplier. Composing over k steps multiplies the diagonals elementwise. $\square$

Compare this with Proposition 8.2, where the product was of *learned matrices* whose norms the network had no direct incentive to keep near one. Here the product is of *gate activations*, which the network controls explicitly and can drive to one whenever remembering is useful. This is the same structural trick as the residual connection of Chapter 7: convert a multiplicative path into an additive one and the gradient stops vanishing. Recognising the two as one idea is worth more than memorising either.

Key Idea

Gates are memory management, not magic. A sigmoid output of 0.97 means “keep almost all of this”; gates are soft, differentiable switches rather than branches. Keras initialises $\mathbf{b}_f = \mathbf{1}$ so that an untrained cell begins by *remembering everything* and learns what to discard, because a cell that began by forgetting would kill its own gradient signal before learning anything. Concretely, the cell can hold “a month-end spike is approaching” in one coordinate of $\mathbf{c}_t$ across hundreds of hourly steps and release it when the calendar features fire.

8.6.2 The GRU cell

The gated recurrent unit of Cho and co-authors merges the two states into one and couples two of the gates, on the reasoning that erasing and writing are really one decision (Figure 8.16).

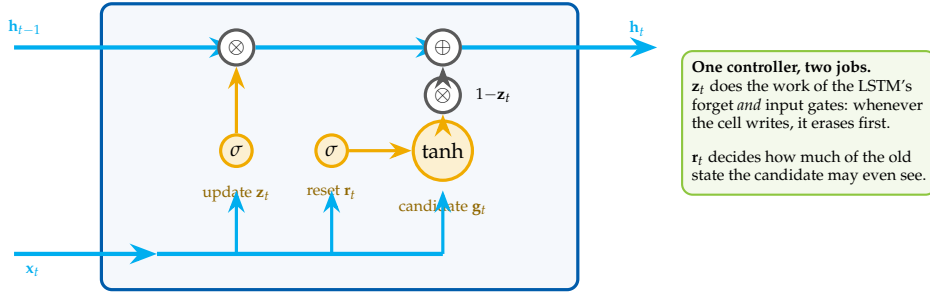


Figure 8.16: One state, two gates. The final update is a convex combination, so the GRU cannot write without erasing.

Definition 8.5 (GRU cell).

$$\begin{aligned}
 \mathbf{z}_t &= \sigma(\mathbf{W}_{xz}^\top \mathbf{x}_t + \mathbf{W}_{hz}^\top \mathbf{h}_{t-1} + \mathbf{b}_z) && \text{update gate} \\
 \mathbf{r}_t &= \sigma(\mathbf{W}_{xr}^\top \mathbf{x}_t + \mathbf{W}_{hr}^\top \mathbf{h}_{t-1} + \mathbf{b}_r) && \text{reset gate} \\
 \mathbf{g}_t &= \tanh(\mathbf{W}_{xg}^\top \mathbf{x}_t + \mathbf{W}_{hg}^\top (\mathbf{r}_t \otimes \mathbf{h}_{t-1}) + \mathbf{b}_g) && \text{gated view of the past} \\
 \mathbf{h}_t &= \mathbf{z}_t \otimes \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \otimes \mathbf{g}_t && \text{convex mixture} \quad (8.9)
 \end{aligned}$$

The last line preserves the property that matters. With $\mathbf{z}_t \rightarrow \mathbf{1}$ the state is copied unchanged, so $\partial \mathbf{h}_t / \partial \mathbf{h}_{t-1} \rightarrow \mathbf{I}$ and the argument of Theorem 8.1 applies verbatim. The GRU obtains the same protected path with one state and two gates instead of two states and three.

Table 8.1 compares them. The honest summary is that the accuracy difference is small and dataset-dependent, so the course default is to start with the GRU, which trains faster, and to try the LSTM only if the backtest asks for it.

Table 8.1: The two gated cells compared. d_x is the input width and d_h the state width.

	LSTM	GRU
States	2 ($\mathbf{c}_t, \mathbf{h}_t$)	1 ($\mathbf{h}_t$)
Gates	3	2
Parameters per unit	$4(d_x + d_h + 1)d_h$	$3(d_x + d_h + 1)d_h$
Training speed	slower	roughly 25% faster
Accuracy in practice	roughly equivalent	

Neither cell removes the two structural limits of recurrence. The state is a fixed-size bottleneck, so everything the model knows about 168 hours

must fit in d numbers; and the computation is inherently *sequential*, since $\mathbf{h}_t$ cannot be computed before $\mathbf{h}_{t-1}$, which forfeits the parallelism a GPU exists to provide. Section 8.7 removes both.

8.6.3 A forecaster we can actually train

Figure 8.17 shows the architecture we use for the agent-volume task, and Listing 8.2 implements it. The causal convolution at the front is the cheap win: it compresses 168 steps to 42 before the recurrent layers see them, so the Jacobian product of Proposition 8.2 has a quarter as many factors.

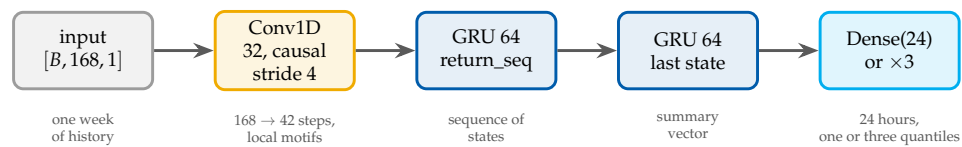


Figure 8.17: Causal convolutional front-end, stacked GRU, direct multi-horizon head. Every design choice in this diagram is defended in the surrounding text.

```

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(L, 1)),           # [batch, 168, 1]
    # 1D-conv front-end: shortens the sequence 4x, sees local motifs
    tf.keras.layers.Conv1D(32, kernel_size=6, strides=4,
                           padding="causal", activation="relu"),
    tf.keras.layers.GRU(64, return_sequences=True),
    tf.keras.layers.GRU(64),                       # last state only
    tf.keras.layers.Dense(H)                       # direct 24-step output
])

model.compile(
    loss=tf.keras.losses.Huber(),                  # robust to KES spikes
    optimizer=tf.keras.optimizers.Adam(1e-3, clipnorm=1.0),
    metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")]
)
  
```

Listing 8.2: A direct multi-horizon GRU forecaster.

Three choices in Listing 8.2 are deliberate and easy to get wrong. Causal padding pads only on the left, so the convolution at position t never sees $t + 1$; ordinary symmetric padding here would be a leak. The clipnorm argument handles the exploding half of Proposition 8.2. And the Huber loss keeps one flash-crowd hour from dominating the gradient, which squared error would not.

8.6.4 From a point to a fan

Proposition 8.1 told us that a head trained on L_q estimates the q -th quantile. Attaching three such heads to the same recurrent trunk therefore produces

a forecast *fan* at negligible extra cost (Figure 8.18), and the fan, not the central line, is what the liquidity desk buys.

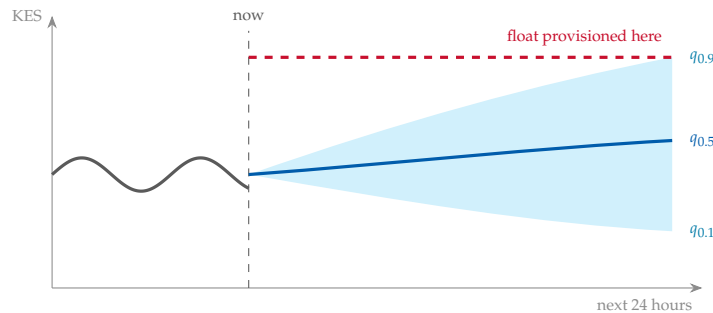


Figure 8.18: Provisioning at $q_{0.9}$ accepts a stock-out roughly one day in ten, a choice the desk can defend in writing. The fan is the product, not a by-product.

```
QS = [0.1, 0.5, 0.9]

def pinball(q):
    def loss(y, yhat):
        # y: [B,H], yhat: [B,H]
        e = y - yhat
        return tf.reduce_mean(tf.maximum(q * e, (q - 1.0) * e))
    return loss

inp = tf.keras.layers.Input(shape=(L, 1))
h = tf.keras.layers.GRU(64)(inp)
outs = [tf.keras.layers.Dense(H, name=f"q{int(q*100)}")(h) for q in QS]

qmodel = tf.keras.Model(inp, outs)
qmodel.compile(optimizer="adam", loss=[pinball(q) for q in QS])
```

Listing 8.3: Three quantile heads on one recurrent trunk.

Nothing in Listing 8.3 enforces $\hat{y}^{(0.1)} \leq \hat{y}^{(0.5)} \leq \hat{y}^{(0.9)}$, and on hard examples the heads do occasionally cross. Either add a monotonicity penalty or simply sort the three outputs at inference time; the second costs nothing and is what we do.

Bidirectional layers in a forecaster

A Bidirectional wrapper runs a second recurrent pass from right to left, and on classification tasks it usually helps. In forecasting it is a guaranteed leak wherever it wraps a layer that emits predictions, because the backward pass carries information from $t + 1$ into the output at t . The resulting validation scores are spectacular and entirely fictional. Encoding a *completed* past window bidirectionally is legitimate; anything downstream of the forecast origin is not.

Fine: encoding a completed past window



Never: wrapping a layer that emits forecasts

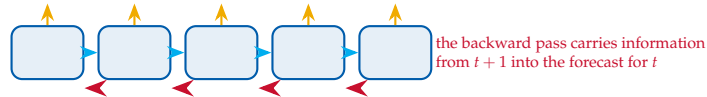


Figure 8.19: The same wrapper, used two ways, with opposite consequences. Only the upper use is admissible in a forecaster.

Two further cautions apply once one model serves five thousand agents. Scale *each* agent's series by its own training statistics and supply a static agent embedding, or the handful of largest agents will dominate the loss and the model will learn their shape alone. And handle cold starts and gaps explicitly: new agents have no history, network downtime creates missing hours, so mask or impute and add an `is_imputed` indicator channel, because the model should know when it is looking at a guess.

Temporal leakage checklist

Before believing any backtest number, verify all six: scalers fitted on the training span only; no window straddling a split boundary; causal padding everywhere; target-derived features lagged by at least the forecast latency; backtest folds strictly ordered in time; and every known-future covariate genuinely knowable at the forecast origin. A single violation in this list is usually worth several points of apparent accuracy, all of them imaginary.

8.7 Attention

Everything so far has routed information through a state of fixed width. This section removes that constraint. Attention is the mechanism that lets a model reach back to any earlier position directly, and because it is the foundation of Chapter 9 as well as of the Temporal Fusion Transformer in the next section, we build it here from the problem it was invented to solve rather than presenting the formula and working backwards.

8.7.1 The bottleneck

Consider the seq2seq forecaster of Figure 8.11. The encoder consumes $L = 168$ hourly observations and hands the decoder a single state vector $\mathbf{h}_L \in \mathbb{R}^{64}$. Everything the decoder will ever know about the past week must fit in those 64 numbers.

Two distinct problems follow. The obvious one is capacity: a week of hourly volumes, float balances and downtime is a great deal of information to compress into a fixed vector, and the compression is lossy in ways nobody controls. The subtler one is *path length*. For the decoder to use what happened at hour 1, that information must survive 167 successive state updates. Even with the protected path of Theorem 8.1, the cell must have decided, at hour 1, that this was worth keeping, without yet knowing what would be asked of it.

Key Idea

The question attention asks Recurrence compresses the past into a state and hopes the compression kept the right things. Attention keeps the past in full and, at the moment of prediction, *looks up* what it needs. The change is from “remember everything that might matter” to “search for what matters now”.

8.7.2 Attention as soft dictionary lookup

The cleanest way to understand the mechanism is as a differentiable dictionary. An ordinary lookup table holds key–value pairs and, given a query, returns the value whose key matches exactly. That operation is useless in a network because it is not differentiable: change the query infinitesimally and the returned value either does not move at all or jumps.

Attention softens it. Instead of returning one value, return a weighted average of *all* values, with weights given by how well each key matches the query (Figure 8.20). Now an infinitesimal change in the query produces an infinitesimal change in the weights and hence in the output, so the whole thing can be trained by gradient descent.

Definition 8.6 (Scaled dot-product attention). Let $\mathbf{Q} \in \mathbb{R}^{n_q \times d_k}$, $\mathbf{K} \in \mathbb{R}^{n_v \times d_k}$ and $\mathbf{V} \in \mathbb{R}^{n_v \times d_v}$ be matrices of queries, keys and values. Scaled dot-product attention is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \underbrace{\text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)}_{\mathbf{A} \in \mathbb{R}^{n_q \times n_v}} \mathbf{V}, \quad (8.10)$$

with the softmax taken along each row, so that every row of the *attention matrix* $\mathbf{A}$ is non-negative and sums to one.

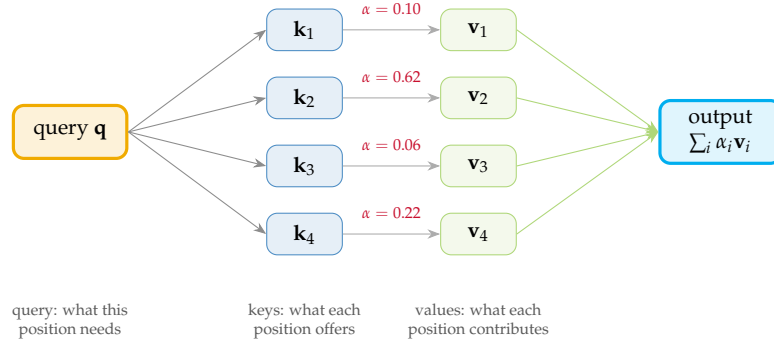


Figure 8.20: A differentiable dictionary. The weights α_i sum to one and are computed from query–key similarity, so the output is a convex combination of the values, dominated here by position 2.

Each ingredient earns its place. The **dot product** $\mathbf{q}^\top \mathbf{k}$ measures similarity, and it is the cheapest such measure available, being one matrix multiplication for all pairs at once. The **softmax** turns arbitrary real scores into a convex combination, so the output stays on the same scale as the values however many positions there are. The **scaling** by $\sqrt{d_k}$ is the one part that looks arbitrary and is not, so it is worth a moment.

Suppose the entries of $\mathbf{q}$ and $\mathbf{k}$ are independent with mean zero and unit variance. Then $\mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$ has mean zero and variance d_k , so its typical magnitude grows like $\sqrt{d_k}$. For $d_k = 64$ the scores would routinely reach ± 8 , and a softmax over scores that far apart is nearly a hard maximum: one weight near one, the rest near zero, and gradients near zero everywhere. Dividing by $\sqrt{d_k}$ restores unit variance and keeps the softmax in the region where it has a usable gradient. This is the same reasoning that governed initialisation in Chapter 6, applied to a different layer.

8.7.3 Self-attention, and what queries, keys and values are

In *self*-attention the three matrices are all linear projections of the same sequence. If $\mathbf{X} \in \mathbb{R}^{L \times d}$ holds the representations of L time steps, then

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad (8.11)$$

with $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ learned. Every position simultaneously asks a question, advertises what it can answer, and offers content. For our agent series, a plausible learned behaviour is that the query at hour t encodes “I am hour 14 on a Tuesday of a payroll week”, the key at hour $t - 168$ encodes “I am hour 14 on a Tuesday of a payroll week”, the two match strongly, and the value at $t - 168$, the volume actually observed then, flows into the representation at t .

Notice what has been bought. That interaction crossed 168 steps in *one* operation. There is no state to survive, nothing to remember in advance, and no product of Jacobians to decay.

Proposition 8.3 (Path length and cost). *For a sequence of length L and width d , self-attention connects any two positions through a single operation, so the gradient path length between them is $\mathcal{O}(1)$, against $\mathcal{O}(L)$ for a recurrent layer. Its cost is $\mathcal{O}(L^2d)$ time and $\mathcal{O}(L^2)$ memory for the matrix $\mathbf{A}$, against $\mathcal{O}(Ld^2)$ time and $\mathcal{O}(Ld)$ memory for recurrence, and unlike recurrence its computation across positions is parallel.*

Proof. The product $\mathbf{QK}^\top$ forms L^2 inner products of length d_k , giving $\mathcal{O}(L^2d)$ operations and an $L \times L$ matrix; multiplying by $\mathbf{V}$ costs the same order. Each output row depends on the inputs only through (8.10), so all rows may be computed simultaneously, and position i influences position j through the single entry A_{ji} . A recurrent layer performs L updates each costing $\mathcal{O}(d^2)$, and each must wait for its predecessor; the influence of position i on position j passes through $j - i$ successive updates. $\square$

Proposition 8.3 is the whole trade. Attention buys constant path length and parallelism, and pays for them with quadratic cost in the sequence length. At $L = 168$ the attention matrix has 28,224 entries per head per example, which is nothing; at $L = 100,000$ it is prohibitive, which is why Chapter 9 spends time on efficient variants.

8.7.4 Causal masking

Definition 8.6 lets every position attend to every other, including later ones. In a forecaster that is the leakage of Figure 8.19 in a new costume. The remedy is a *causal mask*: before the softmax, set the score of every disallowed pair to $-\infty$,

$$\tilde{s}_{ij} = \begin{cases} s_{ij} & j \leq i, \\ -\infty & j > i, \end{cases} \quad (8.12)$$

so that $\exp(-\infty) = 0$ and those positions receive exactly zero weight. The mask is applied to the scores rather than to the weights for a reason worth noticing: zeroing weights after the softmax would leave the surviving weights summing to less than one, whereas masking before it renormalises the remainder correctly. Figure 8.21 shows the resulting lower-triangular pattern. In Keras this is one argument, `use_causal_mask=True`.

8.7.5 Multiple heads

One attention operation computes one weighted average, and therefore expresses one notion of relevance. But hour t plausibly needs several at once: the same hour yesterday, the same hour last week, and the recent trend

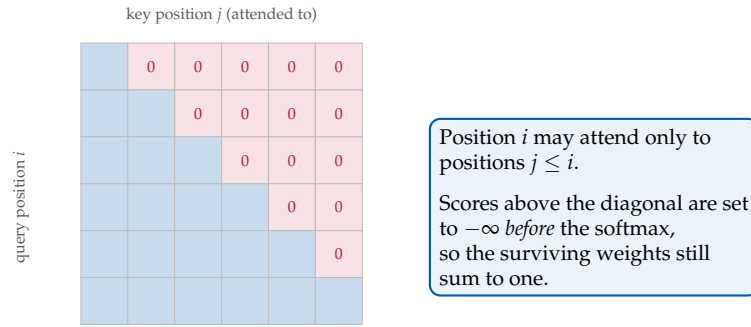


Figure 8.21: The lower-triangular mask of (8.12). Without it, a forecaster attends to its own future and reports validation scores that cannot be reproduced in production.

over the last three hours. A single softmax must divide its unit budget of weight among all three.

Multi-head attention runs n_h independent attention operations in parallel on different learned projections and concatenates the results,

$$\text{MultiHead}(\mathbf{X}) = [\text{head}_1; \dots; \text{head}_{n_h}] \mathbf{W}_O,$$

$$\text{head}_h = \text{Attention}(\mathbf{X}\mathbf{W}_Q^{(h)}, \mathbf{X}\mathbf{W}_K^{(h)}, \mathbf{X}\mathbf{W}_V^{(h)}). \quad (8.13)$$

Each head projects into a subspace of width d/n_h , so the total cost is essentially unchanged; what changes is that different heads can specialise, and in trained models they demonstrably do (Figure 8.22).

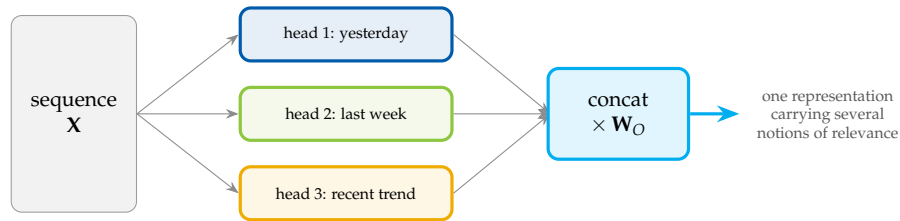


Figure 8.22: Each head projects into a narrower subspace and learns its own relevance pattern. The labels are illustrative of what heads are observed to specialise in on seasonal data; nothing assigns them in advance.

8.7.6 Reading an attention map

Because $\mathbf{A}$ is an explicit matrix of non-negative weights summing to one, it can be plotted, and the plot is a genuine statement about what the model used. Figure 8.23 shows the pattern a well-behaved hourly forecaster produces: bright bands at lags 24 and 168.

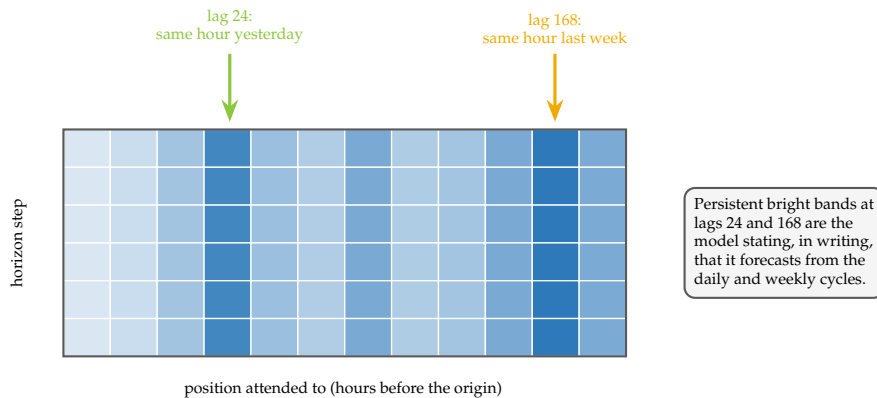


Figure 8.23: Schematic attention pattern for an hourly forecaster. Regenerate from the trained model before any of this appears in a governance pack.

Attention weights are not explanations

An attention map says which *representations* were mixed, and those representations have already been contaminated by earlier layers, so a large weight on position $t - 168$ does not establish that the raw observation there caused the forecast. Attention is strong evidence about the model's internal routing and weak evidence about causal attribution. Report it as the former. The Temporal Fusion Transformer improves matters, as Section 8.8 explains, by sharing the value projection across heads so that one aggregate map is meaningful, but even then the claim is about the model and not about the world.

Attention alone, however, has thrown something away. Equation (8.10) is *permutation equivariant*: shuffle the rows of $\mathbf{X}$ and the outputs shuffle identically, because nothing in the formula refers to position. For sequences this is a real loss, and there are two repairs. Chapter 9 adds explicit positional encodings. The Temporal Fusion Transformer takes the other route and lets a recurrent layer supply the ordering, which is the design we turn to now.

8.8 The Temporal Fusion Transformer

We now have every component: a covariate taxonomy, quantile outputs, recurrence for local order, and attention for long-range lookup. The Temporal Fusion Transformer of Lim and co-authors assembles exactly these into one forecaster, and it is worth studying less as a specific model than as a worked example of how the pieces compose.

Four requirements motivate it, and each maps to one component. Real systems have **heterogeneous inputs** that deserve different treatment, which motivates per-family variable selection. They need **multi-horizon**

quantiles in one pass, which motivates the pinball heads we already have. They have **long-range dependencies**, which motivates attention. And they must answer the question *which inputs drove this forecast*, which motivates both the selection weights and the interpretable attention variant.

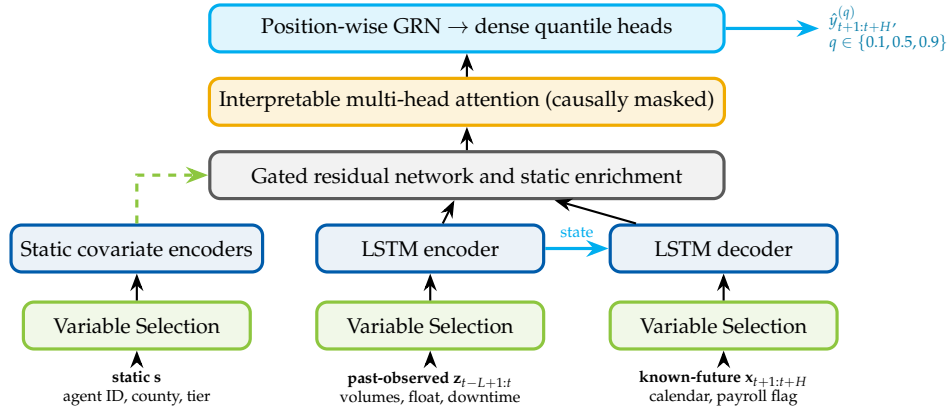


Figure 8.24: One forward pass, bottom to top. Every component is skippable through its gate, so on a simple dataset the TFT can learn to behave like a plain LSTM.

8.8.1 Gated residual networks

The TFT wraps almost every transformation in the same block, whose purpose is to let the network decide *how much nonlinearity it actually needs*:

$$\begin{aligned}\eta_2 &= \text{ELU}(\mathbf{W}_2 \mathbf{a} + \mathbf{W}_3 \mathbf{c} + \mathbf{b}_2), & \eta_1 &= \mathbf{W}_1 \eta_2 + \mathbf{b}_1, \\ \text{GLU}(\gamma) &= \sigma(\mathbf{W}_4 \gamma + \mathbf{b}_4) \otimes (\mathbf{W}_5 \gamma + \mathbf{b}_5), \\ \text{GRN}(\mathbf{a}, \mathbf{c}) &= \text{LayerNorm}(\mathbf{a} + \text{GLU}(\eta_1)),\end{aligned}\tag{8.14}$$

where $\mathbf{a}$ is the primary input and $\mathbf{c}$ an optional static context vector. The gated linear unit can drive its sigmoid term to zero, at which point the whole block collapses to the identity, so unnecessary complexity is *learned away* rather than tuned away (Figure 8.25). Note the family resemblance to the LSTM and GRU gates: the same multiplicative switching, applied depth-wise instead of time-wise.

8.8.2 Variable selection

Each of the three input families passes through its own *variable selection network*. Every variable j is first embedded to $\xi_t^{(j)}$; a GRN conditioned on the static context then produces weights over the m variables, and the layer

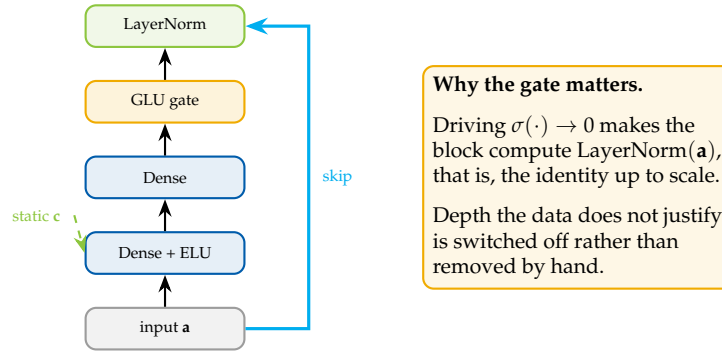


Figure 8.25: The TFT’s universal wrapper, equation (8.14). It is the residual block of Chapter 7 with a learned gate on the residual branch.

outputs their weighted combination:

$$\mathbf{v}_t = \text{softmax}(\text{GRN}(\Xi_t, \mathbf{c}_s)) \in \mathbb{R}^m, \quad \tilde{\xi}_t = \sum_{j=1}^m v_t^{(j)} \text{GRN}_j(\xi_t^{(j)}). \quad (8.15)$$

Two properties make this more than a tidy detail. Noisy inputs receive weights near zero, which is regularisation by architecture rather than by penalty. And because the weights are computed per instance, a rural agent and a city-centre agent may rely on different variables at the same hour, while averaging $\mathbf{v}_t$ over the test set still yields a global importance ranking at no extra cost (Figure 8.26).

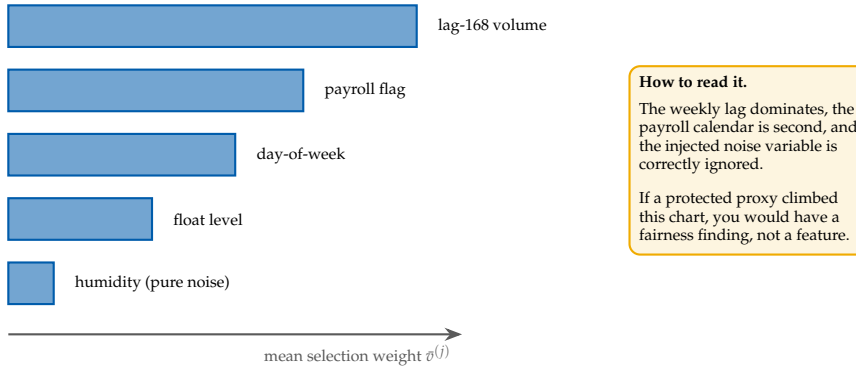


Figure 8.26: The chart that goes into the model documentation. Illustrative values; regenerate from the real backtest before distribution.

8.8.3 Interpretable attention, and the rest of the model

The attention block is Definition 8.6 with the causal mask of (8.12) and one modification. In standard multi-head attention each head has its own value

projection, so the heads weight *different* representations and their maps cannot sensibly be averaged. The TFT instead shares $\mathbf{W}_V$ across heads and averages the attention matrices,

$$\tilde{\mathbf{A}} = \frac{1}{n_h} \sum_{h=1}^{n_h} \text{softmax} \left(\frac{\mathbf{Q}\mathbf{W}_Q^{(h)} (\mathbf{K}\mathbf{W}_K^{(h)})^\top}{\sqrt{d_{\text{attn}}}} \right), \quad (8.16)$$

so that a *single* aggregate map explains which past hours drove each forecast. Some expressive power is given up; a readable figure for the model-risk committee is bought.

The remaining structure follows Figure 8.24. An LSTM encoder-decoder supplies local ordering, which is why the TFT needs no positional encodings; the static context vector is injected into selection, gating and attention alike, so agent identity conditions every stage; and the output is a position-wise GRN feeding dense quantile heads trained on the summed pinball loss

$$\mathcal{L} = \sum_{q \in \mathcal{Q}} \sum_{h=1}^H L_q(y_{t+h}, \hat{y}_{t+h}^{(q)}), \quad \mathcal{Q} = \{0.1, 0.5, 0.9\}. \quad (8.17)$$

One *global* model is fitted across all five thousand agents, with static embeddings distinguishing them. That cross-learning is where deep forecasters earn their keep against per-series ARIMA: an agent with three weeks of history borrows seasonal structure from the thousands of agents that resemble it.

```
import tensorflow as tf
from tensorflow.keras import layers, Model, Input, optimizers
L, H, d, Q = 168, 24, 64, [0.1, 0.5, 0.9]

def grn(x, d):
    # Gated Residual Network,
    # eq (8.14)
    h = layers.Dense(2*d)(layers.Dense(d, activation="elu")(x))
    val, gate = tf.split(h, 2, axis=-1)
    # GLU: eta_1 * sigmoid(gate)
    return layers.LayerNormalization()(layers.Dense(d)(x) + val * tf.sigmoid(gate))

past = Input((L, n_past)); futr = Input((H, n_known)); stat = Input((n_static,))
s = grn(stat, d)
# static context vector
enc, h, c = layers.LSTM(d, return_sequences=True, return_state=True)(
    grn(past, d), initial_state=[layers.Dense(d)(s)] * 2)
dec = layers.LSTM(d, return_sequences=True)(grn(futr, d), initial_state=[h, c])
seq = grn(layers.Concatenate(1)([enc, dec]) + s[:, None, :], d)
# static enrichment
att = layers.MultiHeadAttention(num_heads=2, key_dim=d // 2)(
```

```

        seq, seq, use_causal_mask=True)          # causal mask, eq (8.16)
y    = layers.Dense(len(Q))(grn(seq + att, d)[: , -H: , :])      # 3
    quantile heads

def pinball(y_true, y_pred):                      # summed quantile loss, eq
    (8.20)
    e = y_true[... , None] - y_pred
    q = tf.constant(Q, tf.float32)
    return tf.reduce_mean(tf.maximum(q * e, (q - 1.0) * e))

tft = Model([past, futr, stat], y)
tft.compile(optimizer=Adam(1e-3, clipnorm=1.0), loss=pinball)

```

Listing 8.4: A compact Temporal Fusion Transformer in `tf.keras`. The covariate taxonomy becomes three input tensors.

Listing 8.4 omits the variable-selection networks for space; they are a per-feature GRN followed by a softmax, exactly as in (8.15), and Exercise 10 asks you to add them.

8.8.4 Choosing a model

Table 8.2 states the position this book takes. It is deliberately unromantic, because model choice in forecasting is an evaluation question rather than a fashion question.

Table 8.2: A pragmatic decision table. Run the backtest; let seasonal naïve keep everyone honest.

Situation	Reach for
One low-frequency series, need something defensible tomorrow	Seasonal naïve, then ETS, then SARIMA
Many series, tabular covariates, moderate horizons	LightGBM on lag and calendar features
Many series, local temporal structure, few covariates	Stacked GRU or LSTM, direct multi-horizon head
Rich static and known-future covariates, quantiles <i>and</i> interpretability required	Temporal Fusion Transformer
Very long contexts, zero-shot or few-shot needs	Pretrained time-series foundation models, Chapter 9

The TFT is not free. Attention memory grows as $\mathcal{O}(L^2)$ by Proposition 8.3, the parameter count far exceeds a GRU's, and the hyperparameter surface is larger. It earns its cost when covariates are rich, horizons long and interpretability is a requirement rather than a preference. On a single series with no covariates it is an expensive way to reproduce an LSTM.

8.9 The case study end to end

We can now assemble the agent-liquidity system. The decision loop is fixed by operations (Figure 8.27): each evening the model forecasts the next twenty-four hours for every agent, the plan provisions priority agents at $q_{0.9}$, the runner’s route is dispatched, and the realised volumes return as tomorrow’s training data.

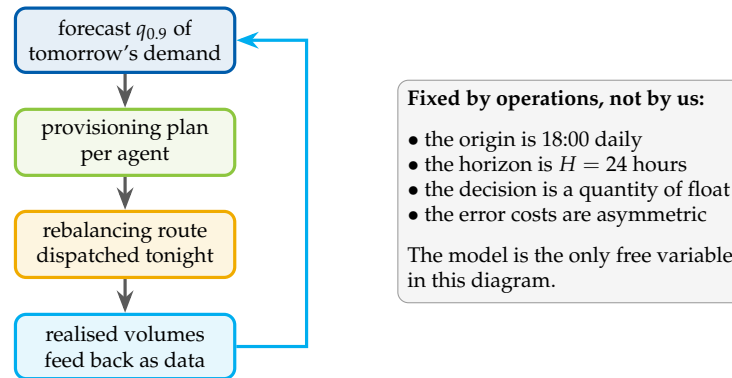


Figure 8.27: The forecast exists to support one recurring decision. Every modelling choice in this chapter is answerable to it.

Filling in Definition 8.1 for this problem gives Table 8.3. One entry repays attention. Nearby-agent aggregate volume feels like context and is therefore easy to file as known-future, but it is *measured*, so it arrives with the same lag as our own telemetry and belongs in $\mathbf{z}$. Filing it wrongly would produce a model that cannot be run at 18:00 at all.

Table 8.3: Every feature declares when it becomes known. The middle column is the leakage test.

Family	Known at 18:00?	Features
Target	–	hourly withdrawal and deposit volume, KES, per agent
Static $\mathbf{s}$	always	agent embedding, county, urban or rural, tier, years active
Past-observed $\mathbf{z}$	only up to t	volumes, float balances, downtime minutes, nearby-agent aggregate volume
Known-future $\mathbf{x}$	for all $t + h$	hour, day of week, public holidays, school terms, payroll and market-day flags

The protocol follows Section 8.3: twelve monthly rolling origins, MASE and pinball loss reported at $h \in \{6, 12, 24\}$, baselines of seasonal naïve, LightGBM on lags and our GRU, and one operational metric that the busi-

ness actually recognises, the float-out rate under $q_{0.9}$ provisioning. Table 8.4 gives the shape of the result.

Table 8.4: Twelve monthly rolling origins. **Placeholder figures:** regenerate on the final dataset before distribution.

Model	MASE @ 24h	Pinball (mean)	Float-out rate @ $q_{0.9}$
Seasonal naïve	1.00	n/a	11.2%
LightGBM (lags)	0.87	0.142	8.9%
GRU (direct, quantile)	0.82	0.131	7.6%
Temporal Fusion Transformer	0.76	0.118	5.4%

Read the last column rather than the first. A drop in MASE from 1.00 to 0.76 is a modelling result; a drop in float-outs from 11.2% to 5.4% is the result the business will fund. The two are related but not interchangeable, and a model that improves MASE while leaving float-outs unchanged has optimised the wrong thing, most often by improving the easy overnight hours where volumes are near zero and MASE is cheap to win.

Kenya Context

A model that allocates float across five thousand small businesses is not a neutral piece of arithmetic, and three obligations attach to it.

Under the **Data Protection Act, 2019**, per-agent telemetry is personal data wherever the agent is a sole proprietor, which most are. Purpose limitation and data minimisation apply, so telemetry collected to plan rebalancing may not be quietly repurposed to score the agent's own creditworthiness without a fresh lawful basis.

Float advanced against future commissions is a **loan**, which places the model inside the credit-model perimeter that the **CBK Digital Credit Providers Regulations, 2022** govern: validation reports, challenger models, a documented human override path and complaint handling are expected rather than optional. The variable-importance chart of Figure 8.26 and the attention map of Figure 8.23 are what a validation pack looks like when it is done cheaply.

The fairness question here is not the familiar one about rejection. It is about *systematic under-forecasting*. County and urban-rural covariates can proxy for ethnicity and income, and an agent whose demand is persistently under-predicted is persistently under-provisioned, loses transactions through no fault of their own, and earns less. Audit residuals by segment, not only in aggregate, and treat a segment with consistently negative mean error as a defect rather than as noise. Note also that choosing $q_{0.9}$ rather than $q_{0.5}$ is a policy decision that allocates the cost of uncertainty to the financing line instead of to the agent; it should be written down and approved, not buried in a configuration file.

Finally, these series drift, and for entirely mundane reasons: a competitor opens across the road, a tariff changes, the rains arrive late, an election is called. None of this is a modelling failure and none of it is visible without instrumentation. Rolling MASE per segment, with retraining triggers, is the subject of Chapter 11, and the forecasting model is one node in the system that Chapters 10 and 11 build.

Exercises

1. **The covariate taxonomy.** The agent-liquidity team proposes six features. For each, state which family of Definition 8.1 it belongs to, or explain why it cannot be used at all, in one sentence each: (a) the agent's county; (b) the number of transactions in the previous hour; (c) tomorrow being a public holiday; (d) tomorrow's rainfall, taken from the meteorological service's published forecast; (e) tomorrow's *actual* rainfall; (f) the agent's total commission earned over the following week.

2. **Window arithmetic.** A single agent has one year of hourly data, $N = 8,760$ observations, and we use $L = 168$, $H = 24$ and stride one. (a) How many windows can be cut from the undivided series? (b) The series is split 70/15/15 chronologically, and windows are cut *within* each block so that none straddles a boundary. How many windows does each block yield, and how many are lost relative to (a)? (c) Explain in two sentences why losing those windows is the correct behaviour rather than a defect to be engineered away.
3. **The pinball loss and the quantile.** (a) With $\hat{y}^{(q)} = 100$, compute L_q for $y = 120$ and for $y = 80$, at $q = 0.9$ and at $q = 0.1$, and comment on the asymmetry. (b) Let Y take values 10, 20, 30, 40, 50 with probabilities 0.1, 0.2, 0.4, 0.2, 0.1. Tabulate F , then use Proposition 8.1 to state the minimiser of $\mathbb{E}[L_q(Y, a)]$ for $q = 0.5$ and for $q = 0.9$, and verify the answers by direct evaluation. (c) Explain why a model trained with MAE systematically under-predicts the *mean* of a right-skewed volume series, and why this is correct behaviour rather than a bug.
4. **Metrics that disagree.** Over six test hours the realised volumes are $y = (120, 95, 140, 160, 110, 130)$, the model predicts $\hat{y} = (115, 105, 130, 150, 120, 125)$, and seasonal naïve predicts $(110, 100, 135, 150, 105, 140)$. Compute MAE, sMAPE and MASE for the model. What does the MASE value tell you, and what would you recommend to the team on the strength of it?
5. **Why plain recurrence forgets.** Using Proposition 8.2: (a) with $\lambda = 0.9$, bound the influence of the first observation of a 168-step window on the final state; (b) find the smallest k for which the bound falls below 10^{-6} ; (c) an LSTM coordinate holds its forget gate at $f = 0.99$ throughout the window. Using Theorem 8.1, compute the corresponding factor and compare. (d) In two or three sentences, explain why the LSTM's product is easier for the network to control than the plain RNN's.
6. **Counting gate parameters.** A recurrent layer has input width $d_x = 32$ and state width $d_h = 64$. (a) Using Table 8.1, compute the parameter count for an LSTM and for a GRU, and confirm the ratio. (b) The team proposes replacing a single GRU of width 128 with two stacked GRUs of width 64. Compute both parameter counts, assuming $d_x = 32$ into the first layer, and comment on what is gained and lost beyond the parameter count.
7. **Why attention divides by $\sqrt{d_k}$.** Let $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ have independent entries with mean zero and unit variance. (a) Show that $\mathbb{E}[\mathbf{q}^\top \mathbf{k}] = 0$ and $\text{Var}[\mathbf{q}^\top \mathbf{k}] = d_k$. (b) For $d_k = 64$, give the typical magnitude of an unscaled score, then compute the softmax weights of the score vector

$(8, 0, -8)$, and of the same vector after scaling by $1/\sqrt{d_k}$. (c) Explain, referring to the gradient of the softmax, why the unscaled case is a training problem and not merely an aesthetic one.

8. **Attention against recurrence.** Take $L = 168$ and $d = 64$. (a) Using Proposition 8.3, compute the leading-order operation counts for one self-attention layer and for one recurrent layer, and give their ratio. (b) At what sequence length do the two costs coincide, in terms of L and d ? (c) Repeat the comparison for $L = 4,096$. (d) State the two advantages attention retains at every length, and the one disadvantage that grows.

9. **Causal masking, carefully.** (a) For a window of $L = 168$ positions, how many of the L^2 query–key pairs survive the mask of (8.12), and what fraction is that? (b) A colleague implements masking by computing the full softmax over scores $(2.0, 1.0, 0.5)$ and then setting the last weight to zero. Compute their weights and the correctly masked weights, and quantify the error. (c) Explain in two sentences what would go wrong, in terms of the validation curve, if the mask were omitted entirely from a forecaster.

10. **Variable selection, implemented.** Extend Listing 8.4 with a variable-selection network for the past-observed stream, following (8.15). Embed each of the past-observed variables separately, pass each through its own GRN, produce softmax weights from a further GRN over the concatenation conditioned on the static context, and return the weighted sum together with the weights. Then state, in one short paragraph, how you would use the returned weights to satisfy the model-documentation obligation described in the Kenya Context box, and what you would do if the county embedding appeared among the three highest-weighted inputs.

Key Terms

Attention	A differentiable lookup that returns a weighted average of values, with weights from query–key similarity; see Definition 8.6.
Backtesting	Evaluation by rolling the forecast origin forward, so that every test block lies strictly after its training block.
Causal mask	Setting scores for future positions to $-\infty$ before the softmax, so a forecaster cannot attend to its own future.
Constant error carousel	The additive memory path in a gated cell, along which the gradient passes multiplied only by forget-gate activations.
Forecast origin	The time t at which the forecast is made; everything after it is unavailable by construction.
Gated residual network	The TFT’s universal wrapper, a residual block whose branch is gated so it can collapse to the identity.
Horizon	The number of future steps H to be predicted; set by the decision the forecast supports, not tuned.
Known-future covariate	A feature available for the whole horizon at the forecast origin, typically calendar-derived.
Lookback	The number of past steps L the model sees; read off the autocorrelation function.
MASE	Mean absolute error divided by that of seasonal naïve; scale-free, and below one exactly when you beat the baseline.
Multi-head attention	Several attention operations in parallel on different projections, concatenated, so distinct notions of relevance can coexist.
Past-observed covariate	A measured feature available only up to the forecast origin.
Pinball loss	The asymmetric loss whose minimiser is the q -th quantile; see Proposition 8.1.
Seasonal naïve	The baseline $\hat{y}_{t+h} = y_{t+h-m}$; the reference against which MASE is defined.
Sliding window	The construction that turns a series into supervised $(\mathbf{X}, \mathbf{y})$ pairs.
Static covariate	A feature constant over time, such as an agent identity embedding.
Variable selection network	A GRN plus softmax producing per-instance input weights, giving feature importance for free.

Further Reading

[Géron \(2022\)](#), chapter 15, is the closest companion to this chapter and uses the same TensorFlow and Keras stack; it covers windowing, recurrent layers and the convolutional front-end in roughly the order followed here, and its treatment of `timeseries_dataset_from_array` complements Listing 8.1. [Bishop and Bishop \(2024\)](#), sections 12.1 and 12.2, gives the more theoretical account of recurrence and of the bottleneck that motivates attention, and chapter 12 as a whole is the natural bridge into Chapter 9.

On the cells themselves, the original LSTM paper of [Hochreiter and Schmidhuber \(1997\)](#) is still worth reading for the clarity of its diagnosis, and the gating argument of Theorem 8.1 is easier to follow there than in most later summaries; [Cho et al. \(2014\)](#) introduces the GRU as part of an encoder–decoder, so it also gives the seq2seq structure in its original setting. [Bahdanau et al. \(2015\)](#) is the paper that invented attention as a repair for the bottleneck of Section 8.7, and reading it before [Vaswani et al. \(2017\)](#) makes the later paper feel inevitable rather than arbitrary.

On forecasting practice rather than architecture, [Hyndman and Athanassopoulos \(2021\)](#) is free online, and its chapters on decomposition, baselines and evaluation are the best short treatment of the material in Sections 8.2 and 8.3 available anywhere; read it before believing any deep-forecasting result, including your own. [Lim et al. \(2021\)](#) develops the Temporal Fusion Transformer and is unusually readable for an architecture paper, with the interpretability experiments repaying attention. For the regulatory material the primary instruments are [Republic of Kenya \(2019\)](#) and [Central Bank of Kenya \(2022\)](#), and [Huyen \(2022\)](#) treats the drift and monitoring concerns that Chapters 10 and 11 develop in full.

Part III

**MLOps and Production
Systems**

Appendices

Capstone Project

Solutions to Exercises

This appendix collects worked solutions, one section per chapter. Solutions are deliberately fuller than a marking scheme requires: they model the style of argument, framing before formalism, business units before metrics, honesty caveats stated explicitly, that the assignments and the capstone are graded against.

Chapter 1: Introduction to Machine Learning

Solution to Exercise 1 (Framing). (a) Streaming fraud detection with a 300-alerts/day operations team.

Mitchell specification.

- *T*: score each transaction at initiation, before settlement, and emit an alert for those most likely to be fraudulent.
- *E*: historical transaction logs with fraud labels arriving with a delay of days to weeks, from chargebacks, customer complaints, and investigator dispositions; labels are incomplete (undetected fraud is labelled “legitimate”).
- *P*: fraud value captured (or recall) *at a fixed budget of 300 alerts per day* — operationally, precision@*k* with $k = 300/\text{day}$, weighted by transaction value.

Framing block. **Objective:** minimize net fraud losses subject to the investigation-capacity constraint and a false-positive customer-friction budget. **ML task:** binary classification (or anomaly scoring) on a *stream*; the drift speed of adversarial behaviour argues for online or frequently retrained batch learning (Table 1.4), evaluated prequentially. **Metric:** value-weighted recall at the 300-alert budget; secondary, time-to-detect for new fraud patterns. **Baseline:** the incumbent rules engine — its capture rate at the same alert volume is the number to beat. **Constraints:** per-transaction

scoring latency in the tens of milliseconds; adversarial drift; delayed and censored labels; rollback capability for bad online updates.

Two honesty points earn full marks: the label censoring (undetected fraud contaminates the negatives, so measured recall is optimistic), and the feedback loop — blocking transactions changes which frauds are ever observed, the streaming analogue of the selective-labels problem of [Section 1.4](#).

(b) Thirty-day readmission from clinical pathway data.

Mitchell specification.

- *T*: at discharge, estimate the probability that the patient is readmitted within 30 days.
- *E*: historical admission episodes — diagnoses, procedures, length of stay, vitals trajectories, prior utilization — joined to the observed readmission outcome.
- *P*: recall (or expected cost captured) at the capacity of the post-discharge intervention programme (follow-up calls, home visits); PR-AUC as the threshold-free summary, given a low base rate.

Framing block. **Objective:** allocate a fixed post-discharge intervention capacity to the patients whose readmission it is most likely to prevent. **ML task:** binary classification at the discharge-decision point; batch retraining is adequate (clinical drift is slow relative to fraud). **Metric:** recall at the intervention budget; per-segment calibration, since resource allocation across wards and demographics will be scrutinized. **Baseline:** the incumbent clinical risk score (e.g. a LACE-style index) or current clinician triage. **Constraints:** explainability to clinicians; data-protection compliance for health records; fairness across demographic groups; label subtleties — death within 30 days is a competing risk that must be handled explicitly, not silently coded as “no readmission”.

The deepest framing observation: the deployment intent is *interventional* (we act on high-risk patients), so post-deployment outcome data is shaped by the model’s own decisions — future retraining must account for the intervention, exactly the hidden feedback loop of [Section 1.4](#).

Solution to Exercise 2 (ML or not).

(a) Blocking transactions above a regulatory limit: rule. The requirement is a rule — crisp, complete, auditable, and legally fixed. ML could only approximate it, adding error where zero error is mandated and destroying auditability. Right-hand column of [Table 1.1](#) on every criterion.

(b) SIM-swap account takeover: ML, wrapped in rules. The signal is high-dimensional and behavioural (device changes, dormancy-then-drain patterns, atypical beneficiaries), and the adversary adapts, so any static rulebook rots within months — the left-hand column on every criterion.

The mature design keeps hard rules as guardrails (e.g. mandatory step-up authentication within N hours of a SIM change) with an ML risk score ranking the grey zone; the rules also generate the labelled outcomes that train the model, the “rules first, ML second” pattern of [Section 1.2.3](#).

(c) Routing support tickets by language: ML, but off the shelf. Hand-written rules (character ranges, keyword lists) are brittle across code-switching and short texts, so learning is justified; but language identification is a solved commodity problem, and a lightweight pretrained classifier meets the accuracy, latency, and cost bar. The build-or-not lesson: “use ML” does not imply “train your own” — the total cost-of-ownership argument of [Table 1.1](#) favours the smallest maintained artifact that clears the requirement.

Solution to Exercise 3 (ERM: mean and median as optimal constants).

Squared error. For a constant predictor c , the empirical risk is

$$J(c) = \frac{1}{n} \sum_{i=1}^n (y_i - c)^2, \quad J'(c) = -\frac{2}{n} \sum_{i=1}^n (y_i - c),$$

and setting $J'(c) = 0$ gives $c^* = \frac{1}{n} \sum_i y_i = \bar{y}$, the sample mean; $J''(c) = 2 > 0$ confirms the minimum.

Absolute error. $G(c) = \frac{1}{n} \sum_i |y_i - c|$ is convex and piecewise linear, so we argue by subgradient. For c not equal to any y_i ,

$$G'(c) = \frac{1}{n} \left(\#\{i : y_i < c\} - \#\{i : y_i > c\} \right),$$

which is negative while fewer than half the points lie below c and positive once more than half do. The minimum is therefore attained where the two counts balance: at the sample median (for even n , anywhere on the interval between the two middle order statistics — the minimizer need not be unique).

Connection to [Section 1.6.2](#). Minimizing RMSE trains the model toward the conditional *mean*, which is pulled by extreme outcomes; minimizing MAE trains it toward the conditional *median*, which is robust to them. Choosing between [Eq. \(1.4\)](#) is therefore choosing *which summary of the target distribution the model learns* — a business decision (how do errors actually cost?) dressed as a metric choice.

Solution to Exercise 4 (Cost-sensitive thresholds).

Two-way derivation. For an applicant with calibrated default probability $\hat{p}$, compare conditional expected costs:

$$\mathbb{E}[C \mid \text{approve}] = \hat{p} c_{\text{FN}}, \quad \mathbb{E}[C \mid \text{decline}] = (1 - \hat{p}) c_{\text{FP}}.$$

Decline exactly when $\hat{p} c_{\text{FN}} > (1 - \hat{p}) c_{\text{FP}}$; rearranging,

$$\hat{p} > \frac{c_{\text{FP}}}{c_{\text{FP}} + c_{\text{FN}}} = t^*,$$

which is Eq. (1.6). The derivation *uses* calibration: $\hat{p}$ must be interpretable as a probability for the expectations to be meaningful, which is why calibration is a precondition, not a nicety.

Three-way extension. Add the action *refer*, costing a review fee c_R and (idealizing) resulting in the correct decision, so it incurs no error cost. The three conditional expected costs are $\hat{p} c_{FN}$ (approve), $(1 - \hat{p}) c_{FP}$ (decline), and c_R (refer). Comparing pairwise:

- approve beats refer iff $\hat{p} c_{FN} < c_R$, i.e. $\hat{p} < t_L := c_R / c_{FN}$;
- decline beats refer iff $(1 - \hat{p}) c_{FP} < c_R$, i.e. $\hat{p} > t_U := 1 - c_R / c_{FP}$.

The optimal policy is therefore: *approve* if $\hat{p} < t_L$, *refer* if $t_L \leq \hat{p} \leq t_U$, *decline* if $\hat{p} > t_U$. The referral band is non-empty iff $t_L < t_U$, i.e.

$$c_R < \frac{c_{FN} c_{FP}}{c_{FN} + c_{FP}},$$

the harmonic combination of the two error costs; cheap review widens the band, expensive review collapses it to the single threshold t^* — which, one checks, always lies inside $[t_L, t_U]$ when the band exists. This is exactly the “ $\hat{p}$ near t^* routes to a human” edge of Fig. 1.9, now with the band derived rather than asserted. (A fuller model would let the human err too; the band then narrows in proportion to reviewer accuracy, a good extension discussion.)

Solution to Exercise 5 (Leakage audit).

(a) Number of repayment reminders sent: target leakage. Reminders exist only after origination and are triggered by delinquency — the feature is a downstream *consequence* of the label. At application time it is identically empty, so a model trained with it learns to read the outcome off its own paperwork and collapses in production (Definition 1.3).

(b) Median mobile-money balance over the 90 days before application: safe — conditional on discipline. The window $[t_0 - 90d, t_0)$ respects the as-of contract of Table 1.6. The residual risks to state for full marks: the feature must be computed from event data as it stood at t_0 (late-arriving corrections or restated ledgers re-introduce temporal leakage), and the training-time computation must share code with the serving-time computation or training/serving skew replaces leakage as the failure mode.

(c) Standard deviation of balance over the loan term: temporal leakage. The loan term lies strictly *after* the origination decision the model is meant to inform. The feature is computable in the historical warehouse (which is why it slips into training sets) but can never be computed at prediction time. Textbook violation of the as-of rule.

(d) Merchant-ID one-hot with random row-wise splits: group leakage. Repeat merchants appear on both sides of a random split, so the identifier

lets the model memorize *who* defaulted rather than learn *which behaviours* predict default; validation scores inflate and the model is useless for the new applicants it exists to score (whose IDs it has never seen). Remedies: split by merchant group *and* time; drop the raw identifier in favour of behavioural aggregates with proper as-of semantics; if entity-level history is genuinely predictive, encode it as backdated features (e.g. prior loans, prior repayment record as of t_0), which are legitimate.

Solution to Exercise 6 (Bias–variance: the vanishing cross-terms).

Setup as in Section 1.9.1: $y = g(x) + \varepsilon$ with $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$, ε independent of the training sample S ; $\hat{f}$ is trained on S ; $\bar{f}(x) = \mathbb{E}_S[\hat{f}(x)]$. Fix x and expand (all expectations over S and ε jointly):

First cross-term. Expanding $\mathbb{E}[(g + \varepsilon - \hat{f})^2] = \mathbb{E}[(g - \hat{f})^2] + 2\mathbb{E}[\varepsilon(g - \hat{f})] + \mathbb{E}[\varepsilon^2]$, the middle term factorizes because ε is independent of S and $\hat{f}$ is a function of S (and $g(x)$ is a constant for fixed x):

$$\mathbb{E}[\varepsilon(g - \hat{f})] = \mathbb{E}[\varepsilon] \mathbb{E}_S[g - \hat{f}] = 0 \cdot \mathbb{E}_S[g - \hat{f}] = 0.$$

Independence is used to factorize; the zero mean of ε kills the product. This leaves $\mathbb{E}[(g - \hat{f})^2] + \sigma^2$.

Second cross-term. Insert and subtract $\bar{f}$: $(g - \hat{f}) = (g - \bar{f}) + (\bar{f} - \hat{f})$, so

$$\mathbb{E}_S[(g - \hat{f})^2] = (g - \bar{f})^2 + 2(g - \bar{f}) \mathbb{E}_S[\bar{f} - \hat{f}] + \mathbb{E}_S[(\bar{f} - \hat{f})^2].$$

Here $(g - \bar{f})$ is a deterministic quantity (both terms are fixed numbers once x is fixed), so it exits the expectation; and by the *definition* of $\bar{f}$, $\mathbb{E}_S[\bar{f} - \hat{f}] = \bar{f} - \mathbb{E}_S[\hat{f}] = 0$. The cross-term vanishes, leaving $\text{Bias}^2 + \text{Variance} + \sigma^2$, which is Eq. (1.7). Summary of where each assumption bites: independence of ε and S kills the first cross-term; the definition of $\bar{f}$ as the S -average kills the second.

Solution to Exercise 7 (Splits, empirically).

A model answer on synthetic data with controlled drift (students' numbers on real data will differ in size, not in sign):

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import average_precision_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(8401)
n, d = 30_000, 10
t = np.sort(rng.uniform(0, 1, n))          # "application date" in [0,1]
X = rng.normal(size=(n, d))
w = np.array([2.0, -1.5, 1.0, 0.5, 0, 0, 0, 0, 0, 0])
```

```

drift = 2.5 * t                                     # concept drifts over time
logits = X @ w + drift * X[:, 3] - 3.2
y = rng.binomial(1, 1 / (1 + np.exp(-logits)))

model = Pipeline([("s", StandardScaler()),
                  ("m", LogisticRegression(max_iter=2_000,
                                           class_weight="balanced"))])

# (a) Random shuffled split -- time ignored
Xa_tr, Xa_te, ya_tr, ya_te = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=8401)
pa = model.fit(Xa_tr, ya_tr).predict_proba(Xa_te)[: , 1]

# (b) Temporal split -- train on the past, test on the future
cut = int(0.75 * n)                                # rows already time-ordered
pb = (model.fit(X[:cut], y[:cut])
      .predict_proba(X[cut:]))[: , 1]

print(f"random split PR-AUC: {average_precision_score(ya_te, pa):.3f}")
print(f"temporal split PR-AUC: {average_precision_score(y[cut:], pb):.3f}")

```

Listing B.1: Random vs. temporal split under drift: the random split reports an optimistic number that no deployment can honour.

Direction and explanation. The random split scores higher. Under shuffling, training and test rows are interleaved in time, so the training set contains examples from the *same regime* as every test example — the model has effectively seen the future, and the drift term never has to be extrapolated. The temporal split forces genuine forward prediction into a drifted regime, which is exactly what deployment does (Fig. 1.6). **The honest number is the temporal one;** the gap between the two is not noise but a first, free estimate of how much drift will cost in production — worth reporting in the decision memo of Section 1.10.

Solution to Exercise 8 (Fairness).

Let F_A, F_B be the score CDFs of the calibrated model $\hat{p}$ in the two regions, with base rates $\pi_A = \mathbb{E}[\hat{p} \mid A] = 0.03$ and $\pi_B = \mathbb{E}[\hat{p} \mid B] = 0.09$ (calibration plus the tower property force the group mean score to equal the group base rate). Approval is $\hat{p} \leq t$ for a single global t .

Impossibility across thresholds. Demographic parity at *every* threshold would mean $F_A(t) = F_B(t)$ for all t , i.e. identical score distributions — hence identical means, contradicting $0.03 \neq 0.09$. So there is no global-threshold policy family satisfying parity uniformly.

The single-threshold case. At one fixed t , parity requires the accident $F_A(t) = F_B(t)$ at that exact point. But any calibrated model that actually uses the risk information distinguishing the regions places more probability mass at higher scores for region B (in the natural case, F_B is stochastically dominated by — i.e. sits below — F_A), giving $F_B(t) < F_A(t)$: region B's approval rate is strictly lower at every useful threshold. Even in contrived

cases where the CDFs cross at some t , the coincidence is knife-edge — it fails for neighbouring thresholds and evaporates at the first retrain — so it cannot ground a policy. The structural conclusion: with unequal base rates, a calibrated score plus a single global threshold *cannot* deliver demographic parity; achieving parity would require group-specific thresholds, which raises its own legal and ethical questions, or abandoning calibration — the conflict promised in [Section 1.11](#).

Proposal to the regulator. A defensible position for the running case: retain per-group *calibration* (an uncalibrated score corrupts every downstream decision through [Eq. \(1.6\)](#)) and adopt *equal opportunity* — equal true-positive rates among repayers — as the operative criterion, i.e. good borrowers in both regions face the same approval odds; then justify any residual approval-rate gap by the base-rate difference while committing to close it upstream (product design, data enrichment for the thin-file region). **Post-deployment monitoring:** per-region approval rate, TPR/FPR among matured labels, calibration curves, and their drift over time — the fairness axis of the Week-13 monitors, reported segment-by-segment as [Section 1.11](#) prescribes.

[Solution to Exercise 9 \(Hybrid architecture\).](#)

A full-marks answer reproduces the topology of [Fig. 1.9](#) with the component classification made explicit:

- **Classical ML (the decision core).** The gradient-boosted or logistic scoring model with its leakage-safe feature pipeline ([Listing 1.5](#)), served behind the versioned API of [Listing 1.7](#). It alone emits $\hat{p}$; it is trained, calibrated, validated, and monitored under the discipline of Chapters 1–4 and 11–13.
- **LLM components (the language edges).** (i) A RAG service over lending policy, KYC regulation, and product documents, answering underwriter questions with citations; (ii) an extraction model converting unstructured inputs (statements, free-text loan purpose, chat) into *typed features* that enter the classical pipeline — validated like any other upstream data source; (iii) an agent orchestrating the workflow: gather data, call the scoring API, draft the reason-coded communication.
- **Human review.** Applications with $\hat{p} \in [t_L, t_U]$ — the referral band derived in Solution 4 — route to a credit officer, with the agent’s brief attached.

Where calibration must be preserved end-to-end — the marking crux: the probability $\hat{p}$ must flow *numerically unaltered* from the scorer through the API to the thresholding policy and into any human-facing brief. The

agent may *explain* the score; it must never *re-estimate*, round, re-rank, or paraphrase it into a different number, and no LLM output may feed back into the decision path — the moment a stochastic text generator sits between the calibrated score and the threshold of [Eq. \(1.6\)](#), the optimality of the policy, the audit trail, and the fairness monitoring of Solution 8 are all void. Diagrammatically: every arrow into the approve/refer/decline node originates at the classical scorer; LLM arrows terminate at humans or at the scorer's *inputs*, never at its output.

Chapter 2: Data Preparation and Feature Engineering

Solution to Exercise 2.1. Consider the electronic medical records of a network of Kenyan outpatient clinics.

MCAR. A batch of laboratory results is lost when the connection between the lab analyser and the records system fails for an afternoon. Which patients' rows are affected is determined by the timing of the outage, not by the patients or their values: $P(M \mid X) = P(M)$ ([Definition 2.1](#)). Mean imputation of the missing lab values yields an *unbiased* estimate of the population mean (the missing rows are a random sample), although it understates the variance (Exercise 2.3).

MAR. Blood-pressure readings are missing far more often at rural dispensaries that lack a working monitor than at urban clinics. Conditional on the *observed* facility-type column, missingness carries no further information about the reading itself ([Definition 2.2](#)). Unconditional mean imputation is *biased* whenever blood pressure differs systematically by facility type (urban and rural patient populations differ): the imputed mean is dominated by urban readings. Imputation that conditions on facility type — a group-wise mean, or the model-based imputers of the chapter — removes the bias.

MNAR. HIV-test results are missing because some patients decline testing, and the propensity to decline is associated with the patient's own (unobserved) status and risk perception: $P(M \mid X)$ depends on X_{mis} ([Definition 2.3](#)). Mean imputation is *biased*, and no imputation scheme built from the observed data alone removes the bias; per [Remark 2.1](#), the honest treatment records the refusal itself as an indicator feature and interprets downstream estimates with the selection mechanism in mind.

In summary, mean imputation is unbiased for the mean only under MCAR; under MAR it requires conditioning on the drivers of missingness; under MNAR it is biased regardless.

Solution to Exercise 2.2. Let $R_i \in \{0, 1\}$ indicate that row i is a complete case, and let X_i be its (complete-data) value. MCAR states $P(R \mid X) = P(R)$,

i.e. $R \perp X$. The density of X among complete cases is, by Bayes' rule,

$$f(x | R = 1) = \frac{P(R = 1 | X = x) f(x)}{P(R = 1)} = \frac{P(R = 1) f(x)}{P(R = 1)} = f(x),$$

where the middle equality uses independence. The complete cases are therefore distributed identically to the full population — a random subsample. Consequently, with $n_{cc} = \sum_i R_i$ the (random) number of complete cases,

$$\mathbb{E} \left[\frac{1}{n_{cc}} \sum_{i: R_i=1} X_i \mid n_{cc} > 0 \right] = \mathbb{E}[X],$$

by taking the expectation first conditionally on the pattern $(R_1, \dots, R_n)$ — under which each retained X_i has mean $\mathbb{E}[X]$ by independence — and then over the pattern. The complete-case mean is unbiased; the only cost of MCAR missingness is the loss of efficiency from the reduced sample size. $\square$

Solution to Exercise 2.3. With point-estimate imputation (`sample_posterior=False`), every imputed entry is set to the conditional-mean prediction $\hat{\mathbb{E}}[X_{\text{mis}} | X_{\text{obs}}]$ of the final regression sweep of Eq. (2.2). Decompose the true variance by the law of total variance:

$$\text{Var}(X) = \underbrace{\mathbb{E}[\text{Var}(X | X_{\text{obs}})]}_{\text{residual spread}} + \underbrace{\text{Var}(\mathbb{E}[X | X_{\text{obs}}])}_{\text{explained spread}}.$$

The imputed values realise only the second term: they sit exactly on the conditional-mean surface, with zero residual spread. The completed column's sample variance therefore converges to $\text{Var}(\mathbb{E}[X | X_{\text{obs}}])$, understating $\text{Var}(X)$ by the entire residual component $\mathbb{E}[\text{Var}(X | X_{\text{obs}})]$, and the understatement grows with the missing fraction.

Downstream, any procedure that treats the completed data as if observed — ordinary least squares, a t -test, a confidence interval for a feature mean — inherits standard errors that are too small: intervals are too narrow and p -values anti-conservative, because the analysis pretends the imputed entries carry the same information as real measurements. The remedies are those of proper multiple imputation: draw fills from the posterior (`sample_posterior=True`), create M completed datasets, and combine estimates with Rubin's rules, which add the between-imputation variance the point-estimate shortcut discards.

Solution to Exercise 2.4. Two mechanisms, both consistent with the chapter, explain a *drop* in AUC after trimming $|z| > 3$ rows.

(a) *The outliers were the signal.* In credit risk the extreme tail of behaviour is disproportionately informative about default: erratic whale deposits, balance-inflation spikes, abnormal withdrawal bursts (see the pitfall in

Section 2.3). Deleting $|z| > 3$ rows removes a non-random, minority-class-enriched slice of the training data — defaulters exhibit extreme behaviour more often than repayers — so the model loses precisely the discriminative region of the feature space, and its ranking quality on an untrimmed evaluation set falls.

(b) *The analyst created a training–serving mismatch.* Rows were removed from the *training* data, but the validation/test data (and, eventually, production traffic) still contain extreme rows. The model is now evaluated on a region of the input space it was never fitted on — a self-inflicted violation of the distribution-overlap requirement of **Section 2.6**. A subsidiary aggravating factor: on heavy-tailed transaction data the Gaussian $|z| > 3$ rule flags a large fraction of perfectly legitimate rows (the z-score’s μ and σ are themselves inflated by the tail), so the deletion is both extensive and systematically targeted at high-activity customers.

Solution to Exercise 2.5. From Eq. (2.7), the maximum maps to

$$x'_{\max} = \frac{x_{\max} - x_{\min}}{x_{\max} - x_{\min}} = 1$$

identically — the scaled value of the largest point is 1 whatever its magnitude, which is exactly why a single extreme outlier cannot be “detected” in the scaled data: it is normalised into the boundary.

Let m be the median. Its scaled value is $m' = (m - x_{\min}) / (x_{\max} - x_{\min})$. Replace the maximum by $10 x_{\max}$ (all other order statistics unchanged); the new scaled median is

$$m'' = \frac{m - x_{\min}}{10 x_{\max} - x_{\min}}, \quad \frac{m''}{m'} = \frac{x_{\max} - x_{\min}}{10 x_{\max} - x_{\min}}.$$

For $x_{\min} = 0$ this ratio is exactly $1/10$: the median’s scaled value collapses by a factor of ten, and with it the entire bulk of the distribution is crushed toward 0 while the outlier alone occupies the top of the unit interval. This is the fragility that motivates the robust scaler of **Eq. (2.8)**.

Solution to Exercise 2.6. Within level c , model each target as conditionally Bernoulli, $y_i \mid \theta_c \sim \text{Bernoulli}(\theta_c)$, and place the conjugate prior

$$\theta_c \sim \text{Beta}(\alpha, \beta), \quad \alpha = m \bar{y}, \quad \beta = m(1 - \bar{y}),$$

so that the prior mean is $\alpha / (\alpha + \beta) = \bar{y}$ (the global rate) and the prior strength — the number of pseudo-observations — is $\alpha + \beta = m$. Observing n_c rows of level c with $s_c = \sum_i y_i = n_c \bar{y}_c$ successes, conjugacy gives the posterior $\theta_c \mid \text{data} \sim \text{Beta}(\alpha + s_c, \beta + n_c - s_c)$, whose mean is

$$\mathbb{E}[\theta_c \mid \text{data}] = \frac{\alpha + s_c}{\alpha + \beta + n_c} = \frac{m \bar{y} + n_c \bar{y}_c}{m + n_c} = \text{TE}(c),$$

which is [Eq. \(2.11\)](#) exactly. The limits confirm the shrinkage behaviour: as $n_c \rightarrow 0$, $\text{TE}(c) \rightarrow \bar{y}$ (an unseen or rare level is pulled entirely to the prior), and as $n_c \rightarrow \infty$, $\text{TE}(c) \rightarrow \bar{y}_c$ (abundant evidence overwhelms the prior). The smoothing parameter m is thus interpretable as the number of global-prior pseudo-counts each level must “outweigh” before its own history dominates — and setting it too small is precisely what lets a level seen once memorise its own label ([Section 2.5.3](#)). $\square$

Solution to Exercise 2.7. *Mechanism.* Individually, a start date and an end date are weak predictors of churn. Jointly they encode the label almost perfectly at extraction time: an employee who has churned has a populated `contract_end_date` lying before the snapshot date, whereas a current employee has a null, sentinel, or future-dated value. Any model able to combine the two features (or a feature and the implicit snapshot date) reconstructs “has this contract already terminated?” — which is the churn label, or a near-deterministic proxy for it. The end date is a *post-outcome field*, written by the HR system as a consequence of the very event being predicted: cause 6 of [Section 2.7.2](#), hiding behind two innocuous-looking columns that pass a univariate correlation screen.

A screen for pairwise leakage. Extend the univariate screen of [Section 2.7](#) to constructed pairs: (i) for every pair of same-typed columns (two dates, two amounts), automatically generate their difference (and, for amounts, their ratio) and rank these derived features by $|\rho(\cdot, y)|$ or mutual information with the target; (ii) as a model-based variant, fit a depth-2 decision tree on each feature pair alone and flag any pair whose standalone AUC approaches the full model’s. Either variant catches start/end-date leakage immediately (the difference is tenure-to-termination; the tiny tree learns “end date < snapshot”). The screen complements, but does not replace, the lineage question — *which system writes this field, and when?* — that identifies post-outcome fields even when their statistical footprint is subtle.

Solution to Exercise 2.8. Assume the hash of [Eq. \(2.10\)](#) assigns each of the K levels independently and uniformly to one of d buckets. Two given levels collide with probability $1/d$, and there are $\binom{K}{2}$ pairs, so by linearity of expectation the expected number of colliding pairs is

$$\mathbb{E}[\text{\#collisions}] = \binom{K}{2} \frac{1}{d} = \frac{K(K-1)}{2d}.$$

For $K = 10^4$ and $d = 256$:

$$\frac{10^4 \times (10^4 - 1)}{2 \times 256} = \frac{99\,990\,000}{512} \approx 1.95 \times 10^5$$

colliding pairs — with an average bucket load of $K/d \approx 39$ levels, so *every* bucket is shared. This is not necessarily fatal (tree models tolerate the induced noise, and the load is spread evenly), but it quantifies the trade-off:

at this cardinality, $d = 256$ buys a fixed 256-column footprint at the price of heavy aliasing, and raising d to 2^{12} – 2^{14} reduces the expected pair collisions by one to two orders of magnitude while remaining far cheaper than a 10^4 -column one-hot encoding.

Solution to Exercise 2.9. *Leakage mechanism.* Support-line contact is not exogenous to repayment trouble. Two concrete routes: (i) borrowers who are already struggling call to ask about extensions, penalties, or restructuring *after* the loan being scored has been disbursed — so if the 7-day window is computed relative to label-observation time (or to “now” in the training warehouse) rather than to the scoring timestamp t , the feature ingests post- t information in direct violation of the as-of rule (Eq. (2.1)); (ii) worse, some inbound calls are *returns of collections outreach* — the collections team calls delinquent borrowers, who call back — making the count a laundered copy of the delinquency flag, exactly the post-outcome pattern of `days_since_collections_call` in Listing 2.11 (cause 6 of Section 2.7.2).

Redesign. Keep the legitimate signal (a customer’s baseline propensity to need support may genuinely correlate with risk) by: fixing the window to end strictly before the scoring timestamp, $[t - 7d, t)$, enforced by the feature platform’s point-in-time semantics (Section 2.8); excluding call types causally downstream of the credit process (collections callbacks, repayment-difficulty queries about an *active* loan), which requires the call-category field and its write-time lineage; and validating the rebuilt feature with the correlation screen — its association with the target should fall from “implausible” to “modest” if the leak, and not the baseline behaviour, was carrying the load.

Solution to Exercise 2.10. A five-sentence rebuttal. First, native NaN routing (Chen and Guestrin, 2016) solves only the mechanical problem of fitting a tree through incomplete rows; it does not answer the diagnostic questions — *why* is the value missing, and is the missingness itself predictive? — that Lab 2’s MCAR/MAR/MNAR audit exists to settle. Second, under MNAR the explicit indicator of Remark 2.1 is a first-class, interpretable feature whose effect can be measured, reviewed for fairness, and explained to a credit officer, whereas a learned default branch direction is an implicit, per-split artefact that cannot. Third, the pipeline must remain model-agnostic: the moment we benchmark against a regulatory logistic scorecard (Eq. (2.13)) or a neural baseline (Chapter 6), models without native handling need the imputation stage anyway, and comparing models across different missing-data treatments confounds the comparison. Fourth, training-serving consistency (Section 2.8) requires that whatever missing-value semantics the model learned be reproduced exactly in the online path — a fitted, serialised imputer inside the Pipeline of Listing 2.12 is a deployable contract, while “the library handles it” is not. Fifth, most of Lab 2’s effort — parsing, validity rules, entity resolution, leakage

hunting — is not imputation at all, and no amount of native NaN support dedupes a customer with three SIM cards.

Chapter 3: Classical Machine Learning and Rigorous Evaluation

Solution to Exercise 3.1. A defensible committee-minute sentence: “*Holding every other feature in the model fixed, applicants with late-night transaction activity have $e^{0.62} \approx 1.86$ times the odds of default of otherwise-identical applicants without it.*” The load-bearing words are *odds* and *holding every other feature fixed*.

The important caveat: the coefficient measures a *conditional association*, not a *causal effect*. It is a partial effect within this particular model and this particular preprocessing pipeline (Chapter 2): add, remove, or re-encode correlated features and the coefficient — and hence the “effect” of late-night activity — can change substantially. It licenses no intervention claim (“discouraging late-night transactions would reduce defaults” does not follow). A secondary caveat worth a footnote in the minute: an odds ratio is not a risk ratio. At the portfolio’s 3% base rate the two are numerically close ($OR \approx RR$ when the outcome is rare), but the identification should still be avoided in a document regulators may read.

Solution to Exercise 3.2. *Derivative of the sigmoid.* With $\sigma(z) = (1 + e^{-z})^{-1}$,

$$\frac{\partial \sigma}{\partial z} = \frac{e^{-z}}{(1 + e^{-z})^2} = \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)).$$

Gradient of the log loss. Write the per-example loss with $\hat{p} = \sigma(z)$, $z = \theta^\top \mathbf{x}$:

$$\ell(\theta) = -y \log \hat{p} - (1 - y) \log(1 - \hat{p}).$$

By the chain rule,

$$\frac{\partial \ell}{\partial z} = -\frac{y}{\hat{p}} \hat{p}(1 - \hat{p}) + \frac{1 - y}{1 - \hat{p}} \hat{p}(1 - \hat{p}) = -y(1 - \hat{p}) + (1 - y) \hat{p} = \hat{p} - y,$$

and since $\partial z / \partial \theta = \mathbf{x}$, $\nabla_{\theta} \ell = (\hat{p} - y) \mathbf{x}$. Averaging over the m training examples gives Eq. (3.4): $\nabla_{\theta} J = \frac{1}{m} \sum_i (\hat{p}_i - y_i) \mathbf{x}_i$. The remarkable cancellation of the $\hat{p}(1 - \hat{p})$ factor is a signature of pairing the sigmoid with its *matched* loss (cross-entropy); the same cancellation reappears at the output layer of every neural classifier in Chapter 6.

Solution to Exercise 3.3. From Eq. (3.11), $F_{\beta} = (1 + \beta^2)PR / (\beta^2 P + R)$.

Limit $\beta \rightarrow 0$: the expression tends to $PR/R = P$ directly.

Limit $\beta \rightarrow \infty$: divide numerator and denominator by β^2 :

$$F_\beta = \frac{(1 + \beta^{-2}) PR}{P + \beta^{-2} R} \xrightarrow{\beta \rightarrow \infty} \frac{PR}{P} = R.$$

So β interpolates between a pure-precision and a pure-recall score, which is exactly why it should be set from the cost structure.

Numerical part. With $P = 0.40$, $R = 0.75$, $\beta = 2$:

$$F_2 = \frac{(1 + 4) \cdot 0.40 \cdot 0.75}{4 \cdot 0.40 + 0.75} = \frac{1.50}{2.35} \approx 0.638,$$

noticeably above $F_1 = 2 \cdot 0.3/1.15 \approx 0.522$ because F_2 rewards this classifier's recall-heavy profile.

Solution to Exercise 3.4. Fix an instance with calibrated probability p of being positive. The two actions have expected costs

$$\mathbb{E}[\text{cost} \mid \text{flag}] = p c_{TP} + (1 - p) c_{FP}, \quad \mathbb{E}[\text{cost} \mid \text{pass}] = p c_{FN} + (1 - p) c_{TN}.$$

Flagging is optimal iff $p c_{TP} + (1 - p) c_{FP} \leq p c_{FN} + (1 - p) c_{TN}$, i.e.

$$(1 - p) (c_{FP} - c_{TN}) \leq p (c_{FN} - c_{TP}).$$

Under the “reasonableness” conditions $c_{FP} > c_{TN}$ and $c_{FN} > c_{TP}$ (each error costs more than the corresponding correct decision — otherwise one action dominates and no threshold exists), both bracketed terms are positive and we may divide:

$$t^* = \frac{c_{FP} - c_{TN}}{(c_{FP} - c_{TN}) + (c_{FN} - c_{TP})}.$$

Setting $c_{TP} = c_{TN} = 0$ recovers Eq. (3.13). Only cost *differences* within each true class matter — adding a constant to a whole row of the cost matrix leaves t^* unchanged. Concretely: if every flag triggers a manual review costing KES 200 regardless of outcome ($c_{TP} = c_{FP, \text{new}} - c_{FP, \text{old}} = 200$ added to the flag column), then with the case-study numbers $t^* = (1000)/(1000 + 9800) \approx 0.093$: review costs push the threshold up, and the scorer should flag slightly less eagerly.

Solution to Exercise 3.5. Let the positive scores be $s_1^+, \dots, s_{m_+}^+$ and the negative scores $s_1^-, \dots, s_{m_-}^-$, and assume first that all scores are distinct. Sort all $m_+ + m_-$ instances by decreasing score and trace the empirical ROC curve from $(0, 0)$: each positive encountered moves the curve *up* by $1/m_+$, each negative moves it *right* by $1/m_-$.

Consider the horizontal step contributed by a particular negative j . Its height above the FPR axis at that moment is

$$(\text{number of positives already passed})/m_+ = \#\{i : s_i^+ > s_j^-\}/m_+.$$

The area of that step is therefore $\#\{i : s_i^+ > s_j^-\} / (m_+ m_-)$. Summing over all negatives,

$$\text{AUC} = \frac{1}{m_+ m_-} \sum_{j=1}^{m_-} \sum_{i=1}^{m_+} \mathbb{1}[s_i^+ > s_j^-] = \frac{U}{m_+ m_-},$$

where $U = \sum_{i,j} \mathbb{1}[s_i^+ > s_j^-]$ is precisely the Mann–Whitney U statistic of the positive-class scores. Dividing by the number of pairs turns the count into the empirical probability $P(s^+ > s^-)$ for a uniformly random (positive, negative) pair, which is [Proposition 3.2](#).

Ties. If k_+ positives and k_- negatives share a score, the ROC convention traces a straight diagonal segment through the tied block, contributing area equal to what the pairs would contribute at $\frac{1}{2}$ each; correspondingly U is defined with $\frac{1}{2}$ per tied pair. This yields the tie-corrected statement $\text{AUC} = P(s^+ > s^-) + \frac{1}{2}P(s^+ = s^-)$ of [Eq. \(3.15\)](#).

Solution to Exercise 3.6. Suppose scores are independent of labels. Fix any threshold t and let $A = \{\mathbf{x} : s(\mathbf{x}) \geq t\}$ be the alert set. By independence, the label distribution *inside* A equals the marginal label distribution, so

$$\mathbb{E}[\text{Precision}(t)] = P(y = 1 \mid \mathbf{x} \in A) = \pi,$$

the prevalence — at *every* threshold, hence at every achievable recall level. The PR “curve” of a label-independent scorer is therefore the horizontal line at height π , and its area (average precision) is

$$\text{AP} = \int_0^1 \pi d(\text{recall}) = \pi.$$

On the case-study portfolio $\pi \approx 0.03$, so chance PR–AUC ≈ 0.03 . Reporting “PR–AUC = 0.31” without the baseline invites two opposite misreadings: against an implicit 0.5 anchor (borrowed from ROC intuition) it looks poor, whereas against the true chance level it is a tenfold lift. The convention of [Section 3.4.3](#) — always state PR–AUC *together with* the prevalence — exists to force the correct reading.

Solution to Exercise 3.7. From [Eq. \(3.17\)](#), $\text{se} = \sqrt{r(1-r)/n_+} \leq 0.02$ requires

$$n_+ \geq \frac{r(1-r)}{\text{se}^2} = \frac{0.7 \times 0.3}{0.02^2} = \frac{0.21}{0.0004} = 525 \text{ positives.}$$

At 3% prevalence that means $525/0.03 = 17,500$ validation rows — for the validation split *alone*, before training data is even counted.

The implication is sobering and is the entire motivation of [Section 3.6](#): on a small credit portfolio (say 10,000 labelled loans in total, ~ 300 defaulters), *no single split can estimate recall to within two points*. The honest options

are exactly the chapter’s toolkit: aggregate information across stratified folds (Definition 3.7), attach bootstrap intervals to whatever held-out estimate is reported (Section 3.6.4), and refuse to interpret differences smaller than the resulting uncertainty band.

Solution to Exercise 3.8. With $n_{01} = 63$ (only model B correct) and $n_{10} = 41$ (only model A correct), Eq. (3.19) gives

$$\chi^2 = \frac{(|63 - 41| - 1)^2}{63 + 41} = \frac{21^2}{104} = \frac{441}{104} \approx 4.24.$$

Since $4.24 > \chi^2_{1,0.95} = 3.84$, we reject at the 5% level the null hypothesis that the two models have equal error rates: model B’s advantage on the discordant cases is unlikely to be chance ($p \approx 0.04$).

The 9,800 concordant cases are irrelevant because they carry no information about the *difference* between the models: whether both are right or both are wrong on a case, that case shifts both models’ accuracies by the same amount. McNemar’s test conditions on the set of disagreements and asks only whether the 63:41 split is compatible with a fair coin — which is also why the test remains valid on heavily imbalanced test sets where accuracy itself is uninformative (Section 3.3.1).

Solution to Exercise 3.9. Applying Eq. (3.20) coordinate-wise with $\lambda = 0.4$:

$$(1, 0, 0) + 0.4 [(0, 1, 0) - (1, 0, 0)] = (1, 0, 0) + 0.4 (-1, 1, 0) = (0.6, 0.4, 0).$$

The result is invalid as a data point because a one-hot block encodes a *categorical* fact — the applicant’s region — and $(0.6, 0.4, 0)$ asserts membership of 60% in one region and 40% in another. No real applicant generates such a row; it violates the encoder’s sum-to-one, single-one structure, and any coefficient interpretation (Example 3.1) built on “the region-A indicator” silently breaks.

SMOTE-NC’s repair. SMOTE-NC interpolates only the *numeric* coordinates via Eq. (3.20), and sets each *categorical* feature of the synthetic point to the majority value of that feature among the k minority-class nearest neighbours (with the categorical features also entering the distance computation through a fixed penalty term). The synthetic row is then a legal row: valid one-hot blocks, interpolated numerics.

Residual risk. The synthetic point marries numeric values interpolated between two (possibly different-region) borrowers with a categorical value voted from the neighbourhood, so it can realise *combinations* of features that never co-occur in reality — e.g. transaction magnitudes typical of Nairobi CBD attached to a rural-county label. The class-conditional joint distribution is thereby distorted, and, as Section 3.7.2 already warned for plain

SMOTE, the altered class prior means the resulting scores are not calibrated: recalibrate (Section 3.5) before applying Proposition 3.1.

Solution to Exercise 3.10. Let $w = c_{FN}/c_{FP}$ and suppose training re-weights each positive example by w (equivalently, imagine each positive duplicated w times). For a scorer that is calibrated *on the re-weighted distribution*, the probability it reports is the re-weighted posterior. If p denotes the true (original-distribution) posterior $P(y = 1 \mid \mathbf{x})$, re-weighting the positive class by w transforms it to

$$p_w = \frac{w p}{w p + (1 - p)},$$

since the class-conditional densities are untouched and only the effective prior odds are multiplied by w . Thresholding the weighted model at 0.5 flags iff

$$p_w \geq \frac{1}{2} \iff w p \geq 1 - p \iff p \geq \frac{1}{1 + w} = \frac{c_{FP}}{c_{FP} + c_{FN}} = t^*,$$

which is exactly the unweighted model thresholded at t^* of Eq. (3.13). This is Elkan (2001)'s theorem: for Bayes-optimal (calibrated) scorers, cost-proportionate re-weighting and threshold moving induce the *same* decision rule.

Why try threshold moving first, then? Because the equivalence is exact only in the idealised limit, and every practical consideration favours the decision-level route. (i) It is *free and reversible*: no retraining, and the cost matrix can be revised — or differ across products — while one model serves them all, each business line simply reading a different operating point off the same score. (ii) It *preserves calibration*: re-weighting deliberately distorts the class prior, so the weighted model's outputs are no longer probabilities on production traffic and must be recalibrated before anyone prices risk with them. (iii) With finite samples and misspecified models the two routes genuinely diverge, and re-weighting can degrade the ranking it was meant to help. (iv) It is *auditable*: “we score with model M and decline above 7.4% calibrated risk, derived from this cost matrix” is a one-line policy a regulator can inspect. Escalate to weighting or resampling only when the evidence says the *ranking itself* — not the decision rule — is the weak link (Section 3.7.1).

Chapter 4: Decision Trees, Ensembles and Hyperparameter Tuning

Solution to Exercise 4.1. *Independent voters.* With $p = 0.55$, the number of correct voters is $K \sim \text{Binomial}(M, 0.55)$ and the majority is correct when

$K > M/2$. The normal approximation $K \approx \mathcal{N}(Mp, Mp(1-p))$ is justified for these M by the usual rule $Mp(1-p) \gg 1$ (it equals 2.7, 25.0 and 247.7 respectively; for $M = 11$ the approximation is rough and the exact sum is preferable, but it errs by under two points here). Writing $z = (M/2 - Mp) / \sqrt{Mp(1-p)}$ and $P_{\text{maj}} \approx 1 - \Phi(z) = \Phi(-z)$:

M	$-z$	P_{maj} (normal)	P_{maj} (exact)
11	0.332	0.630	0.633
101	1.005	0.843	0.846
1001	3.164	0.9992	0.9992

The committee converges on certainty, exactly as [Proposition 4.1](#) promises.

Correlated blocks. If the M voters consist of ten blocks of $M/10$ perfectly correlated clones, every clone in a block casts the same vote, so the committee behaves as *ten* independent voters, whatever M is. For $M = 1001$ (≈ 100 clones per block) the relevant computation is the $M = 10$ (even; take $M = 11$ as the odd comparator) row, not the $M = 1001$ row: $P_{\text{maj}} \approx 0.63$, and no increase of M at fixed block structure improves it. The *effective committee size* is the number of independent error patterns, not the number of members. This is [Eq. \(4.5\)](#) in disguise — within-block correlation $\rho = 1$ contributes pure floor — and it is precisely the situation of a forest whose trees share their root split: trees that agree on the first partition of the data are partial clones, adding `n_estimators` buys little, and the productive intervention is decorrelation (`max_features`), not more trees.

Solution to Exercise 4.2. Let $\bar{f} = \sum_m c_m f_m$ with $\sum_m c_m = 1$, $c_m \geq 0$, $\text{Var}[f_m] = \sigma^2$ and pairwise correlation ρ . Then

$$\begin{aligned} \text{Var}[\bar{f}] &= \sum_m c_m^2 \sigma^2 + \sum_{m \neq m'} c_m c_{m'} \rho \sigma^2 \\ &= \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \left(\sum_m c_m \right)^2 \right] = \sigma^2 \left[(1 - \rho) \sum_m c_m^2 + \rho \right], \end{aligned}$$

using $\sum_{m \neq m'} c_m c_{m'} = (\sum_m c_m)^2 - \sum_m c_m^2 = 1 - \sum_m c_m^2$. Since $\rho \leq 1$, minimising the variance means minimising $\sum_m c_m^2$ subject to $\sum_m c_m = 1$. By Cauchy–Schwarz (or a Lagrange multiplier), $\sum_m c_m^2 \geq \frac{1}{M} (\sum_m c_m)^2 = \frac{1}{M}$, with equality iff $c_m = 1/M$ for all m : *uniform weights are optimal*, and they recover [Proposition 4.2](#).

The optimality is conditional on the exchangeability assumptions. If the members have *unequal* variances — equivalently, unequal skill, as in [Listing 4.2](#) where a tuned LightGBM is simply a better estimator than the logistic baseline — the same Lagrangian calculation with member variances σ_m^2 (and $\rho = 0$ for clarity) gives optimal weights $c_m \propto 1/\sigma_m^2$: precision weighting. Soft voting with unequal weights is the practical, integer-granularity version of this, and stacking ([Section 4.8](#)) is its data-driven completion — the meta-learner *estimates* the weights instead of assuming exchangeability.

Solution to Exercise 4.3. *The limit without logarithms.* From $1 - x \leq e^{-x}$ with $x = \frac{1}{n}$,

$$\left(1 - \frac{1}{n}\right)^n \leq e^{-1}.$$

For the matching lower bound, apply $1 - x \geq e^{-x/(1-x)}$ (valid for $0 \leq x < 1$, and itself a rearrangement of $1 + u \leq e^u$ with $u = x/(1-x)$) to get $\left(1 - \frac{1}{n}\right)^n \geq e^{-n \cdot \frac{1/n}{1-1/n}} = e^{-n/(n-1)} \rightarrow e^{-1}$. The two bounds pinch the sequence to e^{-1} . For $n = 20$, $\left(\frac{19}{20}\right)^{20} = 0.3585$, already within 2.5% of $e^{-1} \approx 0.3679$ — the asymptotics arrive early, which is why “37% out of bag” is a safe rule of thumb even for modest samples.

Why an OOB-chosen threshold misbehaves. (i) *Statistical:* each OOB prediction $\bar{f}_{-i}^{\text{OOB}}(x_i)$ in Eq. (4.6) averages only the $\approx 0.37M$ trees that missed row i , whereas the deployed model averages all M . A mean of $0.37M$ trees is a noisier, differently calibrated score than a mean of M trees, so the probability distribution on which the OOB threshold was optimised is not the distribution the production model emits; with the asymmetric costs of the case study (KES 10,000 vs KES 800), even a small calibration shift moves the cost-optimal cut-off materially. (ii) *Temporal:* the OOB estimate randomises over rows, so its implicit population is the historical mixture of the training window. The threshold’s economics depend on the *future* default base rate and score distribution, which drift; the temporal validation window of Chapter 3 is the only estimate that rehearses the deployment condition. In short, OOB may rank models acceptably, but the decision threshold should be set — and re-set, per Chapter 11 — on out-of-time data.

Solution to Exercise 4.4. After stage m , the new weight mass on the rows that f_m got wrong is

$$W_{\text{wrong}}^{(m+1)} = \sum_{i: f_m(x_i) \neq y_i} w_i^{(m)} e^{+\alpha_m} = e^{\alpha_m} \varepsilon_m W^{(m)}, \quad W_{\text{corr}}^{(m+1)} = e^{-\alpha_m} (1 - \varepsilon_m) W^{(m)},$$

where $W^{(m)} = \sum_i w_i^{(m)}$. Substituting $e^{\alpha_m} = \sqrt{(1 - \varepsilon_m)/\varepsilon_m}$ from Proposition 4.4,

$$W_{\text{wrong}}^{(m+1)} = \sqrt{\frac{1 - \varepsilon_m}{\varepsilon_m}} \varepsilon_m W^{(m)} = \sqrt{\varepsilon_m (1 - \varepsilon_m)} W^{(m)} = W_{\text{corr}}^{(m+1)}.$$

The two masses are equal, so the weighted error of f_m on the updated weights is exactly $\frac{1}{2}$: each learner is rendered useless against the distribution it leaves behind. The implication is that AdaBoost can never profitably select the same hypothesis twice in a row — the re-weighting is an *orthogonalisation* step that forces every stage to contribute something the previous stage did not, which is the sequential counterpart of the decorrelation that bagging obtains by randomisation. (Hypotheses can recur later, once the weight distribution has moved on.)

Solution to Exercise 4.5. (a) *Binary log-loss.* With $y \in \{0, 1\}$, logits F , and $\sigma(F) = (1 + e^{-F})^{-1}$,

$$L(y, F) = -[y \ln \sigma(F) + (1 - y) \ln(1 - \sigma(F))].$$

Using $\sigma'(F) = \sigma(F)(1 - \sigma(F))$,

$$\frac{\partial L}{\partial F} = -\left[y \frac{\sigma'}{\sigma} - (1 - y) \frac{\sigma'}{1 - \sigma}\right] = -[y(1 - \sigma) - (1 - y)\sigma] = \sigma(F) - y,$$

so the pseudo-residual [Eq. \(4.7\)](#) is $r = y - \sigma(F)$: the error in predicted probability, as claimed in [Section 4.6.3](#).

(b) *Pinball loss.* At quantile level $\tau \in (0, 1)$,

$$L_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F, \end{cases} \quad r = -\frac{\partial L_\tau}{\partial F} = \begin{cases} \tau, & y > F, \\ \tau - 1, & y < F, \end{cases}$$

(the loss is not differentiable at $y = F$; any subgradient value in $[\tau - 1, \tau]$ serves). The boosted model then nudges its prediction up by a constant amount whenever it under-predicts and down whenever it over-predicts, with asymmetric magnitudes τ and $1 - \tau$; the fixed point of that asymmetry is the τ -quantile.

Use in collections. A model of the $\tau = 0.9$ quantile of repayment delay answers, per borrower, “by what day will this loan have been repaid with 90% confidence?” That is the planning quantity a collections operation needs: it sizes the follow-up queue conservatively, sets the day on which an unpaid loan becomes genuinely anomalous (rather than merely late-ish) and therefore worth an agent’s call, and prices expected recovery timelines into cash-flow forecasts. The conditional *mean* delay would understate all three, because delay distributions are heavy-tailed and the cost of under-provisioning is asymmetric — the same cost-sensitivity logic as [Chapter 3](#), applied to a regression target.

Solution to Exercise 4.6. Under squared error $L = \frac{1}{2}(y - F)^2$ we have $g_i = \partial_F L = F(x_i) - y_i = -r_i$ (minus the ordinary residual) and $h_i = \partial_F^2 L = 1$. Hence for leaf j , $G_j = -\sum_{i \in I_j} r_i$ and $H_j = |I_j|$, and [Eq. \(4.11\)](#) gives

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} r_i}{|I_j| + \lambda} \xrightarrow{\lambda=0} \frac{1}{|I_j|} \sum_{i \in I_j} r_i,$$

the mean residual in the leaf — so vanilla gradient boosting ([Section 4.6.3](#)) is recovered as the unregularised special case. For $\lambda > 0$,

$$w_j^* = \frac{H_j}{H_j + \lambda} \cdot \underbrace{\frac{1}{H_j} \sum_{i \in I_j} r_i}_{\text{mean residual}},$$

a shrinkage of the mean residual by the factor $H_j/(H_j + \lambda) \in (0, 1)$. The factor is close to 1 for well-populated leaves and small for thin ones: a leaf holding $H_j = 500$ loans with $\lambda = 5$ keeps 99% of its estimate, while a leaf holding 5 loans keeps only 50%. The ridge penalty is thus an automatic scepticism dial — conclusions drawn from few borrowers are pulled toward “no adjustment” — which is exactly the behaviour one wants in a 4.1%-default portfolio where thin leaves are dominated by noise, and it complements the hard floor that `min_child_samples` imposes in [Listing 4.4](#).

Solution to Exercise 4.7. *Counterexample.* Hold every feature fixed except the delinquency feature `days_past_due` (x , in days), and suppose the unconstrained model’s fitted risk is piecewise constant:

$$F(x) = \begin{cases} 0.10, & 0 \leq x < 10, \\ 0.30, & 10 \leq x < 20, \\ 0.20, & 20 \leq x < 30, \\ 0.50, & x \geq 30. \end{cases}$$

Any reasonable notion of the *average* response is increasing — the least-squares linear fit of F on x has positive slope, and the overall trend runs from 0.10 to 0.50 — yet the model is not monotone: it assigns *lower* risk at $x = 25$ than at $x = 15$.

Governance failure. Consider two applicants identical in every respect except that applicant A has 15 days of arrears and applicant B has 25. Suppose the approval threshold sits at 0.25: A is declined, B is approved. The adverse-action reason issued to A necessarily cites delinquency — it is the only feature in play — yet B, with *strictly worse* delinquency, was approved. The stated reason is therefore falsified by the lender’s own decisions: A’s appeal, a regulator’s file review, or a journalist’s paired test will surface the inconsistency, and under the automated-decision provisions of the Kenya DPA (2019) the lender must stand behind an explanation its model contradicts. The average trend is irrelevant to this failure because adverse-action reasons are given to *individuals*, who live at specific points of the curve, not on its regression line. The monotonic constraint of [Definition 4.5](#) removes the failure mode by construction, at the cost of the (noise-driven) dip — which is precisely the trade described in [Section 4.7.3](#).

Solution to Exercise 4.8. By [Proposition 4.5](#), an equal-resolution grid with $B = 729$ and $d = 6$ uses $r = 729^{1/6} = 3$ distinct values per axis, so each of the two important hyperparameters is probed at just *three* values, each combination repeated across the $3^4 = 81$ settings of the four irrelevant axes. Random search probes 729 distinct values of each important axis.

Probability the grid never enters the best decile of both important axes simultaneously. Model each axis as $[0, 1]$ with the “best decile” an interval of length

0.1 whose location is unknown, and take the grid's three values per axis to be evenly spaced with a uniformly random offset, $\{u, u + \frac{1}{3}, u + \frac{2}{3}\}$ with $u \sim \text{U}[0, \frac{1}{3})$ (assumptions stated: independent axes, uniform target location, one-third spacing). Since the target interval $0.1 < \frac{1}{3}$, at most one grid value can fall in it, and the probability that one does is $3 \times 0.1 = 0.3$ per axis. Because the grid is a Cartesian product, hitting the decile marginally on both important axes guarantees a trial at the intersection; the two axes are independent, so

$$P(\text{grid hits both}) = 0.3 \times 0.3 = 0.09, \quad P(\text{grid never does}) = 0.91.$$

For random search, a single trial lands in both deciles with probability $0.1 \times 0.1 = 0.01$, so

$$P(\text{random never does}) = (1 - 0.01)^{729} = 0.99^{729} \approx e^{-7.29} \approx 7 \times 10^{-4}.$$

The grid misses the jointly-best region nine times out of ten; random search finds it essentially always. This is Fig. 4.11 with numbers attached, and it is the practical content of low effective dimensionality (Bergstra and Bengio, 2012).

Solution to Exercise 4.9. With $N = 81$, $b = 25$ and $\eta = 3$, the rungs are:

Rung	1	2	3	4	5
Configurations alive	81	27	9	3	1
Budget each (rounds)	25	75	225	675	2025
Cost of rung	2025	2025	2025	2025	2025

Each rung costs exactly $81 \times 25 = 2025$ configuration-rounds — the $1/\eta$ survival rate and the $\times \eta$ budget growth cancel — so the total is $5 \times 2025 = 10,125$ configuration-rounds. Running all 81 configurations to the full 2,025 rounds would cost $81 \times 2025 = 164,025$: successive halving explores the same 81 candidates for 6.2% of the cost, a $16.2\times$ saving, at the price of trusting rung-1 rankings.

Where the trust fails. The schedule assumes that performance after 25 boosting rounds predicts performance after 2,025. The clearest counterexample in our own chapter is the learning rate (Eq. (4.8)): a configuration with $\nu = 0.02$ is deliberately slow — at 25 rounds it has barely moved from the base rate and looks terrible next to a $\nu = 0.3$ configuration — yet it is exactly the kind of run that attains the *lowest* final validation loss. Aggressive halving systematically executes the low- ν candidates at rung 1 and crowns a fast-starting, earlier-overfitting configuration. Mitigations: hedge across brackets (Hyperband, Li et al., 2017), delay pruning (`n_warmup_steps` in Listing 4.6), or normalise the comparison by learning-curve extrapolation rather than raw score.

Solution to Exercise 4.10. Each measured score is $\hat{\mathcal{V}}_t = v + \varepsilon_t$ with $\varepsilon_t = sZ_t$ and $Z_t \sim \mathcal{N}(0, 1)$ i.i.d. The search reports

$$\max_t \hat{\mathcal{V}}_t = v + s \max_t Z_t \implies \mathbb{E}[\max_t \hat{\mathcal{V}}_t] = v + s \mathbb{E}[\max_{t \leq T} Z_t],$$

since v is common and the maximum commutes with the shift. All T configurations are *equally good* by construction, so the entire excess over v is optimism manufactured by selection. Using the standard extreme-value approximation $\mathbb{E}[\max_{t \leq T} Z_t] \approx \sqrt{2 \ln T}$: for $T = 100$, $\sqrt{2 \ln 100} = \sqrt{9.21} \approx 3.03$, so with fold-level noise $s = 0.004$ in PR-AUC the expected reported best is

$$v + 0.004 \times 3.03 \approx v + 0.012.$$

An HPO study of one hundred trials will thus claim roughly $+0.012$ PR-AUC that is not there, even when tuning does nothing at all. Two readings matter for Lab 4. First, the magnitude is the same order as the optimism gap students typically measure between `study.best_value` and the sealed out-of-time score — part of that gap is drift, but a predictable slice of it is pure winner’s curse, and Table 4.7’s tuned-model row would read ≈ 0.28 rather than 0.294 if the inner-loop best were taken at face value. Second, the formula shows the levers: optimism grows only as $\sqrt{\ln T}$ (slowly — more trials are not catastrophic) but linearly in s , so the cheapest honesty gain is reducing fold noise: more folds, repeated CV, or larger validation windows. And, of course, reporting from data the search never touched (Fig. 4.13) removes the curse entirely — which is the point of the chapter’s final pitfall box.

Chapter 5: Unsupervised Learning and Dimensionality Reduction

Solution to Exercise 5.1. Let $J^*(k)$ denote the optimal inertia with k clusters. Take an optimal solution for k with centroids $\mu_1, \dots, \mu_k$ and assignment c . Build a feasible $(k + 1)$ -cluster solution by choosing any point x_r that is not already alone in its cluster, adding the centroid $\mu_{k+1} = x_r$, and reassigning x_r to it. Every other point keeps its assignment and hence its contribution to Eq. (5.1); the contribution of x_r falls from $\|x_r - \mu_{c(r)}\|^2 \geq 0$ to 0. Therefore $J^*(k + 1) \leq J^*(k)$. At $k = m$ we may place one centroid on each point, giving $J^*(m) = 0$.

Consequently “minimise inertia over k ” always returns $k = m$, which is the degenerate clustering in which every wallet is its own segment: the criterion is vacuous because model complexity is unpenalised. The elbow heuristic is not vacuous because it does not minimise J ; it examines the *rate* at which J falls and looks for the point past which additional clusters

buy little. That is an informal appeal to a complexity penalty, which BIC (Eq. (5.9)) makes formal — and this is precisely why BIC has a genuine interior minimum where inertia does not.

Solution to Exercise 5.2. Partition $C_1 = \{0, 1, 2\}$, $C_2 = \{10, 11, 12\}$. By symmetry it suffices to do C_1 .

- $x = 0$: $a = \frac{1+2}{2} = 1.5$; $b = \frac{10+11+12}{3} = 11$; $s = (11 - 1.5)/11 = 0.8636$.
- $x = 1$: $a = \frac{1+1}{2} = 1$; $b = \frac{9+10+11}{3} = 10$; $s = (10 - 1)/10 = 0.9000$.
- $x = 2$: $a = \frac{2+1}{2} = 1.5$; $b = \frac{8+9+10}{3} = 9$; $s = (9 - 1.5)/9 = 0.8333$.

C_2 mirrors these exactly, so $\bar{s} = \frac{1}{3}(0.8636 + 0.9000 + 0.8333) = 0.8657$.

Partition $C'_1 = \{0, 1\}$, $C'_2 = \{2, 10, 11, 12\}$.

- $x = 0$: $a = 1$; $b = \frac{2+10+11+12}{4} = 8.75$; $s = (8.75 - 1)/8.75 = 0.8857$.
- $x = 1$: $a = 1$; $b = \frac{1+9+10+11}{4} = 7.75$; $s = 0.8710$.
- $x = 2$: $a = \frac{8+9+10}{3} = 9$; $b = \frac{2+1}{2} = 1.5$; $s = (1.5 - 9)/9 = -0.8333$.
- $x = 10$: $a = \frac{8+1+2}{3} = 3.6\bar{6}$; $b = \frac{10+9}{2} = 9.5$; $s = (9.5 - 3.6\bar{6})/9.5 = 0.6140$.
- $x = 11$: $a = \frac{9+1+1}{3} = 3.6\bar{6}$; $b = 10.5$; $s = 0.6508$.
- $x = 12$: $a = \frac{10+2+1}{3} = 4.3\bar{3}$; $b = 11.5$; $s = 0.6232$.

$\bar{s} = \frac{1}{6}(0.8857 + 0.8710 - 0.8333 + 0.6140 + 0.6508 + 0.6232) = 0.4686$.

Comment. The natural partition scores far higher (0.866 versus 0.469), so the silhouette correctly prefers it. More instructive is the single strongly negative value, $s(2) = -0.833$: the point $x = 2$ is on average much closer to C'_1 than to the cluster it was assigned to. This is exactly what the silhouette *diagram* is for. An analyst looking only at the mean would see a mediocre 0.469; an analyst looking at the per-point profile sees immediately *which* point is misassigned and can fix it. Note also that $\bar{s}$ for the bad partition is still positive, because four of six points are content — averages hide misassignment, profiles do not.

Solution to Exercise 5.3. Take the six points, with axis 1 in Kenyan shillings and axis 2 as a count of transactions:

	P_1	P_2	P_3	P_4	P_5	P_6
x_1 (KES)	1000	1100	1200	9000	9100	9200
x_2 (count)	2	40	78	2	40	78

Raw data. Squared distances are dominated by x_1 , whose spread is ~ 8000 against ~ 76 for x_2 . K -means with $k = 2$ therefore splits on value: $\{P_1, P_2, P_3\}$ versus $\{P_4, P_5, P_6\}$. (Check: the within-cluster sum of squares for this split is $2 \cdot (20000 + 2 \cdot 1352.7) \approx 4.5 \times 10^4$, whereas splitting on count gives clusters spanning 8000 in x_1 and costs $\sim 1.3 \times 10^8$.)

Standardised data. After centring and dividing by the sample standard deviations ($\sigma_1 \approx 4271$, $\sigma_2 \approx 34.9$), both axes have unit spread and the configuration becomes a symmetric 2×3 grid at $x_1 \approx \pm 0.94$ and $x_2 \approx \{-1.09, 0, 1.09\}$. The two candidate splits now cost the same by symmetry along x_1 , but the three-level x_2 axis has the larger total spread (± 1.09 against ± 0.94), so K -means prefers to cut along x_2 , returning e.g. $\{P_1, P_4\}$ (low count) against $\{P_2, P_3, P_5, P_6\}$. Perturb the numbers slightly to break the tie in whichever direction you like; the point is that the two answers are different and both are “correct” for their respective metrics.

Which to defend. The standardised one — but the honest defence is not that standardisation is a rule. It is that the raw-data partition answers a question nobody asked: “are these transactions large or small in KES?” is answerable with a single threshold and needs no clustering at all. If the business genuinely wants value tiers, use a threshold and say so. Clustering earns its keep only when several dimensions jointly matter, and that requires them to be commensurable. Where a monetary feature is heavy-tailed, prefer a quantile or log transform to plain standardisation, since a handful of whale wallets will otherwise dominate the standardised axis too.

Solution to Exercise 5.4. Construction. Work in $\mathbb{R}^2$ with $\varepsilon = 1.05$ and $\text{minPts} = 8$. Build two clusters, each a dense blob plus one outlying “antenna” point:

- Cluster A : ten points inside a ball of radius 0.05 centred at $(-0.6, 0)$, plus a single point $p_A = (0, 0)$.
- Cluster B : ten points inside a ball of radius 0.05 centred at $(2.6, 0)$, plus a single point $p_B = (2, 0)$.
- The candidate border point $q = (1, 0)$.

Every blob point is core: it sees the other nine blob points (mutual distance ≤ 0.1) and its antenna (distance $0.6 \pm 0.05 \leq \varepsilon$), so $|N_\varepsilon| = 11 \geq 8$. *Each antenna is core:* p_A sees its blob (ten points at ≈ 0.6), itself, and q (distance exactly $1 \leq \varepsilon$), so $|N_\varepsilon(p_A)| = 12 \geq 8$. *q is not core:* its ε -ball contains only q , p_A and p_B , since both blobs sit at distance $\approx 1.6 > \varepsilon$; thus $|N_\varepsilon(q)| = 3 < 8$. Finally A and B are distinct components, because $d(p_A, p_B) = 2 > \varepsilon$ and q — the only object between them — is not core, so it cannot propagate reachability.

Therefore q is a border point directly density-reachable from p_A and from p_B , which belong to different clusters. DBSCAN assigns q to whichever cluster is expanded first: process p_A before p_B and q joins A ; reverse the visitation order and q joins B . Its label is genuinely order-dependent.

Why this does not undermine reproducibility. By Proposition 5.3, the core-point set and the partition of core points into clusters are order-invariant. The ambiguity therefore never changes the number of clusters, their shape, or the membership of any core point — only the label of a point sitting in a low-density bridge, whose ϵ -ball is by construction almost empty. Operationally: the segments you name and price are determined by their cores, and a point whose label flips is exactly one about which you should not be making a confident decision. If bit-for-bit determinism is required for audit, fix the input ordering and record it; or use HDBSCAN, which assigns such points by membership strength rather than by visitation order.

Solution to Exercise 5.5. With $k = 2$, $\pi_1 = \pi_2 = \frac{1}{2}$, and shared variance σ^2 , the Gaussian normalising constants cancel in Eq. (5.5):

$$\gamma_{i1} = \frac{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right)}{\exp\left(-\frac{(x_i - \mu_1)^2}{2\sigma^2}\right) + \exp\left(-\frac{(x_i - \mu_2)^2}{2\sigma^2}\right)} = \frac{1}{1 + \exp\left(\frac{(x_i - \mu_1)^2 - (x_i - \mu_2)^2}{2\sigma^2}\right)}.$$

Expanding the difference of squares, $(x - \mu_1)^2 - (x - \mu_2)^2 = -2x(\mu_1 - \mu_2) + \mu_1^2 - \mu_2^2 = -2(\mu_1 - \mu_2)\left(x - \frac{\mu_1 + \mu_2}{2}\right)$. Hence

$$\gamma_{i1} = \sigma_{\text{logistic}}(w(x_i - x_0)), \quad w = \frac{\mu_1 - \mu_2}{\sigma^2}, \quad x_0 = \frac{\mu_1 + \mu_2}{2},$$

where $\sigma_{\text{logistic}}(z) = (1 + e^{-z})^{-1}$. So the responsibility is exactly a logistic function of x with **slope parameter** $w = (\mu_1 - \mu_2)/\sigma^2$ and **midpoint** at the arithmetic mean of the two component means, where $\gamma_{i1} = \frac{1}{2}$.

As $\sigma^2 \rightarrow 0^+$ with $\mu_1 \neq \mu_2$ fixed, $|w| \rightarrow \infty$: the logistic curve steepens into a step at x_0 , so $\gamma_{i1} \rightarrow \mathbb{1}\{x_i \text{ nearer } \mu_1\}$ for every $x_i \neq x_0$. This is precisely Proposition 5.5 in one dimension: the midpoint x_0 is the perpendicular bisector (here, a single point), and the soft assignment collapses to K -means' hard nearest-centroid rule. The general lesson is that σ^2 acts as a temperature: large σ^2 gives soft, hedged memberships; small σ^2 gives confident hard ones. Reporting a segment membership of 0.51 versus 0.49 as though it were a fact is a failure to read this parameter.

Solution to Exercise 5.6. Divergence. Take $m \geq 2$ distinct points and the family

$$\theta_\epsilon : \quad \pi_1 = \pi_2 = \frac{1}{2}, \quad \mu_1 = x_1, \quad \sigma_1^2 = \epsilon, \quad \mu_2 = \bar{x}, \quad \sigma_2^2 = s^2 > 0 \text{ fixed.}$$

The first component's density at x_1 is $\mathcal{N}(x_1 \mid x_1, \epsilon) = (2\pi\epsilon)^{-1/2}$, so the contribution of observation 1 to the log-likelihood is at least $\log(\frac{1}{2}(2\pi\epsilon)^{-1/2}) \rightarrow +\infty$ as $\epsilon \rightarrow 0^+$. Every other observation $i \geq 2$ retains the second component's contribution, bounded below by $\log(\frac{1}{2}\mathcal{N}(x_i \mid \bar{x}, s^2)) > -\infty$, which does not depend on ϵ . Hence $\ell(\theta_\epsilon) \rightarrow +\infty$: the likelihood is unbounded above and no global maximiser exists. Geometrically, the fitted model has spiked onto a single data point and explains nothing.

Effect of reg_covar. Adding $\lambda > 0$ to the diagonal replaces σ_j^2 by $\sigma_j^2 + \lambda$, so the achievable density at a point is bounded by $(2\pi\lambda)^{-1/2}$ and the per-observation log-likelihood by $\log(\pi_{\max}(2\pi\lambda)^{-1/2}) < \infty$. Summing over m observations bounds ℓ above, restoring existence of a maximum on the (now compact-in-effect) parameter region, and the degenerate spikes are removed from the search space entirely.

Price paid. Three costs. (i) The estimator is biased: every fitted covariance is inflated by λ , so genuinely tight clusters are reported as looser than they are, and density-based anomaly scores are correspondingly flattened. (ii) The reported likelihood, and therefore BIC, is no longer that of the model you claim to be fitting, so BIC comparisons are only valid at a *fixed* λ . (iii) λ is a scale-dependent hyperparameter: the same value means something different on standardised and unstandardised features, which is one more reason to fix the scaling before the mixture. In practice keep λ at the smallest value that prevents collapse (the scikit-learn default 10^{-6} on standardised data is usually right) and treat a fit that needs a large λ as evidence that k is too big.

Solution to Exercise 5.7. Free parameters split into weights, means and covariances. Weights contribute $k - 1$ (they sum to one); means contribute kn . Covariances:

covariance_type	Covariance parameters	Total p
full	$k \frac{n(n+1)}{2}$	$(k - 1) + kn + k \frac{n(n+1)}{2}$
tied	$\frac{n(n+1)}{2}$	$(k - 1) + kn + \frac{n(n+1)}{2}$
diag	kn	$(k - 1) + 2kn$
spherical	k	$(k - 1) + kn + k$

For $n = 32, k = 5$: $k - 1 = 4$ and $kn = 160$.

- full: $\frac{32 \cdot 33}{2} = 528$ per component, so $p = 4 + 160 + 5(528) = \mathbf{2804}$.
- tied: $p = 4 + 160 + 528 = 692$.
- diag: $p = 4 + 2(160) = \mathbf{324}$.
- spherical: $p = 4 + 160 + 5 = 169$.

With $m = 150,000$, $\ln m = \ln(1.5 \times 10^5) \approx 11.9184$. For full to beat diag on BIC we need

$$p_{\text{full}} \ln m - 2 \ln \hat{L}_{\text{full}} < p_{\text{diag}} \ln m - 2 \ln \hat{L}_{\text{diag}},$$

i.e.

$$\begin{aligned} \ln \hat{L}_{\text{full}} - \ln \hat{L}_{\text{diag}} &> \frac{1}{2}(2804 - 324)(11.9184) \\ &= \frac{1}{2}(2480)(11.9184) \approx \mathbf{14,778}. \end{aligned}$$

Averaged over the sample this is only $14,778/150,000 \approx 0.0985$ nats per observation — about a 10% relative improvement in average likelihood. So with 150,000 wallets, full covariances are a *cheap* upgrade: modest correlation structure within segments will pay for 2480 extra parameters comfortably. Repeat the arithmetic at $m = 2000$ and the required gap becomes ≈ 0.5 nats per observation, at which point diag usually wins. The general moral is that the “right” covariance type is a function of sample size, not of taste.

Solution to Exercise 5.8. Independent coordinates. A short simulation (numpy default generator, seed 0; $m = 1000$ points and one query point drawn from the same distribution) gives ratios of the order

n	2	10	100	1000
i.i.d. coordinates	121	3.04	0.44	0.12
5-factor latent model	179	10.4	8.88	7.55

(means over 20 replications; the $n = 2$ column is enormous and wildly variable because $D_{\min}$ can be arbitrarily small in low dimension, so read the *trend* across columns, not the individual digits). In the i.i.d. row the contrast between nearest and furthest neighbour collapses roughly like $n^{-1/2}$, exactly as Proposition 5.6 predicts from $\text{Var}[D_n^2]/(\mathbb{E}[D_n^2])^2 = \mathcal{O}(1/n)$.

Latent factor model. Generate $u_i \in \mathbb{R}^5$ i.i.d. standard normal and set $x_i = Au_i + 0.05 \eta_i$ with $A \in \mathbb{R}^{n \times 5}$ fixed and η_i small isotropic noise (second row of the table). The ratio now stays around 8–10 from $n = 10$ all the way to $n = 1000$: raising the ambient dimension by two orders of magnitude does essentially nothing, because the distances are governed by the five latent coordinates and the extra measured dimensions carry almost no independent variability.

Explanation. The proof of Proposition 5.6 needs the coordinate contributions δ_t to be (near-)independent, so that $\text{Var}[D_n^2]$ grows like n while $\mathbb{E}[D_n^2]$ grows like n , forcing the relative variance to 0. Under the factor model the δ_t are strongly dependent: the effective, or intrinsic, dimension is 5, not n , and it is the intrinsic dimension that governs concentration. This is the entire justification for the chapter’s strategy. We do not escape the curse by wishing it away; we escape it because real behavioural features are redundant, and PCA finds the small number of directions along

which the data actually varies. It is also why one should measure the explained-variance curve before concluding that a 200-feature problem is high-dimensional: it may have an intrinsic dimension of 20.

Solution to Exercise 5.9. Proof. From the proof of Proposition 5.7,

$$E(W) = \frac{1}{m} \sum_i \|x_i\|^2 - \text{tr}(W^\top S W).$$

For centred data $\frac{1}{m} \sum_i \|x_i\|^2 = \text{tr}(S) = \sum_{t=1}^n \lambda_t$. At the optimum $W = W_d$ we have, by Theorem 5.1, $\text{tr}(W_d^\top S W_d) = \sum_{t \leq d} \lambda_t$. Hence

$$E(W_d) = \sum_{t=1}^n \lambda_t - \sum_{t \leq d} \lambda_t = \sum_{t > d} \lambda_t,$$

as claimed: the mean squared reconstruction error is exactly the variance you threw away.

Numerical verification. With $S = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ from Example 5.2, $\lambda_1 = 6$, $\lambda_2 = 2$, $v_1 = \frac{1}{\sqrt{2}}(1, 1)^\top$. For $d = 1$ the formula predicts $E = \lambda_2 = 2$. Take three centred points consistent with S , for instance

$$x_1 = (3, 1)^\top, \quad x_2 = (-1, -3)^\top, \quad x_3 = (-2, 2)^\top.$$

Projection: $z_i = v_1^\top x_i$ gives $z_1 = 4/\sqrt{2} = 2\sqrt{2}$, $z_2 = -4/\sqrt{2} = -2\sqrt{2}$, $z_3 = 0$. Reconstruction $\hat{x}_i = z_i v_1$: $\hat{x}_1 = (2, 2)^\top$, $\hat{x}_2 = (-2, -2)^\top$, $\hat{x}_3 = (0, 0)^\top$. Residuals: $x_1 - \hat{x}_1 = (1, -1)^\top$ with squared norm 2; $x_2 - \hat{x}_2 = (1, -1)^\top$, squared norm 2; $x_3 - \hat{x}_3 = (-2, 2)^\top$, squared norm 8. Mean = $(2 + 2 + 8)/3 = 4$.

The mean over this particular triple is 4, not 2, because three points do not reproduce S : their own sample covariance is $\begin{pmatrix} 14/3 & 2/3 \\ 2/3 & 14/3 \end{pmatrix}$, whose second eigenvalue is $14/3 - 2/3 = 4$ — and 4 is exactly what the formula predicts for *their* covariance. The identity holds sample-wise, and checking it this way is a useful reminder that λ_t are properties of the sample you actually projected, not of an assumed population.

Solution to Exercise 5.10. A defensible one-page answer covers four things.

What to measure next. (i) Whether the disparity is in the *clustering* or in the *limit rule*: compute segment membership rates by county, then, within segment, compare limits — if limits are equal within segment, the mechanism is composition, not pricing. (ii) The behavioural distance between this segment and its neighbours: if the silhouette for this cluster is weak, the segment may be an artefact and the disparity a modelling choice rather than a finding. (iii) Whether the underlying features are themselves proxies: regress county on the top principal components and see how predictable it

is; high predictability means the “behavioural” representation substantially encodes geography. (iv) Outcome data where available — if this segment’s historical default rate genuinely differs, the disparity may be justified; if it does not, the 40% limit reduction lacks a risk basis.

Instruments engaged. Both discussed in the chapter’s regulatory box. The Data Protection Act, 2019 governs the derived segment as personal data: lawful basis, purpose limitation, and the data subject’s rights of access and objection apply to an inferred behavioural profile as much as to the raw transaction log. The CBK Digital Credit Providers Regulations, 2022 require pricing and terms to be disclosed and non-exploitative and credit decisions to be explicable — an unexplainable “cluster 3” limit rule does not meet that standard, and a limit reduction concentrated on two counties invites scrutiny of whether the differentiation is risk-based or merely correlational.

Two remedies with costs. (i) *Re-fit excluding geography-loaded features* (agent-location density, merchant-mix features tied to regional paybills). Cost: some genuine predictive signal is lost, so expected credit losses rise; and the remedy is incomplete, since geography can be reconstructed from residual features. (ii) *Keep the segmentation but sever it from pricing*, using segments only for product design and communication while limits are set by the audited, monotone scorecard of [Chapter 4](#) with fairness constraints. Cost: loses the operational simplicity of segment-based pricing, and requires a second governance process for the scorecard. A third, cheaper option worth listing: cap the maximum limit differential across segments, which bounds the harm without discarding the model.

When shipping unchanged is defensible. If the disparity is *explained* by a legitimate, disclosed, risk-relevant behavioural difference that survives conditioning on geography — for example, the segment is defined by seasonal single-harvest income, the lower limit reflects a measured higher default rate for bullet-repayment products, the county concentration is a consequence of where that livelihood occurs rather than of county itself, and an applicant from either county with non-seasonal income lands in a different segment. The test is counterfactual: does an otherwise identical borrower from a different county receive the same limit? If yes, the segment is tracking livelihood, not location, and the position is arguable — provided the reasoning, the tests and the residual disparity are documented before launch, and monitored afterwards.

Chapter 6: Deep Learning Fundamentals

Solution to Exercise 6.1. By Proposition [6.1](#), the three linear hidden layers collapse to a single affine map. Writing $\mathbf{W}^{(1)} \in \mathbb{R}^{128 \times 40}$, $\mathbf{W}^{(2)} \in \mathbb{R}^{64 \times 128}$,

$\mathbf{W}^{(3)} \in \mathbb{R}^{32 \times 64}$ and $\mathbf{w}^{(4)} \in \mathbb{R}^{1 \times 32}$, the network computes

$$\hat{p} = \sigma(\tilde{\mathbf{w}}^\top \mathbf{x} + \tilde{b}), \quad \tilde{\mathbf{w}}^\top = \mathbf{w}^{(4)} \mathbf{W}^{(3)} \mathbf{W}^{(2)} \mathbf{W}^{(1)} \in \mathbb{R}^{1 \times 40}.$$

The model class is therefore **logistic regression on the original 40 features** — nothing more. Note in particular that the widths 128, 64, 32 do not enlarge the class: the composite matrix has rank at most 1 regardless, because the output layer is one-dimensional.

Counting parameters layer by layer, using Remark 6.1:

$$\underbrace{41 \times 128}_{5248} + \underbrace{129 \times 64}_{8256} + \underbrace{65 \times 32}_{2080} + \underbrace{33 \times 1}_{33} = 15,617,$$

against $40 + 1 = 41$ for the equivalent logistic regression. The network therefore carries $15,617 - 41 = 15,576$ redundant parameters, roughly 381 times the minimum, while representing exactly the same set of functions.

Two qualifications are worth making, because the exercise is not *only* a counting exercise. First, redundant parameterisation is not free at training time: it multiplies memory traffic and gradient computation for no gain in expressiveness. Second, the over-parameterised form is not perfectly equivalent *as an optimisation problem*: the loss surface of a deep linear network is non-convex in θ even though the induced function class is the convex-friendly logistic one, and the factorised parameterisation induces an implicit bias in the trajectory of gradient descent. The correct diagnosis remains that the architecture is a mistake: if a linear model is wanted, fit a linear model; if non-linearity is wanted, insert an activation function.

Solution to Exercise 6.2. Part 1. By Proposition 6.2, $\sigma'(z) = s(1-s)$ with $s = \sigma(z)$. Solving $s(1-s) = 0.01$, that is $s^2 - s + 0.01 = 0$, gives

$$s = \frac{1 \pm \sqrt{1 - 0.04}}{2} = \frac{1 \pm \sqrt{0.96}}{2} \in \{0.010102, 0.989898\}.$$

Since $s \mapsto s(1-s)$ is a downward parabola, $\sigma'(z) < 0.01$ precisely when s lies outside $[0.010102, 0.989898]$. Converting the upper root back to the logit scale,

$$z = \ln \frac{s}{1-s} = \ln \frac{0.989898}{0.010102} = \ln(97.99) = 4.5849,$$

and by the symmetry $\sigma(-z) = 1 - \sigma(z)$ the lower root gives -4.5849 . Hence

$$\sigma'(z) < 0.01 \iff |z| > 4.585 \quad (\text{to three decimal places}).$$

A pre-activation of magnitude only about 4.6 — entirely unremarkable for unstandardised inputs — already suppresses the gradient by a factor of at least 25 relative to its maximum of $1/4$.

Part 2. With $\|\mathbf{W}^{(l)}\| = 1$ and $\gamma = 1/4$, Proposition 6.10 gives the bound $(\gamma w)^{L-1} = 4^{-(L-1)}$. Requiring $4^{-(L-1)} \leq 10^{-6}$,

$$(L-1) \ln 4 \geq 6 \ln 10 \implies L-1 \geq \frac{6 \times 2.302585}{1.386294} = 9.97,$$

so $L-1 = 10$ multiplicative factors suffice, that is a network of $\mathbf{L} = 11$ layers. Ten sigmoid layers are enough to reduce the error signal reaching the first layer by six orders of magnitude *even when the weight matrices are perfectly scaled*. This is the quantitative content of the claim that deep sigmoid stacks do not train, and it is why Section 6.9 treats initialisation and Section 6.10 treats normalisation as necessities rather than refinements.

Solution to Exercise 6.3. Take the convention of Example 6.2, in which $\mathbf{z}^{(1)} = \mathbf{W}^{(1)\top} \mathbf{x} + \mathbf{b}^{(1)}$, and set $\mathbf{x} = (-0.5, 1.2)^\top$:

$$z_1^{(1)} = 1(-0.5) + 0.5(1.2) = 0.1, \quad z_2^{(1)} = -2(-0.5) + 1(1.2) - 0.4 = 1.8,$$

so $\mathbf{z}^{(1)} = (0.1, 1.8)^\top$ and, since both components are positive, $\mathbf{h}^{(1)} = \text{ReLU}(\mathbf{z}^{(1)}) = (0.1, 1.8)^\top$. The output logit is

$$z^{(2)} = 2(0.1) - 1(1.8) - 0.1 = -1.7, \quad \hat{p} = \sigma(-1.7) = \frac{1}{1 + e^{1.7}} = 0.15447.$$

With $y = 1$ the loss is $\mathcal{L} = -\ln(0.15447) = 1.868$, and by Proposition 6.5 the output error signal is

$$\delta^{(2)} = \hat{p} - y = 0.15447 - 1 = -0.84553.$$

It is negative, correctly instructing the network to *increase* $z^{(2)}$ for this borrower.

Applying (6.17) at the output layer,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}^{(2)}} = \delta^{(2)} \mathbf{h}^{(1)} = -0.84553 \begin{pmatrix} 0.1 \\ 1.8 \end{pmatrix} = \begin{pmatrix} -0.08455 \\ -1.52196 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial b^{(2)}} = -0.84553.$$

The second component is eighteen times the first, purely because $h_2^{(1)} = 1.8$ is eighteen times $h_1^{(1)}$: gradient magnitude at a weight is proportional to the activation it multiplies, which is the algebraic reason unscaled activations produce unbalanced updates.

Applying the recursion (6.16) with $\text{ReLU}'(z) = 1$ for $z > 0$,

$$\delta^{(1)} = (\mathbf{w}^{(2)} \delta^{(2)}) \odot \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2(-0.84553) \\ -1(-0.84553) \end{pmatrix} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix},$$

and, since $z_k^{(1)} = \sum_j W_{jk}^{(1)} x_j + b_k^{(1)}$ implies $\partial \mathcal{L} / \partial W_{jk}^{(1)} = x_j \delta_k^{(1)}$,

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \mathbf{x} \delta^{(1)\top} = \begin{pmatrix} 0.84553 & -0.42277 \\ -2.02928 & 1.01464 \end{pmatrix}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(1)}} = \begin{pmatrix} -1.69107 \\ 0.84553 \end{pmatrix}.$$

The comparison with Example 6.2. Neither hidden unit fails to update here: both pre-activations are positive, so $\text{ReLU}'(z_k^{(1)}) = 1$ for $k = 1, 2$ and both rows of $\partial\mathcal{L}/\partial\mathbf{W}^{(1)}$ are non-zero. This is the opposite of Example 6.2, where the second unit had $z_2^{(1)} = -1.70 < 0$, its activation derivative was zero, and the Hadamard product in (6.16) annihilated its error signal, leaving its incoming weights untouched by that example.

The instructive point is that the two examples together show ReLU inactivity is a property of the *example*, not of the unit. Unit 2 was silent for the first borrower and highly active for the second, and over a mini-batch it receives gradient from whichever examples excite it. A unit becomes *dead* in the sense of the pitfall in Section 6.3 only when its pre-activation is negative for *every* example in the training set, at which point no batch can revive it.

Solution to Exercise 6.4. Take $\mathcal{L}(y, \hat{p}) = \frac{1}{2}(y - \hat{p})^2$ with $\hat{p} = \sigma(z)$. By the chain rule and Proposition 6.2,

$$\frac{\partial\mathcal{L}}{\partial z} = \frac{\partial\mathcal{L}}{\partial\hat{p}} \cdot \frac{d\hat{p}}{dz} = -(y - \hat{p}) \sigma'(z) = (\hat{p} - y) \hat{p} (1 - \hat{p}).$$

Comparing with $\partial\mathcal{L}/\partial z = \hat{p} - y$ from Proposition 6.5, the squared-error gradient carries the extra factor $\hat{p}(1 - \hat{p}) = \sigma'(z) \leq \frac{1}{4}$.

That factor is harmful in the worst possible place. Consider an example with $y = 1$ that the network confidently misclassifies, $\hat{p} = 0.01$. The residual $\hat{p} - y = -0.99$ is nearly maximal, but the gradient is $-0.99 \times 0.01 \times 0.99 = -0.0098$: the network learns *a hundred times more slowly* from the example it gets most badly wrong. Cross-entropy, by contrast, delivers the full residual -0.99 . The saturating factor makes the learning rate effectively example-dependent in exactly the wrong direction, and it re-introduces the vanishing-gradient mechanism of Proposition 6.10 at the output layer, where Proposition 6.5 had shown it could be eliminated by pairing the loss with the activation correctly. A secondary objection is that the squared error composed with a sigmoid is non-convex in z even for a single linear layer, whereas cross-entropy is convex, so the choice also costs optimisation guarantees in the simplest case.

Solution to Exercise 6.5. By Proposition 6.6, $\text{Var}[\hat{\mathbf{g}}_B] = \text{Var}[\mathbf{g}_i]/B$, so the standard deviation scales as $B^{-1/2}$. Increasing B from 64 to 512 multiplies the batch size by 8 and therefore multiplies the gradient standard deviation by

$$\sqrt{\frac{64}{512}} = \frac{1}{\sqrt{8}} = 0.354,$$

a reduction of about 64.6%.

Is the trade favourable? Under the stated cost model — time strictly proportional to B — it is not. Eight times the computation buys a factor of

$\sqrt{8} \approx 2.83$ in noise. The comparison that matters is against the alternative use of the same budget: eight successive steps at $B = 64$. Those eight steps consume the same computation, and if the parameters move little over the eight steps their averaged gradient also has standard deviation reduced by $\sqrt{8}$ — the same noise reduction, but with eight actual parameter updates instead of one. Small batches therefore dominate at equal compute in this idealised accounting, which is the theoretical reason mini-batch SGD outperforms batch gradient descent per unit of work.

The assumptions under which this holds, and why practice departs from them:

1. *The variance model.* Examples are drawn i.i.d. (with replacement); without replacement a finite-population correction applies, which matters only when B is a non-negligible fraction of n .
2. *Stationarity within the comparison.* The eight-step argument assumes $\text{Var}[\mathbf{g}_i]$ and ∇J are approximately constant over those eight updates. Near a sharp minimum or with a large learning rate this fails.
3. *Strict proportionality of time to B .* This is the assumption that most clearly breaks on real hardware. A GPU processing a batch of 64 is largely idle; a batch of 512 may cost far less than eight times as much wall-clock time, and in that regime the large batch wins outright.
4. *Fixed learning rate.* Larger batches permit a larger stable η , which can convert the noise reduction into faster progress rather than merely quieter steps.

Finally, noise is not purely a cost: Keskar et al. (2017) give evidence that the extra gradient noise of small batches biases the optimiser toward flatter minima that generalise better, so a pure variance-minimisation objective is the wrong target in the first place.

Solution to Exercise 6.6. Apply Proposition 6.7 with $\lambda_{\min} = 0.02$ and $\lambda_{\max} = 8$.

Largest stable learning rate. Convergence holds if and only if $0 < \eta < 2/\lambda_{\max}$, so

$$\eta_{\text{crit}} = \frac{2}{8} = 0.25,$$

and any $\eta \geq 0.25$ diverges along the sharpest curvature direction.

Optimal learning rate.

$$\eta^* = \frac{2}{\lambda_{\min} + \lambda_{\max}} = \frac{2}{8.02} = 0.24938.$$

Note how close η^* lies to η_{crit} : the optimal step is within 0.25% of the divergence boundary. This is characteristic of ill-conditioned problems and

explains why tuning η on a badly conditioned loss feels like balancing on a knife edge — and why standardising inputs, which reduces κ , is worth more than heroic learning-rate search.

Contraction factor. With $\kappa = 8/0.02 = 400$,

$$\rho = \frac{\kappa - 1}{\kappa + 1} = \frac{399}{401} = \mathbf{0.995012}.$$

Iterations for a hundredfold error reduction. Requiring $\rho^t \leq 10^{-2}$,

$$t \geq \frac{\ln 10^{-2}}{\ln \rho} = \frac{-4.60517}{-0.0049999} = 921.03 \implies t = \mathbf{922} \text{ iterations.}$$

For comparison, a perfectly conditioned problem with $\kappa = 1$ has $\rho = 0$ and converges in a single step. The 922 iterations are the entire price of the condition number, and since $\rho \approx 1 - 2/\kappa$ for large κ , the iteration count grows roughly linearly in κ : halving the condition number by rescaling one badly scaled feature halves the training time.

Solution to Exercise 6.7. Part 1. By Proposition 6.11, $\varsigma^2 = 2/n_{\text{in}} = 2/256 = 0.0078125$, so

$$\varsigma = \sqrt{0.0078125} = \frac{1}{\sqrt{128}} = \mathbf{0.08839}.$$

This is what `he_normal` uses: a zero-mean Gaussian with that standard deviation.

Part 2. Repeat the derivation with $\text{LReLU}_\alpha(u) = \max(\alpha u, u)$, that is $x = u$ for $u > 0$ and $x = \alpha u$ for $u \leq 0$. Let u be symmetric about zero with variance τ_{pre}^2 . Splitting the expectation over the two half-lines and using symmetry, which makes each half contribute $\frac{1}{2}\mathbb{E}[u^2]$,

$$\begin{aligned} \mathbb{E}[x^2] &= \mathbb{E}[u^2 \mathbf{1}\{u > 0\}] + \alpha^2 \mathbb{E}[u^2 \mathbf{1}\{u \leq 0\}] \\ &= \frac{1}{2} \tau_{\text{pre}}^2 + \alpha^2 \cdot \frac{1}{2} \tau_{\text{pre}}^2 = \frac{1}{2} (1 + \alpha^2) \tau_{\text{pre}}^2. \end{aligned}$$

Substituting into $\text{Var}(z) = n_{\text{in}} \varsigma^2 \mathbb{E}[x^2]$ from the first part of Proposition 6.11 and requiring $\text{Var}(z) = \tau_{\text{pre}}^2$ for variance preservation across layers,

$$n_{\text{in}} \varsigma^2 \frac{1}{2} (1 + \alpha^2) \tau_{\text{pre}}^2 = \tau_{\text{pre}}^2 \implies \varsigma^2 = \frac{2}{(1 + \alpha^2) n_{\text{in}}},$$

as required. Two sanity checks confirm the formula. At $\alpha = 0$ the activation is ordinary ReLU and the result reduces to $2/n_{\text{in}}$. At $\alpha = 1$ the activation is the identity, no half of the distribution is discarded, and the result becomes $1/n_{\text{in}}$ — the classical variance-preserving scale for a linear layer. For the usual $\alpha = 0.01$ the correction factor is $1/(1 + 10^{-4})$, numerically indistinguishable from He, so in practice `he_normal` is used for leaky ReLU without adjustment.

Solution to Exercise 6.8. Part 1. In Proposition 6.12 the penalty term is $q(1 - q) \sum_j w_j^2 x_j^2$ with $q = 1 - p$ the keep probability. At $p = 0.5$ we have $q = 0.5$ and

$$q(1 - q) = 0.5 \times 0.5 = \mathbf{0.25},$$

so the implied penalty is $0.25 \sum_j w_j^2 x_j^2$, and the point prediction is additionally shrunk deterministically by the factor $q = 0.5$. Averaged over the training set, the effective per-coordinate ridge coefficient is $0.25 \mathbb{E}[x_j^2]$: dropout is a ridge penalty *weighted by each feature's second moment*. Note also that $q(1 - q)$ is maximised at $q = 1/2$, so $p = 0.5$ is the most aggressive drop rate in this sense; both $p \rightarrow 0$ and $p \rightarrow 1$ send the penalty to zero, the latter for the uninteresting reason that the model is destroyed.

Part 2. Suppose feature j is left in raw KES with typical magnitude 10^6 , so $\mathbb{E}[x_j^2] \approx 10^{12}$, while a standardised feature k has $\mathbb{E}[x_k^2] = 1$. The implied penalties are then in the ratio

$$\frac{0.25 \times 10^{12}}{0.25 \times 1} = 10^{12}.$$

Coordinate j is penalised a trillion times more heavily than coordinate k , which drives w_j to essentially zero: the feature is silently deleted from the model by the regulariser, no matter how predictive it is. The reverse case is equally bad — a feature measured in millionths would be effectively unregularised and free to overfit.

This is the same lesson as Section 6.12, arriving from a different direction. There, unstandardised inputs damaged the *conditioning* of the optimisation via Proposition 6.7; here they damage the *regularisation* by making the implied penalty an arbitrary function of the measurement unit. In both cases the defect is that a modelling decision has become a consequence of whether an analyst recorded an amount in shillings or in thousands of shillings. Standardising the inputs makes $\mathbb{E}[x_j^2] \approx 1$ for every j and restores the intended uniform penalty.

Solution to Exercise 6.9. Counting. Of 120,000 agents, 71% appear fewer than five times, that is $0.71 \times 120,000 = 85,200$ low-support agents, leaving 34,800 with support of five or more.

Without collapsing, the table has one row per agent plus one reserved out-of-vocabulary row:

$$(120,000 + 1) \times 16 = \mathbf{1,920,016} \text{ parameters.}$$

Collapsing the 85,200 low-support agents into the reserved index 0 leaves 34,800 individual rows plus that shared row:

$$(34,800 + 1) \times 16 = \mathbf{556,816} \text{ parameters,}$$

a saving of 1,363,200 parameters, or 71% of the table — unsurprisingly the same proportion as the collapsed vocabulary, since the cost is linear in the number of rows.

Which variant to submit, and why. Submit the collapsed variant. The statistical argument is decisive: each low-support agent has 16 free parameters fitted from at most four labelled observations. With a 4% base rate, the overwhelming majority of those agents have *no defaults at all* in the training set, so the fitted embedding encodes the accident of which handful of borrowers happened to use that kiosk rather than any property of the agent. That is memorisation, and it will not replicate on the test fold.

Three further arguments reinforce the choice. It trains the out-of-vocabulary path: index 0 receives gradient from 85,200 agents' worth of examples, so a genuinely unseen agent at inference time is mapped to a meaningful average rather than to an untrained initialisation — the failure mode of the pitfall in Section 6.13. It reduces the parameter count from roughly 400 times the rest of the network to roughly 115 times, which materially reduces the memory footprint of the served artefact. And it is more defensible under the governance expectations of Section 6.15, since a reviewer can be shown the support threshold as an explicit, documented modelling decision.

What evidence would settle it. Fit both variants under an identical protocol — same frozen splits, same preprocessing, same search budget, same early-stopping criterion — and compare, in order: test PR-AUC and expected cost; performance restricted to the *low-support agent subgroup*, which is where the two variants differ and where the aggregate metric may hide the difference; and calibration by county. A useful diagnostic beyond the headline metrics is to plot the norm of each learned embedding row against that agent's training support in the uncollapsed model: if the rare rows have drifted far from their initialisation, the model is fitting noise, and the plot makes that visible to a non-technical reviewer. Finally, the support threshold itself should be treated as a hyperparameter — 5 is a convention, not a result — and swept over, say, $\{3, 5, 10, 25\}$ on the validation fold.

Solution to Exercise 6.10. Proposition 6.14 establishes that $z \mapsto \sigma(z/T)$ is strictly increasing, hence order-preserving. Everything follows from distinguishing quantities that depend on the scores *only through their order* from those that depend on their *values*.

1. **ROC-AUC: unchanged.** It is a functional of the ranking alone (the probability that a random positive outranks a random negative), and the ranking is preserved.
2. **PR-AUC: unchanged.** Precision and recall at each operating point depend only on which examples fall above the cut, and the family of sets obtainable by thresholding is identical before and after scaling.

3. **Brier score: changes.** It is the mean squared difference between predicted probability and outcome, a function of the values themselves. This is precisely the quantity temperature scaling is fitted to improve; with $T = 1.8 > 1$ the logits shrink toward zero and the probabilities toward 0.5, which reduces the Brier score for an over-confident model and would increase it for an under-confident one.
4. **Expected cost at the fixed threshold $\hat{p} = 0.15$: changes.** A fixed threshold on the *probability* scale is not a fixed threshold on the score scale. Before scaling, $\hat{p} = 0.15$ corresponds to the logit $\text{logit}(0.15) = -1.7346$; after scaling, $\sigma(z/1.8) = 0.15$ requires $z = 1.8 \times (-1.7346) = -3.1223$. A different set of applicants is declined, so the confusion matrix and hence the cost move.
5. **Expected cost at the re-derived cost-optimal threshold: unchanged.** Optimising the threshold means minimising cost over the family of decision sets obtainable by thresholding the score. Since the transformation is a strictly increasing bijection, that family is identical before and after, so the minimum over it is identical too. The optimal *threshold value* moves; the optimal *cost* does not.

Remark B.1. The last two items together carry the practical message. Temperature scaling cannot improve the best achievable cost — item 5 — so it is not a performance technique. What it does is make the probability scale trustworthy, so that the analytically derived threshold is the right one. Under the cost matrix of the running case study the indifference point is where $\hat{p} \cdot 10,000 = (1 - \hat{p}) \cdot 800$, that is

$$\hat{p}^* = \frac{800}{10,800} = 0.0741.$$

Applying that threshold to *uncalibrated* scores gives the cost-optimal decision only by luck; applying it after calibration is a principled derivation. That is the entire value proposition: item 5 says calibration costs nothing in ranking, and this remark says it buys correctness in the mapping from scores to decisions.

Chapter 7 — Computer Vision with Convolutional Neural Networks

Solution to Exercise 7.1. Apply Proposition 7.1: (7.2) for shapes and (7.3) for counts. For `padding="same"` with odd f and stride s , the output side is $\lceil n_{\text{in}}/s \rceil$.

Layer	Output shape	Parameters
Input	$224 \times 224 \times 3$	0
Conv2D(64, 7, s=2, same)	$112 \times 112 \times 64$	$7^2 \cdot 3 \cdot 64 + 64 = 9,472$
MaxPooling2D(3, s=2, same)	$56 \times 56 \times 64$	0
Conv2D(128, 3, same)	$56 \times 56 \times 128$	$3^2 \cdot 64 \cdot 128 + 128 = 73,856$
MaxPooling2D(2)	$28 \times 28 \times 128$	0
Conv2D(256, 3, same)	$28 \times 28 \times 256$	$3^2 \cdot 128 \cdot 256 + 256 = 295,168$
MaxPooling2D(2)	$14 \times 14 \times 256$	0
GlobalAveragePooling2D	256	0
Dense(5, softmax)	5	$256 \cdot 5 + 5 = 1,285$
Total		379,781

With a $384 \times 384 \times 3$ input, **no parameter count changes at all**. The spatial dimensions become $192 \rightarrow 96 \rightarrow 96 \rightarrow 48 \rightarrow 48 \rightarrow 24$, and global average pooling still produces a 256-vector, so the dense head is unaffected. This is Proposition 7.1: kernel tensors are indexed by $(f, f, c_{\text{in}}, c_{\text{out}})$ and never by H or W .

What does change is *activation* memory and compute. The first convolution's output tensor grows from $112^2 \cdot 64$ to $192^2 \cdot 64$ entries, a factor of $(192/112)^2 \approx 2.94$; since every layer scales the same way, both peak memory and multiply-accumulate cost rise by roughly $(384/224)^2 \approx 2.94$. The common student error is to expect a bigger model; the correct statement is a bigger *computation* with the same model.

Solution to Exercise 7.2. Use $r_0 = 1, j_0 = 1, r_\ell = r_{\ell-1} + (f_\ell - 1)j_{\ell-1}, j_\ell = j_{\ell-1}s_\ell$.

ℓ	f_ℓ	s_ℓ	r_ℓ	j_ℓ
1	7	2	$1 + 6 \cdot 1 = 7$	2
2	3	2	$7 + 2 \cdot 2 = 11$	4
3	3	1	$11 + 2 \cdot 4 = 19$	4
4	3	1	$19 + 2 \cdot 4 = 27$	4
5	3	2	$27 + 2 \cdot 4 = 35$	8
6	3	1	$35 + 2 \cdot 8 = 51$	8

After layer 6 the jump is $j = 8$, so each further 3×3 stride-one layer adds $(3 - 1) \cdot 8 = 16$ pixels. Starting from 51: 67, 83, 99, 115. Three extra layers give 99, which does *not* exceed 100; **four** extra layers are required, reaching 115.

Two remarks worth making to students. First, receptive field grows in proportion to the accumulated stride, so layers placed after a downsample are far more "efficient" per layer than early ones — this is why architectures front-load striding. Second, the theoretical receptive field is an upper bound on influence, not a description of it: the *effective* receptive field is

approximately Gaussian and considerably smaller, because centre pixels contribute through many more paths than edge pixels.

Solution to Exercise 7.3. Take the one-dimensional input $x = (1, 0, 0, 0, 0)$, kernel $K = (1, 0, 0)$, padding $p = 0$, stride $s = 2$. The admissible left edges are $t = 0, 2$ (and $t = 3$ is excluded since $3 + 2 > 5$), so

$$\mathcal{C}(x) = (x_0, x_2, x_4) = (1, 0, 0).$$

Now shift the input by one position: $x' = (0, 1, 0, 0, 0)$. Then

$$\mathcal{C}(x') = (x'_0, x'_2, x'_4) = (0, 0, 0).$$

The output of the shifted input is the zero vector, which is not any translate of $(1, 0, 0)$: the feature has disappeared entirely rather than moved. This violates equivariance. Contrast a shift of *two* positions, $x'' = (0, 0, 1, 0, 0)$, which gives $\mathcal{C}(x'') = (0, 1, 0)$ — exactly $\mathcal{C}(x)$ translated by $1 = \tau/s$, as Proposition 7.2 predicts.

Practical consequence. A borrower photographing an identity card by hand produces images whose content is shifted by arbitrary amounts, not by multiples of the stride. A strided network's response to the same document therefore varies with sub-stride jitter, and since the accumulated stride of a ResNet-50 backbone is 32, the relevant jitter is not one pixel but up to thirty-one.

The two mitigations named in Section 7.2.2. First, include small random translations in the augmentation policy, so that approximate invariance is learned from data rather than assumed from the architecture; by the argument of Section 7.4.5 this is legitimate precisely because a small shift genuinely does preserve the label. Second, avoid evaluating on a single fixed crop: averaging predictions over a small ensemble of crops averages out the phase of the sampling grid, at a cost linear in the number of crops.

Solution to Exercise 7.4. With $C = 512$, a plain 3×3 convolution costs $9C^2 = 9 \cdot 512^2 = 2,359,296$ weights.

A bottleneck of width $b = 128$ costs

$$\underbrace{1^2 \cdot 512 \cdot 128}_{65,536} + \underbrace{3^2 \cdot 128^2}_{147,456} + \underbrace{1^2 \cdot 128 \cdot 512}_{65,536} = 278,528,$$

so the plain block is $2,359,296/278,528 = 8.47$ times more expensive.

Break-even solves $9C^2 = 2Cb + 9b^2$, i.e. $9b^2 + 2Cb - 9C^2 = 0$, whose positive root is

$$b = \frac{-2C + \sqrt{4C^2 + 324C^2}}{18} = \frac{C(\sqrt{82} - 1)}{9} \approx 0.895C \approx 458.3.$$

ResNet uses $b = C/4 = 128$, far below break-even, and the reason is that parameter parity is the wrong objective. The design goal is to spend a fixed

budget on as much *depth* as possible, because Theorem 7.1 makes depth trainable and Section 7.2.4 shows it to be a better use of a weight budget than width. Choosing b near break-even would buy no depth at all. Choosing b much smaller than $C/4$ would eventually starve the 3×3 layer of capacity; $C/4$ is the empirical compromise, and the $8.5\times$ saving is what funds the 16 blocks of stages 2–5.

Solution to Exercise 7.5. (a) $\alpha\beta^2\gamma^2 = 1.2 \cdot 1.1^2 \cdot 1.15^2 = 1.2 \cdot 1.21 \cdot 1.3225 = 1.9203$, which is ≈ 2 as required.

(b) Using $d = \alpha^\phi$, $w = \beta^\phi$, $r = \gamma^\phi$ and the FLOP multiplier $(\alpha\beta^2\gamma^2)^\phi$ from Proposition 7.4:

ϕ	d	w	r	FLOP multiplier
1	1.200	1.100	1.150	1.920
2	1.440	1.210	1.322	3.687
3	1.728	1.331	1.521	7.081

(c) Solve $1.9203^\phi = 8$:

$$\phi = \frac{\ln 8}{\ln 1.9203} = \frac{2.0794}{0.6526} = 3.187.$$

The implied resolution factor is $\gamma^{3.187} = 1.15^{3.187} = 1.561$, so the input side becomes $224 \times 1.561 \approx 350$ pixels. In practice one would round to a network-friendly value such as 352 (a multiple of 32, the total stride), and note that depth and width scale to $1.20^{3.187} = 1.77$ and $1.10^{3.187} = 1.37$ respectively.

(d) The arithmetic gives a compute budget, not an accuracy guarantee, and the two should not be confused. Compound scaling was calibrated on ImageNet, where 1.2 million labelled images make capacity the binding constraint. On a five-class document task with a few thousand images the binding constraint is data, and Figure 7.14 shows exactly which regime that puts us in: the flat left-hand region where additional capacity buys nothing. A larger ϕ would raise the parameter count, the training time and the deployed size, increase the risk of overfitting, and in all likelihood lower the held-out macro- F_1 . The defensible use of this calculation is the reverse direction: given a fixed serving budget, it tells you which family member you can afford.

Solution to Exercise 7.6. Write $\bar{A}^k = (H'W')^{-1} \sum_{i,j} A_{i,j}^k$ and $y^c = \sum_k w_k^c \bar{A}^k + b^c$. Since $\bar{A}^k$ is linear in the entries of A^k with coefficient $1/(H'W')$ on each, and b^c is a constant,

$$\frac{\partial y^c}{\partial A_{i,j}^k} = w_k^c \cdot \frac{\partial \bar{A}^k}{\partial A_{i,j}^k} = \frac{w_k^c}{H'W'} \quad \text{for every } (i,j).$$

Substituting into the definition of α_k^c ,

$$\alpha_k^c = \frac{1}{H'W'} \sum_{i,j} \frac{w_k^c}{H'W'} = \frac{1}{H'W'} \cdot H'W' \cdot \frac{w_k^c}{H'W'} = \frac{w_k^c}{H'W'}.$$

Hence

$$L^c = \text{ReLU} \left(\sum_k \frac{w_k^c}{H'W'} A^k \right) = \frac{1}{H'W'} \text{ReLU} \left(\sum_k w_k^c A^k \right),$$

using $\text{ReLU}(\lambda u) = \lambda \text{ReLU}(u)$ for $\lambda > 0$. The map is therefore the class activation map up to a positive constant, and since Grad-CAM maps are normalised to $[0, 1]$ before display, the constant is invisible.

The bias. The term b^c contributes nothing, because it does not depend on A^k and so $\partial b^c / \partial A_{i,j}^k = 0$. Intuitively, a class-independent offset shifts every score equally and therefore carries no spatial information.

What breaks with a hidden layer. If the head is

$$y^c = \sum_m v_m^c \text{ReLU} \left(\sum_k u_{mk} \bar{A}^k \right),$$

the derivative becomes $\partial y^c / \partial A_{i,j}^k = (H'W')^{-1} \sum_m v_m^c u_{mk} \mathbb{1}[\text{unit } m \text{ active}]$, which still does not depend on (i, j) but now depends on the *input* through the activation pattern. The proportionality to a fixed weight vector is lost: there is no longer a single set of class weights w_k^c , and α_k^c becomes example-specific. Grad-CAM remains well defined — that is precisely the generalisation it offers — but the identification with CAM, and with it the guarantee that the map reflects a fixed linear read-out, no longer holds.

Solution to Exercise 7.7. (a) By Proposition 7.5,

$$t^* = \frac{C_{\text{FP}}}{C_{\text{FP}} + C_{\text{FN}}} = \frac{800}{800 + 10,000} = \frac{800}{10,800} = 0.0741.$$

Escalate to human review whenever the tamper probability exceeds about 7.4%.

(b) The always-approve policy never raises a false positive and incurs a false negative on every tampered document, so its expected cost per application is

$$\pi C_{\text{FN}} = 0.04 \times 10,000 = \text{KES } 400.$$

For comparison, always-reject costs $(1 - \pi)C_{\text{FP}} = 0.96 \times 800 = \text{KES } 768$. Always-approve is therefore the better of the two trivial policies, and it is the baseline to beat: a model that cannot get below KES 400 per application is not earning its keep.

(c) Accuracy at $t = 0.5$ is misleading for two compounding reasons. It is measured at the wrong threshold: 0.5 is optimal only when $C_{FP} = C_{FN}$, and here the costs differ by a factor of 12.5, so the reported operating point is not one the business would ever choose. And accuracy itself is uninformative under a 4% positive rate, because the always-approve policy achieves 96% — a number that sounds excellent and corresponds to catching no fraud at all.

Solution to Exercise 7.8. *Inference mode* uses the stored running statistics:

$$\hat{z}_R = \frac{1.5 - 0}{1} = 1.5, \quad \tilde{z}_R = 1.2 \times 1.5 - 0.3 = 1.5.$$

Training mode uses the current batch statistics:

$$\hat{z}_B = \frac{1.5 - 0.8}{2.5} = 0.28, \quad \tilde{z}_B = 1.2 \times 0.28 - 0.3 = 0.036.$$

The difference is $1.5 - 0.036 = 1.464$, i.e. the activation that the head was trained to read as 1.5 arrives as 0.036 — a change of over 97% of its value, and a change of sign relative to the shift β .

Why this compounds. ResNet-50 contains 53 batch-normalisation layers. The discrepancy is an affine map of z with slope $\gamma (\sigma_R^{-1} - \sigma_B^{-1})$, so each layer applies its own such distortion and the distortions compose multiplicatively through the stack. Even a modest per-layer factor of, say, 0.7 in effective scale becomes $0.7^{53} \approx 2 \times 10^{-9}$ over the whole backbone. The features reaching the head bear no relation to the features it was trained on in Phase A, which is why validation accuracy does not degrade gracefully but collapses within one or two epochs of unfreezing. Note also that the corruption is *persistent*: the running statistics themselves are overwritten during training, so the damage survives a switch back to inference mode and the checkpoint on disk is already spoiled. This is why the fix must be applied before training, not diagnosed afterwards.

Solution to Exercise 7.9.

Transformation	Cassava	Document	Reason
Horizontal flip	yes	no	A mirrored leaf is still a diseased leaf; mirrored text on an ID is itself evidence of tampering.
$\pm 90^\circ$ rotation	yes	no	Leaves have no canonical orientation; documents do, and a sideways ID is a capture failure the model should flag, not ignore.
$\pm 8^\circ$ rotation	yes	yes	Both are photographed handheld; small tilt is ordinary capture noise in each domain.
Brightness $\pm 20\%$	yes	yes	Field lighting and indoor lighting both vary widely; brightness is not diagnostic of either label.
Random JPEG compression	yes	caution	Realistic for leaves. For documents, compression artefacts can be a re-photography signal, so this risks erasing evidence.
Random 30×30 occlusion	yes	yes	Both benefit: it discourages reliance on a single region and mimics fingers, glare and shadow.

Formal argument. The decomposition in Section 7.4.5 writes the augmented risk as

$$(\bar{f}(\mathbf{x}) - y)^2 + \text{Var}_T[f(T\mathbf{x})],$$

so the objective explicitly rewards a model whose output does *not* vary across G . That is desirable only when the label is genuinely constant on the orbit $\{T\mathbf{x} : T \in G\}$.

Suppose it is not. Take $\mathbf{x}$ a genuine document with $y = 0$, and T the horizontal flip, with $y(T\mathbf{x}) = 1$ because mirrored text indicates tampering. The augmented objective still trains $f(T\mathbf{x})$ towards $y(\mathbf{x}) = 0$, so the pair $(T\mathbf{x}, 0)$ enters the training set as a *mislabelled* example. Worse, the variance term actively penalises the model for distinguishing $\mathbf{x}$ from $T\mathbf{x}$ — it is not merely noise but a regulariser pulling towards the wrong invariance. The Bayes-optimal classifier for the augmented distribution is therefore strictly worse than the Bayes-optimal classifier for the true one, and no amount of additional data will recover the gap.

Solution to Exercise 7.10. (a) Storage is $P \times b/8$ bytes.

Model	Parameters	float32	float16	int8
EfficientNetV2-B0	7.2M	28.8 MB	14.4 MB	7.2 MB
ResNet50V2	25.6M	102.4 MB	51.2 MB	25.6 MB

(b) Against a 10 MB cap, only **EfficientNetV2-B0 in int8** (7.2 MB) qualifies. B0 in float16 is 14.4 MB and still fails; ResNet50V2 fails at every precision, needing better than $3\times$ compression beyond int8 to fit — achievable

only with pruning or distillation, which is a different project. This is the concrete sense in which backbone choice is a product decision rather than an accuracy decision: on this requirement the ResNet is excluded before its accuracy is even measured.

(c) For values in $[-3, 3]$ quantised symmetrically to 8 bits, the step is

$$\Delta = \frac{2r}{255} = \frac{6}{255} = 0.02353,$$

and the maximum rounding error is $\Delta/2 = 0.01176$, i.e. about 0.2% of the full range.

Whether that matters is best judged against the noise already in the system. The cassava dataset carries substantial label noise — disease grades assigned from photographs by human annotators disagree at rates in the several-percent range — and the model’s own uncertainty is far larger than a 0.2% perturbation of individual weights. Empirically, int8 post-training quantisation with a representative dataset typically costs well under one point of accuracy on this class of task. The honest framing for a design review is therefore: the quantisation error is roughly an order of magnitude smaller than the label noise, so it should not be the first thing to blame if accuracy disappoints. The caveats worth stating are that per-tensor quantisation is riskier than per-channel for layers whose weight ranges vary widely across channels, and that the effect must be *measured* on the held-out set rather than assumed — a one-line comparison of the float and TFLite models that belongs in every deployment notebook.

Chapter 8: Sequence Models and Time-Series Forecasting

Solution to Exercise 8.1 (The covariate taxonomy).

- (a) **County:** static, **s**. It does not vary over the horizon, and it is available at every origin.
- (b) **Transactions in the previous hour:** past-observed, **z**. It is measured, so it exists only up to the origin.
- (c) **Tomorrow being a public holiday:** known-future, **x**. It comes from a calendar published in advance.
- (d) **Tomorrow’s rainfall from the published forecast:** known-future, **x**. What makes it admissible is not that it is accurate but that it is *available* at 18:00 today. Its errors degrade performance; they do not cause leakage.

- (e) **Tomorrow's actual rainfall:** unusable. It is not known at the origin, so a model trained on it cannot be run in production at all. Including it produces excellent backtests and a system that fails on its first live day.
- (f) **Total commission over the following week:** unusable, and worse than (e), because it is a function of the target itself over a window that extends beyond the horizon. This is target leakage in the sense of Chapter 2, and it will produce near-perfect validation scores.

The general test is the one in Figure 8.5: ask whether the value could be written into a file at 18:00 today. If not, it is not a feature.

Solution to Exercise 8.2 (Window arithmetic). (a) A window occupies $L + H = 192$ consecutive observations, and the number of starting positions in a block of length n is $n - (L + H) + 1$. For the undivided series,

$$8,760 - 192 + 1 = 8,569 \text{ windows.}$$

(b) The chronological split gives blocks of 6,132, 1,314 and 1,314 observations. Cutting windows inside each block:

$$\text{train: } 6,132 - 191 = 5,941,$$

$$\text{valid: } 1,314 - 191 = 1,123,$$

$$\text{test: } 1,314 - 191 = 1,123,$$

totalling 8,187 windows, so $8,569 - 8,187 = 382$ are lost. The lost windows are exactly the 2×191 that would have straddled one of the two boundaries.

(c) Each discarded window would have had its input in one block and part of its target in the next, so a model trained on it would have seen validation or test observations during training. Losing 4.5% of the examples is a small price for a validation number that means what it claims; the alternative is a larger training set and an unusable estimate of generalisation.

Solution to Exercise 8.3 (The pinball loss and the quantile). (a) With $\hat{y}^{(q)} = 100$ and $e = y - \hat{y}^{(q)}$:

	$y = 120 \ (e = +20)$	$y = 80 \ (e = -20)$
$q = 0.9$	$0.9 \times 20 = 18$	$0.1 \times 20 = 2$
$q = 0.1$	$0.1 \times 20 = 2$	$0.9 \times 20 = 18$

At $q = 0.9$ an under-forecast costs nine times as much as an over-forecast of the same magnitude, so the minimiser is pushed high; at $q = 0.1$ the asymmetry is reversed. The loss encodes the risk appetite directly, which is why the choice of q is a business decision rather than a modelling one.

(b) The distribution function is

$$F(10) = 0.1, \quad F(20) = 0.3, \quad F(30) = 0.7, \quad F(40) = 0.9, \quad F(50) = 1.0.$$

By Proposition 8.1 the minimiser is any a with $F(a) = q$, interpreted as the smallest a with $F(a) \geq q$ when F jumps. For $q = 0.5$, F first reaches 0.5 at $a = 30$; for $q = 0.9$, at $a = 40$. Direct evaluation confirms both: $\mathbb{E}[L_{0.5}(Y, a)]$ is minimised at $a = 30$ with value 4.0, and $\mathbb{E}[L_{0.9}(Y, a)]$ at $a = 40$ with value 2.0.

(c) At $q = 0.5$ the derivative computed in the proof is $g'(a) = F(a) - 1/2$, so the minimiser is the *median*. For a right-skewed distribution the median lies below the mean, so an MAE-trained model predicts below the mean by construction. This is correct behaviour: MAE was chosen precisely because the occasional enormous hour should not drag the forecast upward. If the mean is genuinely wanted, use squared error, and accept that a single flash crowd will dominate the fit.

Solution to Exercise 8.4 (Metrics that disagree). The model's absolute errors are

$$(5, 10, 10, 10, 10, 5), \quad \text{so MAE} = 50/6 \approx 8.33,$$

and the seasonal-naïve absolute errors are

$$(10, 5, 5, 10, 5, 10), \quad \text{so MAE}_{\text{naïve}} = 45/6 = 7.5.$$

Hence

$$\text{MASE} = \frac{8.33}{7.5} \approx 1.11, \quad \text{sMAPE} \approx 6.79\%.$$

The sMAPE looks respectable and is misleading on its own. The MASE exceeds one, which says the model is *worse than seasonal naïve* on this sample. The recommendation is not to tune the network. It is to ship seasonal naïve, which requires no training pipeline, no GPU and no drift surface, and to redirect the effort to data quality and covariates. Any deep-forecasting review should begin with this comparison, and a good many end there.

Solution to Exercise 8.5 (Why plain recurrence forgets). (a) Proposition 8.2 gives the bound λ^k with $\lambda = 0.9$ and $k = 168$:

$$0.9^{168} \approx 2.05 \times 10^{-8}.$$

In single precision, whose resolution near one is about 10^{-7} , this is numerically indistinguishable from zero: the first observation of the window has no influence on the loss.

(b) Solve $0.9^k < 10^{-6}$, giving $k > \ln(10^{-6}) / \ln(0.9) \approx 131.1$, so $k = 132$ steps. Roughly five and a half days of hourly data suffice to extinguish the signal.

(c) By Theorem 8.1 the factor along that coordinate is $\prod_j f_j = 0.99^{168} \approx 0.185$. The gradient is attenuated by a factor of about five rather than by eight orders of magnitude, and with $f = 0.999$ it would be 0.845, essentially undamped.

(d) In the plain RNN the product is over $\mathbf{W}_h^\top \text{diag}(\tanh')$, a fixed learned matrix acting at every step; keeping its norm near one is a global constraint on a matrix that is simultaneously responsible for all the computation the layer performs. In the LSTM the product is over *gate activations*, which are computed per step, per coordinate, from the input, so the network can hold one coordinate open for long-range memory while others forget quickly. The mechanism is controllable because it is separate from the computation it protects.

Solution to Exercise 8.6 (Counting gate parameters). (a) With $d_x = 32$ and $d_h = 64$, the common factor is $(d_x + d_h + 1)d_h = 97 \times 64 = 6,208$. Hence

$$\text{LSTM: } 4 \times 6,208 = 24,832, \quad \text{GRU: } 3 \times 6,208 = 18,624,$$

a ratio of exactly 4/3, since the LSTM computes four internal layers and the GRU three.

(b) A single GRU of width 128 has $3(32 + 128 + 1) \times 128 = 3 \times 161 \times 128 = 61,824$ parameters. Two stacked GRUs of width 64 have $3(32 + 64 + 1) \times 64 = 18,624$ for the first and $3(64 + 64 + 1) \times 64 = 24,768$ for the second, totalling 43,392: about thirty percent fewer.

Beyond the count, the stack composes two nonlinear recurrences, so it can represent temporal patterns of patterns, which a single wider layer cannot. What is lost is capacity per layer, so each state is a narrower summary, and depth brings the usual optimisation difficulty. The stack is also strictly slower per step in wall-clock time, because the two layers cannot be executed in parallel.

Solution to Exercise 8.7 (Why attention divides by $\sqrt{d_k}$). (a) Write $\mathbf{q}^\top \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i$. By independence and zero means, $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i] \mathbb{E}[k_i] = 0$, so the sum has mean zero. The terms are uncorrelated, so variances add, and $\text{Var}[q_i k_i] = \mathbb{E}[q_i^2 k_i^2] = \mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1$, giving $\text{Var} = d_k$.

(b) For $d_k = 64$ the standard deviation is $\sqrt{64} = 8$, so scores of ± 8 are entirely typical. For the unscaled vector $(8, 0, -8)$,

$$\text{softmax} \approx (0.99966, 0.00034, 1.1 \times 10^{-7}),$$

which is a hard maximum in all but name. Scaling by $1/\sqrt{64} = 1/8$ gives scores $(1, 0, -1)$ and

$$\text{softmax} \approx (0.665, 0.245, 0.090),$$

a genuine weighted average.

(c) The Jacobian of the softmax is $\text{diag}(\mathbf{p}) - \mathbf{p}\mathbf{p}^\top$, whose entries are products of the weights and therefore vanish when one weight approaches one and the rest approach zero. In the unscaled case the gradient with respect to the scores is numerically zero, so the query and key projections receive no learning signal, and the layer is frozen at whatever pattern its initialisation happened to produce. The scaling keeps the softmax in the region where it has a usable derivative, which is the same consideration that governed initialisation in Chapter 6.

Solution to Exercise 8.8 (Attention against recurrence). (a) From Proposition 8.3, attention costs $\mathcal{O}(L^2d)$ and recurrence $\mathcal{O}(Ld^2)$. With $L = 168$ and $d = 64$,

$$L^2d = 168^2 \times 64 = 1,806,336, \quad Ld^2 = 168 \times 64^2 = 688,128,$$

a ratio of 2.625. Attention is about two and a half times more expensive here.

(b) The costs coincide when $L^2d = Ld^2$, that is when $L = d$. The ratio is exactly L/d , which is the useful form to remember: attention is cheaper than recurrence whenever the sequence is shorter than the model is wide.

(c) At $L = 4,096$ the ratio is $4096/64 = 64$: attention costs 1.07×10^9 operations against 1.68×10^7 . The quadratic term dominates completely, which is what motivates the efficient-attention variants of Chapter 9.

(d) Attention retains constant gradient path length between any two positions, so no long-range dependency has to survive a chain of updates, and full parallelism across positions, since no output waits on another. The disadvantage that grows is memory: the $L \times L$ attention matrix must be materialised per head, which is $\mathcal{O}(L^2)$ and becomes the binding constraint long before the operation count does.

Solution to Exercise 8.9 (Causal masking, carefully). (a) Position i may attend to positions $j \leq i$, so the surviving pairs are the lower triangle including the diagonal:

$$\frac{L(L+1)}{2} = \frac{168 \times 169}{2} = 14,196 \quad \text{of} \quad L^2 = 28,224,$$

a fraction of $(L+1)/(2L) \approx 50.3\%$. Masking therefore saves slightly less than half the computation, and asymptotically exactly half.

(b) The full softmax of $(2.0, 1.0, 0.5)$ is $(0.6285, 0.2312, 0.1402)$. Zeroing the last entry afterwards leaves $(0.6285, 0.2312, 0)$, which sums to 0.8598 rather than to one. Masking correctly, by setting the third score to $-\infty$ before the softmax, gives $(0.7311, 0.2689, 0)$. The colleague's first weight is too small by $0.7311 - 0.6285 = 0.1026$, an error of about fourteen percent, and the output is systematically shrunk towards zero because the weights no longer sum to one. The error grows with the number of masked positions,

so it is worst at the start of the horizon, exactly where accuracy matters most.

(c) Without the mask, each position attends to later positions and the model can read the answer it is being asked to predict. Training and validation loss both fall to near zero, the curves look ideal, and the model produces nonsense the first time it is run at an origin where the future genuinely does not exist. The signature is a validation score that is not merely good but implausible, and it should always be investigated rather than celebrated.

Solution to Exercise 8.10 (Variable selection, implemented). A minimal implementation, continuing the notation of Listing 8.4:

```
def variable_selection(x, static_ctx, n_vars, d):
    """x: [B, T, n_vars]. Returns (selected [B, T, d], weights [B, T, n_vars
    ])."""
    # 1. embed each variable separately, then transform it with its own GRN
    per_var = [grn(x[..., j:j+1], d) for j in range(n_vars)] # n_vars x
    [B,T,d]
    stacked = tf.stack(per_var, axis=-2) # [B, T,
    n_vars, d]

    # 2. selection weights from a GRN over the flattened inputs, conditioned
    # on the static context broadcast across time
    flat = layers.Reshape((-1, n_vars * d))(stacked) # [B, T,
    n_vars*d]
    ctx = tf.repeat(static_ctx[:, None, :], tf.shape(flat)[1], axis=1)
    w = layers.Dense(n_vars, activation="softmax")(
        grn(layers.Concatenate(-1)([flat, ctx]), d)) # [B, T,
    n_vars]

    # 3. weighted sum over the variable axis
    selected = tf.reduce_sum(stacked * w[..., None], axis=-2) # [B, T, d
    ]
    return selected, w

past_sel, past_w = variable_selection(past, s, n_past, d)
enc, h, c = layers.LSTM(d, return_sequences=True, return_state=True)(
    past_sel, initial_state=[layers.Dense(d)(s)] * 2)
```

Three implementation points are worth noting. The softmax runs over the variable axis and not over time, so the weights answer “which input mattered at this step” rather than “which step mattered”. Each variable gets its *own* GRN, so a variable that needs a nonlinear transformation can have one without forcing the same on the others. And the static context enters the weight computation, which is what allows a rural agent and a city-centre agent to rely on different inputs at the same hour.

Using the weights. Averaging `past_w` over the backtest test folds gives a global importance ranking, which is the chart of Figure 8.26 and goes

directly into the validation pack that the CBK Digital Credit Providers Regulations expect: it documents which inputs the model relies on, in a form a non-specialist reviewer can read, and it is cheap enough to regenerate on every retrain, so it can be tracked over time as a drift signal.

If the county embedding ranked in the top three, the correct response is investigation rather than removal. County is a plausible proxy for ethnicity and income, so a high weight raises a fairness question, but it is also a legitimate carrier of genuine geographic signal such as market days and harvest calendars. Three steps follow. First, audit the residuals by county: a consistently negative mean error in particular counties means those agents are systematically under-provisioned and losing income, which is a defect regardless of what the aggregate MASE says. Second, test whether the county signal survives once the underlying mechanisms are supplied explicitly, by adding market-day and harvest-calendar features and refitting; if the county weight collapses, it was standing in for those. Third, if it does not collapse, document the residual dependence, the segment-level error table and the mitigation considered, and escalate the decision to keep it rather than making it silently in a configuration file. Deleting the feature without any of this merely hides the dependence, since the model will reconstruct county from the agent embedding.

B.1 Chapter 9: Transformers, LLMs and retrieval

Solution to Exercise 9.1. (a) With $d_k = 2$ the scale factor is $1/\sqrt{2}$, and $\mathbf{QK}^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{pmatrix}$, so

$$\mathbf{S} = \frac{\mathbf{QK}^\top}{\sqrt{2}} = \begin{pmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{pmatrix}.$$

(b) Write $e = \exp(0.7071) = 2.0281$, so that $e^2 = \exp(1.4142) = 4.1133$. Row 1 has exponentials $(e, 1, e)$ summing to 5.0562, and row 2 has the same values permuted; row 3 has exponentials (e, e, e^2) summing to 8.1695. Normalising each row,

$$\alpha = \begin{pmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{pmatrix},$$

and multiplying by $\mathbf{V}$,

$$\mathbf{O} = \alpha \mathbf{V} = \begin{pmatrix} 1.2033 & 1.1922 \\ 0.7967 & 2.0056 \\ 1.0000 & 1.4965 \end{pmatrix}.$$

Two sanity checks worth making every time: each row of α sums to one, and every entry of $\mathbf{O}$ lies inside the range spanned by the corresponding column of $\mathbf{V}$, here $[0, 2]$ and $[0, 4]$. Attention averages; it never extrapolates.

(c) With the causal mask, row 1 can attend only to position 1, so $\alpha_1 = (1, 0, 0)$ and $\mathbf{o}_1 = \mathbf{v}_1 = (2, 0)$. Row 2 attends to positions 1 and 2 with exponentials $(1, e)$, giving $\alpha_2 = (0.3302, 0.6698)$ and $\mathbf{o}_2 = (0.6605, 2.6790)$. Row 3 is unchanged, since it could already see everything. The first output row is simply a copy of the first value vector, which is exactly right: at the first position a decoder has seen no context at all, and any other behaviour would mean the model was reading tokens it will be asked to predict.

Solution to Exercise 9.2. (a) By Proposition 9.1, $\text{Var}(\mathbf{q}^\top \mathbf{k}) = d_k = 64$, so the raw scores have standard deviation $\sqrt{64} = 8$.

(b) Without the scaling,

$$\text{softmax}(8.0, 2.4, -1.6) = (0.9962, 0.0037, 0.0001),$$

whereas dividing by $\sqrt{d_k} = 8$ turns the logits into $(1.0, 0.3, -0.2)$ and gives

$$\text{softmax}(1.0, 0.3, -0.2) = (0.5562, 0.2762, 0.1675).$$

(c) For the largest coordinate, $p(1 - p) = 0.9962 \times 0.0038 = 0.0037$ unscaled, against $0.5562 \times 0.4438 = 0.2468$ scaled: a factor of roughly 66. The unscaled model is not merely peaked, it is *stuck*. Its attention distribution has collapsed onto one token, the output is a copy of one value vector, and the gradient reaching the query and key projections is two orders of magnitude smaller, so the model cannot learn its way out. The $\sqrt{d_k}$ is therefore not cosmetic normalisation but the condition under which the layer is trainable at all, and it is the same saturation pathology as a sigmoid driven to its tails.

Solution to Exercise 9.3. (a) From Proposition 9.2 the ratio of attention cost to projection-plus-FFN cost is $2T^2d/12Td^2 = T/(6d)$. With $d = 4096$, $6d = 24,576$:

T	512	2048	8192	32768
ratio $T/(6d)$	0.021	0.083	0.333	1.333
attention share of the layer	2.0%	7.7%	25.0%	57.1%

(The share is $\text{ratio}/(1 + \text{ratio})$.)

(b) Attention passes half the layer when the ratio reaches 1, that is at $T = 6d = 24,576$ tokens.

(c) The compliance assistant. At $T = 64$ attention is 0.26% of the classifier's layer cost, so optimising it would be optimising nothing, whereas at $T \approx 6000$ the assistant is already paying roughly a quarter of its compute on attention and that share grows with every extra retrieved chunk. The

one-sentence argument: effort spent shortening the assistant’s context, by reranking to five chunks instead of stuffing fifty, buys a super-linear saving, while the same effort spent on the classifier buys nothing measurable.

Solution to Exercise 9.4. (a) Fertility is tokens per word: $156/120 = 1.30$ for tokenizer A and $372/120 = 3.10$ for tokenizer B, a ratio of 2.38.

(b) The corpus is $2 \times 10^6 \times 24 = 48$ million words. Under A that is $48 \times 1.30 = 62.4$ million tokens; under B, $48 \times 3.10 = 148.8$ million. At USD 0.15 per thousand tokens the input-token cost is USD 9,360 under A and USD 22,320 under B, a difference of USD 12,960, which at 130 KES/USD is about **KES 1.68 million** per year, for identical data and identical work.

(c) The calculation prices only the tokens. It misses that the same fertility shrinks the *effective context window*: a 8k-token budget holds roughly 2,580 words under B against 6,150 under A, so tokenizer B forces shorter prompts, fewer retrieved chunks or more truncation. It also misses the quality effect, since a word shattered into five fragments carries less usable signal than one that survives as a single token, which typically shows up as a per-language accuracy gap rather than as a line on an invoice.

Solution to Exercise 9.5. (a) $D \approx 20N = 20 \times 1.5 \times 10^9 = 3.0 \times 10^{10}$ tokens (30 billion). Then $C \approx 6ND = 6 \times 1.5 \times 10^9 \times 3.0 \times 10^{10} = 2.7 \times 10^{20}$ FLOPs.

(b) At 1.2×10^{14} FLOP/s sustained, $C/(1.2 \times 10^{14}) = 2.25 \times 10^6$ seconds = 26.0 GPU-days. Note the policy implication: this is one card for under a month, or eight cards for three days, which is why compute-optimal pre-training at the 1.5B scale is within reach of a university lab, unlike anything above about 10B.

(c) Chinchilla minimises *training* loss for a fixed training budget, but a deployed model pays inference cost on every request for the life of the product, and that cost scales with N , not with D . Spending extra training tokens on a smaller model is therefore a rational trade: you accept a worse loss-per-training-FLOP in exchange for a permanently cheaper model to serve, and for PesaCred, where the classifier scores millions of messages a month, the serving side dominates the arithmetic entirely.

Solution to Exercise 9.6. (a) For one 4096×4096 matrix at $r = 16$, the adapter holds $r(d + k) = 16 \times 8192 = 131,072$ parameters against $dk = 16,777,216$, a fraction of **0.78%**.

(b) Across 32 layers with four projections each,

$$32 \times 4 \times 131,072 = 16,777,216 \approx \mathbf{16.8} \text{ million parameters.}$$

At two bytes each in bfloat16, the saved adapter is about **33.6 MB**.

(c) Full fine-tuning of a 7B model produces a 14 GB artifact per task, roughly 400 times larger. Serving eight tasks means eight 14 GB checkpoints, none of which can share GPU memory, so you need either eight

loaded models or eight sequential loads. With adapters you load *one* base model and hold all eight adapters simultaneously in about 270 MB, switching between them per request. The operational consequences follow immediately: one deployment artifact to version instead of eight, a storage bill three orders of magnitude smaller, and, because of the merge property in Proposition 9.3, no inference penalty for any of it.

Solution to Exercise 9.7. (a) Base weights: $7 \times 10^9 \times 4.13/8 = 3.61$ GB. Adapter: $40 \times 10^6 \times 2 = 0.08$ GB. Optimizer and the associated master copy: $40 \times 10^6 \times 14 = 0.56$ GB. Total fixed cost ≈ 4.25 GB.

(b) On a 16 GB card, roughly $16 - 4.25 = 11.75$ GB remains for activations, attention workspace, the CUDA context and fragmentation. In practice budget one to two GB for context and fragmentation, leaving about 10 GB of usable activation memory, which is comfortable at $T = 64$ for SMS and tight at multi-thousand-token sequences.

(c) Any two of the following. Reduce the per-device batch size and raise the gradient-accumulation steps, which leaves the effective batch unchanged. Enable gradient checkpointing, trading roughly 30% extra compute for a large activation saving. Shorten the maximum sequence length to the true 99th percentile of the corpus rather than to a round number. Or use paged optimizers, so that transient spikes spill to host memory instead of aborting the run.

Solution to Exercise 9.8. The ranks are 1, 3, 2, miss, 1, 5, 2, 1, miss, 4 over ten queries. A miss contributes zero to every metric.

(a) Recall@3 counts ranks ≤ 3 : queries 1, 2, 3, 5, 7, 8, which is six of ten, so **0.60**.

(b) Recall@5 additionally counts ranks 5 and 4: eight of ten, so **0.80**.

(c) $\text{MRR} = \frac{1}{10} (1 + \frac{1}{3} + \frac{1}{2} + 0 + 1 + \frac{1}{5} + \frac{1}{2} + 1 + 0 + \frac{1}{4}) = \frac{4.7833}{10} = \mathbf{0.4783}$.

(d) The reranker moves every gold chunk already in the top 5 to rank 1, so the eight hits all become rank 1 and the two misses stay missing. Recall@3 rises to **0.80**, Recall@5 stays at **0.80**, and MRR rises to **0.80**. Recall@5 is the metric the reranker cannot improve, and the reason is structural rather than incidental: a reranker only reorders the candidate set handed to it by the retriever, so it can never surface a chunk the first-stage retrieval failed to return. This is the practical argument for retrieving generously, say $k = 50$, before reranking to five: recall is bought in the first stage and precision in the second, and no amount of second-stage cleverness recovers a document that was never a candidate.

Solution to Exercise 9.9. (a) All five vectors are unit length, so the cosine similarity is just the inner product with $\mathbf{q} = (0.6, 0.8, 0)$:

$$\cos(\mathbf{q}, \mathbf{a}) = 0.96, \quad \cos(\mathbf{q}, \mathbf{d}) = 0.936, \quad \cos(\mathbf{q}, \mathbf{b}) = 0.80, \quad \cos(\mathbf{q}, \mathbf{c}) = 0.36.$$

The top-2 ranking is **a** then **d**.

(b) For unit vectors the denominator of the cosine is 1, so cosine similarity and inner product coincide and `IndexFlatIP` computes exactly the quantity we want, with no per-query normalisation cost. If the vectors are not normalised, inner product silently becomes $\|\mathbf{e}_a\| \|\mathbf{e}_b\| \cos(\cdot)$ and the index starts ranking by *magnitude* as well as by direction. Long chunks, which tend to have larger norms, then win retrievals they do not deserve. The failure is quiet: nothing errors, recall just degrades, which is why `normalize_embeddings=True` in Listing 9.7 is load-bearing rather than decorative.

(c) Chunk **c** is the correct answer and ranks last, which is a first-stage retrieval failure. The component that exists to fix it is the **cross-encoder reranker** in Figure 9.20: because it scores the query and the chunk jointly rather than comparing two independently computed vectors, it can recognise a relevant passage whose bi-encoder embedding sits in the wrong neighbourhood, provided the passage is inside the candidate set. On the chunking side, the most likely cause is a chunk that mixes several topics, so its single embedding is an average that resembles nothing in particular. Splitting on clause boundaries with a smaller token cap, as in Section 9.14.2, gives each claim its own vector. Adding hybrid BM25 scoring would also help, since the exact statutory phrase probably appears verbatim in **c**.

Solution to Exercise 9.10. (a) Of 50 unanswerable questions, 45 are refused, so the correct-refusal rate is $45/50 = 0.90$. Of the 60 refusals, $60 - 45 = 15$ fell on answerable questions, so the over-refusal rate is $15/150 = 0.10$. The precision of refusals is $45/60 = 0.75$.

(b) Raising the retrieval score threshold means fewer chunks clear the bar, so the system refuses more often. The correct-refusal rate rises towards 1.0, the over-refusal rate also rises as genuinely answerable questions start being turned away, and refusal precision *falls*, because the extra refusals are disproportionately wrong ones. Faithfulness improves, and this is the trap: faithfulness is computed only over answers actually produced, so a system that answers less looks more faithful while helping less. That is precisely the degenerate solution Definition 9.6 exists to expose, and it is why the two refusal rates must be reported together with faithfulness rather than after it.

(c) The choice follows the asymmetric-cost reasoning of Chapter 3, applied to this use case rather than to the credit model. The costs are wildly unequal: an over-refusal wastes an analyst's time, perhaps twenty minutes of looking the clause up manually, whereas a confident hallucinated statement about Section 35 can propagate into a lending policy and an ODPC finding, with regulatory and reputational costs orders of magnitude larger. PesaCred should therefore operate at a *high-threshold*, high-refusal point, accepting an over-refusal rate around 0.10 to 0.15 in exchange for a correct-refusal rate above 0.95, and should make the refusal useful by returning

the top retrieved clauses alongside the refusal so the analyst starts from something rather than nothing. Two conditions make this defensible rather than lazy: the refusal must be visibly a refusal and never a hedged half-answer, and the over-refusal rate must be monitored as a service-level metric, because an assistant that refuses a third of real questions will simply be abandoned, which returns the organisation to unassisted manual search and helps nobody.

B.2 Chapter 10: MLOps, packaging, serving and CI/CD

Solution to Exercise 10.1. (a) By Proposition 10.1, $t^* = 800/(800 + 10,000) = 800/10,800 = 0.0741$.

(b) Expected cost per applicant is total cost divided by the number of applicants. At $t = 0.5$,

$$\frac{260 \times 10,000 + 150 \times 800}{10,000} = \frac{2,600,000 + 120,000}{10,000} = 272 \text{ KES,}$$

and at t^* ,

$$\frac{90 \times 10,000 + 1,900 \times 800}{10,000} = \frac{900,000 + 1,520,000}{10,000} = 242 \text{ KES.}$$

The cost-optimal threshold declines far more people, 1,900 good borrowers against 150, and is still KES 30 per applicant cheaper, because each avoided default saves as much as twelve and a half wrongful declines cost.

(c) The new threshold is $t^* = 1,500/11,500 = 0.1304$. Recomputing the two operating points of part (b) with $C_{FP} = 1,500$: at $t = 0.5$ the cost is $(2,600,000 + 150 \times 1,500)/10,000 = 282.5$ KES, while the *old* optimum $t = 0.074$ now costs $(900,000 + 1,900 \times 1,500)/10,000 = 375$ KES. The threshold that was optimal yesterday is now worse than the naive 0.5, and neither is the new optimum, which lies at 0.13 and requires a fresh confusion matrix to price. The lesson for versioning is direct: a change to the cost matrix is a change to the decision function of Definition 10.1, so it must produce a new model version, re-run the gates and pass through approval like any retraining. A cheap unit test enforces the link, asserting that the threshold stored in the metadata equals $C_{FP}/(C_{FP} + C_{FN})$ computed from the cost matrix stored beside it, so that nobody can edit one without the other.

Solution to Exercise 10.2. (a) By Proposition 10.2, the probability that none of the calls is slow is q^n , so the fraction hitting at least one component's p_{99} is $1 - 0.99^5 = 1 - 0.9510 = 4.9\%$ with five services, and $1 - 0.99^8 = 1 - 0.9227 = 7.7\%$ with eight.

(b) We need $q^5 \geq 0.99$, so $q \geq 0.99^{1/5} = 0.99799$. Each component must meet its target at roughly the **p_{99.8}** level, a much harder target than its own p_{99} . This is the quantitative reason why every added dependency in a request path is a latency liability even when it is individually fast.

(c) The 900 milliseconds exist precisely because of part (a). The components' budgets in Figure 10.11 are typical allocations, and with five dependencies roughly one request in twenty will see at least one of them run into its tail. Headroom absorbs that tail, allows one retry of a timed-out feature lookup (250 ms) without breaching the SLA, and tolerates the variability of a telco gateway that PesaCred does not control. A budget that sums exactly to the SLA guarantees violations on the tail, which on USSD means a borrower whose session simply drops.

Solution to Exercise 10.3. (a) The offered load is $a = \lambda \bar{s} = 120 \times 0.025 = 3.0$ workers' worth of work, so $c = \lceil 3.0/0.6 \rceil = 5$ workers, at utilisation $3/5 = 0.60$.

(b) At month-end, $a = 300 \times 0.025 = 7.5$, and $c = \lceil 7.5/0.6 \rceil = \lceil 12.5 \rceil = 13$ workers, at utilisation $7.5/13 = 0.58$.

(c) Thirteen copies of a 350 MB model need $13 \times 0.35 = 4.55$ GB for the models alone, before the Python interpreter, TensorFlow's own footprint and request buffers. This is the hidden cost of the process-per-worker design: memory scales with workers, and a larger model can make memory, not CPU, the binding constraint.

(d) By Little's law, $L = \lambda W = 300 \times 0.040 = 12$ requests in flight on average. Note that this exceeds the 7.5 requests actually being served (a), the difference of 4.5 being requests that are queued or in transit; that queue is what the utilisation ceiling keeps from growing without bound.

Solution to Exercise 10.4. (a) From $T(b)/b = 8/b + 0.15$: at $b = 1$, **8.15** ms per row; at $b = 32$, $0.25 + 0.15 = 0.40$ ms; at $b = 256$, $0.03125 + 0.15 = 0.181$ ms.

(b) Through the online endpoint, $3 \times 10^6 \times 8.15 \text{ ms} = 24,450 \text{ s}$, about **6.8** hours. Through the batch endpoint, $3 \times 10^6 \times 0.18125 \text{ ms} = 543.75 \text{ s}$, about **9.1** minutes, a speed-up of roughly **45** times. The difference decides whether the nightly refresh fits in its window, and it also frees the online workers of Exercise 10.3 for the daytime traffic they were sized for.

(c) USSD requests arrive one at a time, from independent borrowers. To form a batch the service would have to *wait* for other requests to arrive, and that waiting is queueing delay added to every borrower's response. At 120 requests per second, filling a batch of 32 takes about a quarter of a second, so the average request would wait more than a tenth of a second, most of the model's 150 ms budget, to save 7.75 ms of compute. Dynamic batching with a few milliseconds of waiting is sometimes worth it for GPU models with a large fixed cost, which is one argument for the model server of Section 10.3.2.4; for a small CPU model it is not.

Solution to Exercise 10.5. (a) Using $t = S/B$ with S in bits: $2.1 \times 8 \times 10^9 / 10^7 = 1,680$ s = **28** minutes, and $0.9 \times 8 \times 10^9 / 10^7 = 720$ s = **12** minutes.

(b) Only the changed application layer is pulled: $0.04 \times 8 \times 10^9 / 10^7 = 32$ seconds. Everything below it is already cached at the branch.

(c) With `COPY . .` placed before `pip install`, any change to any file in the repository invalidates the copy layer and therefore every layer above it, including the dependency layer holding TensorFlow. The release then ships the dependency layer as well as the code. If that layer is, say, 0.7 GB, the pull takes $0.7 \times 8 \times 10^9 / 10^7 = 560$ s, about 9.3 minutes instead of 32 seconds, on every release, for a change of perhaps one line. The same ordering mistake also slows every CI build, since the cache is invalidated there too.

Solution to Exercise 10.6. (a) Configuration A: mean 0.8080, standard deviation 0.00515, half-width $2.776 \times 0.00515 / \sqrt{5} = 0.0064$, so the 95% interval is [**0.8016, 0.8144**]. Configuration B: mean 0.8028, standard deviation 0.00814, half-width 0.0101, interval [**0.7927, 0.8129**].

(b) The claim compares an extreme of one sample with the opposite extreme of the other, which is the most optimistic comparison available and would “show” that almost any configuration beats almost any other. The honest comparison is between the means, and there A is ahead by 0.005. Even that difference is not established: the intervals overlap substantially, and a Welch t -test gives $p \approx 0.27$. The correct conclusion is that five seeds cannot distinguish the two configurations, and, if anything, the evidence leans the other way from the colleague’s claim. B’s larger standard deviation is also worth noticing, since a configuration that is less stable across seeds is less attractive for a pipeline that retrains weekly.

(c) Three changes. Run more seeds, since the half-width shrinks like $1/\sqrt{k}$ and twenty seeds halve it. Pair the seeds, using the same seed and data order for both configurations so that shared randomness cancels in the differences; with these numbers the paired t -statistic is about -1.41 , still well short of the critical 2.78, which confirms that no conclusion is warranted. And compare on the business metric, expected cost at t^* on the frozen holdout, rather than AUC, because that is the number the promotion gate will use.

Solution to Exercise 10.7. (a) With $n = 4,000$, the standard error is $310/\sqrt{4,000} = 4.90$ and the interval is $-9 \pm 1.96 \times 4.90 = [-18.6, 0.6]$. Its upper end is above zero, so the gate of Equation (10.9) **fails**, even though the challenger is in truth cheaper. With $n = 10,000$, the standard error is 3.10 and the interval is $[-15.1, -2.9]$, entirely below zero: the gate **passes**.

(b) The gate passes when $1.96 \times 310/\sqrt{n} < 9$, that is $\sqrt{n} > 67.5$, or $n > 4,557.8$. The smallest holdout is **4,558** applicants. This is a power calculation in disguise, and it should be done before the holdout is frozen: a holdout too small to detect the savings the business cares about produces

a gate that blocks good models at random.

(c) Most applicants receive the same decision from both models, so their paired difference is exactly zero, and the standard deviation of d_i is small. The cost of a single applicant, by contrast, varies enormously: with the t^* error rates of Exercise 10.1 (0.9% defaults approved at KES 10,000, 19% good borrowers declined at KES 800), its standard deviation is $\sqrt{0.009 \times 10,000^2 + 0.19 \times 800^2 - 242^2} \approx 981$ KES. An unpaired comparison of two independent means has standard error $\sqrt{2} \times 981 / \sqrt{n} \approx 1,388 / \sqrt{n}$, against $310 / \sqrt{n}$ paired, so it needs $(1,388/310)^2 \approx 20$ times as many applicants, about 91,000, for the same power. Pairing is not a refinement; it is what makes the gate feasible with the data PesaCred has.

Solution to Exercise 10.8. (a) Nairobi: $1,820/2,000 = 0.910$. North Eastern: $95/120 = 0.792$. The gap is $\Gamma = 0.910 - 0.792 = 0.118 > 0.10$, so on point estimates the gate **fails**.

(b) The 95% Wilson interval for Nairobi is **[0.897, 0.922]**; for North Eastern it is **[0.711, 0.855]**. The first is narrow because it rests on two thousand borrowers; the second is more than five times wider because it rests on 120.

(c) The intervals imply a plausible range for the gap from $0.897 - 0.855 = 0.042$, which would pass comfortably, to $0.922 - 0.711 = 0.211$, which would fail badly. A point-estimate gate therefore decides this case largely by chance. Which bound to use depends on who bears each error. A false pass deploys a model that systematically under-serves good borrowers in a region that is already under-banked, a harm that falls on borrowers, that DPA-motivated proxy-discrimination concerns make serious, and that is invisible in aggregate metrics. A false fail delays a better model, a cost borne by the lender. The asymmetry argues for a rule in two tiers: *block* promotion whenever the point estimate or the pessimistic bound exceeds τ , as here, and require a named risk officer to review the evidence and either investigate the region's data coverage or collect more outcomes before an exception is granted. Using the optimistic bound would let small, poorly served regions pass by default, which inverts the purpose of the gate.

Solution to Exercise 10.9. (a) $\hat{p} = 184/200 = 0.920$ and, from Equation (10.11) with $z = 1.96$, $LB = 0.874$, which clears $\tau = 0.85$: the gate **passes**.

(b) $\hat{p} = 176/200 = 0.880$ and $LB = 0.828$, below τ : the gate **fails**. The prompt edit cost eight golden cases, and the gate catches it even though the point estimate still looks respectable.

(c) $\hat{p} = 46/50 = 0.920$, identical to part (a), but $LB = 0.812$, which fails. The same observed quality fails on the smaller set because fifty cases cannot rule out a true pass rate in the low eighties. A golden set must be large enough that a system at the target quality can pass at all; with $\tau = 0.85$ and a system around 0.92, fifty cases are too few and two hundred are about the minimum. The quicker set would not speed the team up; it

would block every release.

Solution to Exercise 10.10. (a) The cost gate passes, since the upper end of $[-14, -2]$ is below zero. The fairness gate passes, $0.08 \leq 0.10$. The behavioural gate passes, $0.6\% \leq 1\%$. The holdout identity check passes. The latency gate **fails**: $162 > 150$ ms. Every gate must pass, so v5 **may not** become @champion, however good its accuracy. The latency budget is not a preference; it is derived from the USSD step time in Figure 10.11.

(b) Two families of options. Make the model faster: distil v5 into a smaller student network, prune or quantise it, or remove expensive features whose contribution is marginal, then re-run all gates, since each change creates a new version. Or change where the time goes: move any feature computation still done at request time into the nightly batch of Section 10.3.1, which frees budget for inference, or serve the model from a dedicated, faster model server (Section 10.3.2.4), at the price of one extra hop that must itself fit the budget. What is *not* an option is raising the gate's threshold on the engineering team's own authority; if the product owner decides to reallocate the headroom, the budget and the gate change together, as a recorded decision.

(c) Rollback is the alias move of Section 10.4.3: @champion is repointed to the version tagged previous_champion, v4, and the CD workflow rebuilds or redeploys the v4 image and routes the canary's traffic back to it. The records produced are the alias-change event in the registry, the GitHub Actions run with its approval, the deployment log, and the decision logs, in which every request carries its model_version, so the exact set of borrowers decided by v5 can be listed. Defaults cannot yet say whether v5 was wrong because loans made a week ago are nowhere near their due dates; outcomes arrive thirty or more days after disbursement. The falling approval rate is only a *proxy*. It could mean v5 is stricter than intended, or that the applicant population shifted, for example after salary day. The way to tell them apart is to compare v5's and v4's decisions on the same requests, which is exactly what a shadow deployment would have provided before the canary, and why shadow comes first for credit models.

Bibliography

- Akiba, T., Sano, S., Yanase, T., Ohta, T., and Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631.
- Arthur, D. and Vassilvitskii, S. (2007). *k*-means++: The advantages of careful seeding. In *Proceedings of the 18th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035.
- Ba, J. L., Kiros, J. R., and Hinton, G. E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *International Conference on Learning Representations (ICLR)*.
- Bantilan, N. (2024). pandera documentation. <https://pandera.readthedocs.io>.
- Baydin, A. G., Pearlmutter, B. A., Radul, A. A., and Siskind, J. M. (2018). Automatic differentiation in machine learning: A survey. *Journal of Machine Learning Research*, 18(153):1–43.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. (2011). Algorithms for hyper-parameter optimization. In *Advances in Neural Information Processing Systems*, volume 24.
- Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305.
- Beyer, K., Goldstein, J., Ramakrishnan, R., and Shaft, U. (1999). When is “nearest neighbor” meaningful? In *International Conference on Database Theory (ICDT)*, pages 217–235.
- Bishop, C. M. and Bishop, H. (2024). *Deep Learning: Foundations and Concepts*. Springer.

- Bjorkegren, D. and Grissen, D. (2020). Behavior revealed in mobile phone usage prediction credit repayment. *The World Bank Economic Review*, 34(3).
- Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations. *Journal of the Royal Statistical Society: Series B*, 26(2):211–243.
- Breck, E., Cai, S., Nielsen, E., Salib, M., and Sculley, D. (2017). The ML test score: A rubric for ML production readiness and technical debt reduction. In *IEEE International Conference on Big Data*.
- Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2):123–140.
- Breiman, L. (2001a). Random forests. *Machine Learning*, 45(1):5–32.
- Breiman, L. (2001b). Statistical modeling: The two cultures. *Statistical Science*, 16(3):199–231.
- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth, Belmont, CA.
- Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.
- Buolamwini, J. and Gebru, T. (2018). Gender shades: Intersectional accuracy disparities in commercial gender classification. In *Proceedings of the 1st Conference on Fairness, Accountability and Transparency (FAccT)*, pages 77–91.
- Campello, R. J. G. B., Moulavi, D., and Sander, J. (2013). Density-based clustering based on hierarchical density estimates. In *Advances in Knowledge Discovery and Data Mining (PAKDD)*, pages 160–172.
- Central Bank of Kenya (2022). Central bank of kenya (digital credit providers) regulations, 2022.
- Chawla, N. V., Bowyer, K. W., Hall, L., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proc. 22nd ACM SIGKDD*, pages 785–794.
- Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using rnn encoder–decoder for statistical machine translation. In *Proceedings of EMNLP*.

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314.
- Davis, J. and Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240.
- Dean, J. and Barroso, L. A. (2013). The tail at scale. *Communications of the ACM*, 56(2):74–80.
- Dempster, A. P., Laird, N. M., and Rubin, D. B. (1977). Maximum likelihood from incomplete data via the EM algorithm. *Journal of the Royal Statistical Society, Series B*, 39(1):1–38.
- Dettmers, T., Pagnoni, A., Holtzman, A., and Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Dietterich, T. G. (1998). Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.
- Docker Inc. (2024). Docker documentation. <https://docs.docker.com>.
- Elkan, C. (2001). The foundations of cost-sensitive learning. In *Proceedings of the 17th International Joint Conference Artificial Intelligence (IJCAI)*, pages 973–978.
- Es, S., James, J., Espinosa-Anke, L., and Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the European Chapter of the ACL (EACL): System Demonstrations*.
- Ester, M., Kriegel, H.-P., Sander, J., and Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 226–231.
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874.
- Freund, Y. and Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.
- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras and TensorFlow*. O’Reilly Media, 2nd edition.

- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. O'Reilly Media, 3rd edition.
- GitHub (2024). GitHub Actions documentation. <https://docs.github.com/actions>.
- Glorot, X. and Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256.
- Google Cloud (2020). MLOps: Continuous delivery and automation pipelines in machine learning. Cloud Architecture Center.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022a). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35.
- Grinsztajn, L., Oyallon, E., and Varoquaux, G. (2022b). Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*.
- Guo, C. and Berkhahn, F. (2016). Entity embeddings of categorical variables. *arXiv preprint arXiv:1604.06737*.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1321–1330.
- Halko, N., Martinsson, P.-G., and Tropp, J. A. (2011). Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review*, 53(2):217–288.
- He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 1026–1034.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778.
- Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8):1735–1780.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6):417–441.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- Huyen, C. (2022). *Designing Machine Learning Systems*. O'Reilly Media.
- Hyndman, R. J. and Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. OTexts, Melbourne, 3 edition.
- Ioffe, S. and Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, pages 448–456.
- Iterative (2024). DVC: Data version control documentation. <https://dvc.org/doc>.
- Jain, S. and Wallace, B. C. (2019). Attention is not explanation. In *Proceedings of NAACL-HLT*.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. (2020). Scaling laws for neural language models. arXiv:2001.08361.
- Kaufman, S., Rosset, S., Perlich, C., and Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM TKDD*, 6(4).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30.
- Keskar, N. S., Mudigere, D., Nocedal, J., Smelyanskiy, M., and Tang, P. T. P. (2017). On large-batch training for deep learning: Generalization gap and sharp minima. In *Proceedings of the 5th International Conference on Learning Representations (ICLR)*.
- Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Kreuzberger, D., Kühn, N., and Hirschl, S. (2023). Machine learning operations (MLOps): Overview, definition, and architecture. *IEEE Access*, 11:31866–31879.

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33.
- Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., and Talwalkar, A. (2017). Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52.
- Lim, B., Arık, S. Ö., Loeff, N., and Pfister, T. (2021). Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764.
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., and Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988.
- Liu, F. T., Ting, K. M., and Zhou, Z.-H. (2008). Isolation forest. In *Eighth IEEE International Conference on Data Mining*, pages 413–422.
- Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137.
- Loshchilov, I. and Hutter, F. (2019). Decoupled weight decay regularization. In *Proceedings of the 7th International Conference on Learning Representations (ICLR)*.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*.
- Melis, G., Dyer, C., and Blunsom, P. (2018). On the state of the art of evaluation in neural language models. In *International Conference on Learning Representations*.
- Micci-Barreca, D. (2001). A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems. *ACM SIGKDD Explorations Newsletter*, 3(1):27–32.
- Niculescu-Mizil, A. and Caruana, R. (2005). Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632.

- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):559–572.
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In *Advances in Large Margin Classifiers*, pages 61–74. MIT Press.
- Prince, S. J. D. (2024). *Understanding Deep Learning*. MIT Press.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., and Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In *Advances in Neural Information Processing Systems*, volume 31.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., and Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Ramírez, S. (2024). FastAPI documentation. <https://fastapi.tiangolo.com>.
- Republic of Kenya (2019). The data protection act, no. 24 of 2019. Government Printer, Nairobi.
- Ribeiro, M. T., Wu, T., Guestrin, C., and Singh, S. (2020). Beyond accuracy: Behavioral testing of NLP models with CheckList. In *Proceedings of the 58th Annual Meeting of the ACL*.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6):386–408.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65.
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3):581–592.
- Schölkopf, B., Smola, A., and Müller, K.-R. (1998). Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319.
- Schwarz, G. (1978). Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464.

- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems* 28.
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., and Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
- Siddiqi, N. (2017). *Intelligent Credit Scoring*. Wiley, 2nd edition.
- Snoek, J., Larochelle, H., and Adams, R. P. (2012). Practical Bayesian optimization of machine learning algorithms. In *Advances in Neural Information Processing Systems*, volume 25.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., and Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.
- Sundararajan, M., Taly, A., and Yan, Q. (2017). Axiomatic attribution for deep networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3319–3328.
- Tan, M. and Le, Q. V. (2019). Efficientnet: Rethinking model scaling for convolutional neural networks. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
- Tan, M. and Le, Q. V. (2021). Efficientnetv2: Smaller models and faster training. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
- van der Maaten, L. and Hinton, G. (2008). Visualizing data using *t*-SNE. *Journal of Machine Learning Research*, 9:2579–2605.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Wattenberg, M., Viégas, F., and Johnson, I. (2016). How to use *t*-SNE effectively. *Distill*.
- Wolpert, D. H. (1992). Stacked generalization. *Neural Networks*, 5(2):241–259.
- Yeo, I.-K. and Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.

- Yosinski, J., Clune, J., Bengio, Y., and Lipson, H. (2014). How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Zaharia, M., Chen, A., Davidson, A., Ghodsi, A., Hong, S. A., Konwinski, A., et al. (2018). Accelerating the machine learning lifecycle with MLflow. *IEEE Data Engineering Bulletin*, 41(4):39–45.